Maestría en Inteligencia de Negocios y Ciencia de Datos

Asignatura: Inteligencia Artificial

Proyecto Integral de Inteligencia Artificial

KuriBot: ChatBot inteligente con deep learning para la mesa de servicios de Geomática de una operación minera

Clasificación de intenciones con BETO ajustado, red bayesiana de credibilidad de alertas y motor de reglas, sobre un caso sintético del sur del Ecuador

Actividad grupal · Semana 4: Proyecto integral de inteligencia artificial aplicada a los negocios

Integrantes: Gabriel Ayala Sánchez
Daniel Castro M.
Ricardo Ruiz L.

Docente: Jorge Luis Zambrano Martínez, Ph.D.

Fecha: 13 de noviembre de 2026

Guayaquil, Ecuador

Datos: sintéticos, elaborados por el equipo para la empresa ficticia Minera del Austro S.A.; los códigos EPSG y la transformación PSAD56–WGS 84 provienen de la base geodésica EPSG.

Código completo en el anexo D y en https://github.com/gaboas2703/kuribot-chatbot-geomatica, con un cuaderno ejecutable para Google Colab.

Resumen

Este proyecto diseña, implementa y evalúa KuriBot, un ChatBot híbrido para la mesa de servicios de Geomática de una empresa minera ficticia del sur del Ecuador. El asistente reconoce 14 intenciones, entre ellas fichas de concesiones, verificación de puntos en el catastro, conversión de coordenadas PSAD56–WGS 84, reportes de posible minería ilegal, solicitudes de vuelo de dron y de cartografía, seguimiento de tickets, dudas geodésicas, derivación a personas y emergencias. Combina cuatro componentes: un clasificador de intenciones basado en BETO ajustado, un extractor de entidades por reglas, una red bayesiana de nueve nodos que estima la credibilidad de las alertas y un motor de reglas con encadenamiento hacia adelante que asigna prioridades, propone acciones y exige supervisión humana. Se compararon cuatro clasificadores sobre un corpus sintético de 744 expresiones, con validación cruzada agrupada por plantilla y dos conjuntos de prueba de 252 frases, uno limpio y otro con errores de tecleo. BETO ajustado obtuvo el mejor F1 macro en validación cruzada (0,848 frente a 0,762 de la línea base TF-IDF) y, tras una segunda iteración con aumento de datos, alcanzó 0,929 en la prueba limpia y 0,874 con ruido. La suite de 34 escenarios de aceptación pasó de 32 a 34 aprobados entre iteraciones, y ocho escenarios nuevos de confirmación aprobaron siete. El análisis ético y regulatorio se apoya en la Ley Orgánica de Protección de Datos Personales, la norma de la Superintendencia de Protección de Datos sobre sistemas de IA, el Reglamento europeo de IA, el marco del NIST y la recomendación de la UNESCO. Los resultados muestran factibilidad técnica; el valor operativo deberá medirse en un piloto controlado.

Palabras clave: ChatBot, clasificación de intenciones, BETO, red bayesiana, sistema basado en reglas, geomática, minería, IA responsable.

Índice

Resumen....1
Introducción....3
Caso de negocio....4
Diagnóstico....4
Actores y alcance....4
Métricas de éxito....4
Inteligencia artificial, machine learning y analítica tradicional....5
Escenario económico....5
Estado del arte....6
ChatBots en la empresa y en la industria....6
Clasificación de intenciones con modelos preentrenados....6
Razonamiento híbrido y minería ilegal....6
Metodología....7
Diseño, corpus y particiones....7
Modelos comparados....7
Protocolo de evaluación....7
Arquitectura del sistema....8
Modelado del componente inteligente....9
Clasificador de intenciones....9
Red bayesiana de credibilidad de alertas....9
Motor de reglas y umbrales de decisión....10
Resultados....11
Implementación....11
Desempeño del componente NLP....11
Iteración 2: ajustes del modelo....12
Desempeño por intención y calibración....13
Pruebas funcionales documentadas....14
Análisis de fallos y ajustes....15
Evaluación integral contra las métricas de éxito....15
Análisis crítico: ética, sesgos y regulación....16
Sesgos identificados....16
Marco regulatorio....17
Lineamientos de uso responsable....17
Limitaciones y trabajo futuro....17
Conclusiones....18
Referencias....19
Anexos....23
A Manual básico de usuario....23
B Ficha del modelo, reproducción y correspondencia con la rúbrica....27
C Evidencia de pruebas....27
D Código fuente completo....30

Introducción

En una operación minera, el área de Geomática funciona como una mesa de servicios silenciosa. Operaciones pregunta si un frente de trabajo sigue dentro del lindero, Seguridad reporta maquinaria desconocida cerca de una quebrada, Legal necesita el estado de una concesión antes de una inspección y Medio Ambiente pide un mapa para el informe del mes. Buena parte de esas consultas se repite y llega por canales informales; otra parte, menos frecuente, es delicada: una coordenada expresada en el datum equivocado puede ubicar un pozo a más de cuatrocientos metros de su posición real, y una alerta de minería ilegal mal priorizada puede exponer al personal de campo o dejar sin verificar una intrusión.

Este proyecto aborda ese problema con KuriBot (*kuri* significa «oro» en kichwa), un asistente conversacional para la mesa de servicios de Geomática de una empresa minera ficticia del sur del Ecuador. El caso se inspira en la práctica profesional de uno de los integrantes en geomática y catastro minero, pero todos los datos son sintéticos. KuriBot integra los contenidos del módulo: un clasificador de intenciones basado en una red neuronal *transformer* preentrenada en español y ajustada al dominio, un motor de reglas que hereda la lógica del sistema experto de priorización de alertas construido en la semana 2, una red bayesiana que estima la credibilidad de una alerta bajo incertidumbre (semana 3) y los criterios de diseño, evaluación y riesgo de los chatbots empresariales de la semana 4 (Zambrano Martínez, 2026).

La pregunta que guía el trabajo es práctica: ¿puede una arquitectura híbrida, que combina comprensión neuronal del lenguaje con razonamiento simbólico y probabilístico, resolver de forma confiable y trazable las consultas recurrentes de Geomática y priorizar alertas de campo sin delegar en la máquina las decisiones que corresponden a personas? El objetivo general es diseñar, implementar y evaluar un ChatBot funcional que combine técnicas de IA apropiadas al contexto minero ecuatoriano. Los objetivos específicos son:

1. Formalizar el caso de negocio y sus métricas de éxito, y justificar por qué exige IA y no solo analítica tradicional.
2. Comparar cuatro clasificadores de intenciones, uno de *machine learning* clásico y tres redes neuronales, sin fuga de información.
3. Implementar un motor de reglas y una red bayesiana que respalden y expliquen las decisiones del diálogo.
4. Construir el prototipo, documentar al menos veinte interacciones de prueba y evaluar sus implicaciones éticas y regulatorias.

El documento presenta primero el caso de negocio y el estado del arte; luego la metodología, la arquitectura y el modelado; después los resultados y su contraste con las métricas de éxito, y cierra con el análisis ético, las limitaciones y las conclusiones. Los anexos contienen el manual de usuario, la ficha del modelo, la evidencia de pruebas y el código completo.

Caso de negocio

Diagnóstico

Minera del Austro S.A. (nombre ficticio) es una empresa de mediana minería aurífera con seis concesiones propias en tres campamentos, colindante con la concesión de un tercero, un bosque protector y una comunidad. Su área de Geomática, con dos o tres analistas, atiende a cinco áreas internas. Para dimensionar el problema se plantea una línea base **hipotética**, que deberá medirse en un piloto: 240 solicitudes al mes, de las cuales cerca del 55 % son repetitivas (fichas de concesiones, conversiones de coordenadas, estado de pedidos y dudas geodésicas) y toman unos diez minutos cada una, y unas quince alertas mensuales de posible actividad minera no autorizada.

El diagnóstico identifica cinco fricciones. La **dependencia de pocas personas**: cuando el analista está en campo, las consultas simples esperan. La **falta de trazabilidad**: las solicitudes llegan por mensajería y llamadas, sin número de caso. El **riesgo geodésico**: en la zona coexisten coordenadas en PSAD56 y en WGS 84, y la transformación oficial de la base EPSG desplaza un punto unos 443 m en el área del proyecto, suficiente para ubicarlo en la concesión equivocada. La **priorización intuitiva de alertas**: dos reportes con la misma redacción pueden merecer respuestas distintas según los antecedentes del sector o los trabajos autorizados en el área. Y la **seguridad del personal**: una alerta con personas armadas no debe tratarse como una solicitud de vuelo más.

Actores y alcance

Los usuarios son colaboradores de Operaciones, Seguridad patrimonial, Legal y Catastro, Medio Ambiente y Relaciones Comunitarias; intervienen además el analista de Geomática, que recibe las derivaciones, el coordinador de vuelos, el supervisor de turno, que aprueba toda acción de campo, y el responsable de protección de datos. El prototipo reconoce 14 intenciones: saludo, despedida, ayuda, consultar una concesión, verificar un punto, convertir coordenadas, reportar una alerta, solicitar un vuelo de dron, solicitar cartografía, consultar una solicitud, resolver una duda geodésica, hablar con una persona, reportar una emergencia y mensajes fuera de alcance. Quedan excluidos de forma deliberada emitir certificados, modificar el catastro, autorizar vuelos o intervenciones (solo propone y registra), identificar personas, conectarse con la Agencia de Regulación y Control (ARCOM) o con servicios meteorológicos reales y dar asesoría legal.

Métricas de éxito

Las metas de la Tabla 1 se fijaron antes de evaluar los modelos y combinan métricas técnicas, de sistema y de negocio (Zambrano Martínez, 2026). Las de negocio quedan como objetivos de un piloto, porque un prototipo con datos sintéticos no puede demostrarlas.

Tabla 1.

Métricas de éxito definidas antes de la evaluación

Dimensión	Indicador	Meta
NLP	F1 macro en prueba limpia (frases redactadas aparte)	≥ 0,90
NLP	F1 macro en prueba con ruido ortográfico	≥ 0,85
NLP	Exhaustividad de la clase «fuera de alcance»	≥ 0,80
NLP	Error de calibración esperado (ECE) tras escalado de temperatura	≤ 0,05
NLP	Latencia del clasificador, percentil 95, en CPU	≤ 300 ms
Sistema	Escenarios de aceptación aprobados	≥ 90 %
Sistema	Emergencias con riesgo para personas derivadas a una persona	100 %
Sistema	Alertas en concesiones de terceros calificadas como intrusión	0
Sistema	Acciones de campo propuestas sin aprobación humana	0

Dimensión	Indicador	Meta
Sistema	Respuestas con datos inventados (tickets o concesiones inexistentes)	0
Sistema	Latencia de extremo a extremo, percentil 95	≤ 1 500 ms
Negocio (piloto)	Contención de consultas repetitivas; tiempo del reporte a la decisión; satisfacción (CSAT)	≥ 50 %; < 5 min; ≥ 4/5

Inteligencia artificial, *machine learning* y analítica tradicional

La IA busca comportamiento inteligente en máquinas: percibir, representar conocimiento, razonar, aprender y actuar (Russell y Norvig, 2021). El *machine learning* es el subconjunto que induce ese comportamiento a partir de datos: un programa aprende de la experiencia *E* respecto de una tarea *T* y una medida *P* si su rendimiento en *T*, medido por *P*, mejora con *E* (Mitchell, 1997). El *deep learning* es la familia del *machine learning* basada en redes con múltiples capas de representación (Goodfellow et al., 2016). La pregunta útil no es «¿IA o ML?», sino qué parte del problema conviene aprender y qué parte conviene programar (Zambrano Martínez, 2026).

En KuriBot solo un componente aprende de datos: el clasificador de intenciones, cuya tarea *T* es asignar a cada mensaje una de 14 intenciones, con la experiencia *E* de 744 expresiones etiquetadas y la medida *P* del F1 macro en frases nunca vistas. El resto es IA que no es *machine learning*: reglas explícitas, una red bayesiana con parámetros elicitados por expertos, una búsqueda de ventanas de vuelo y geoprocesos deterministas (Tabla 2). Una consulta SQL puede devolver el estado de la concesión CM-105, pero no entender que «¿cerro azul está activa?» pide exactamente eso, ni advertir que a unas coordenadas les falta el datum. Tampoco conviene resolver todo con *machine learning*: las alertas verificadas son pocas y no existe un histórico etiquetado para entrenar un modelo de priorización, y la seguridad del personal y los derechos de terceros no pueden quedar a merced de una probabilidad.

Tabla 2.

Diferenciación de enfoques y su papel en KuriBot

Enfoque	Qué aporta	Qué no puede hacer por sí solo	Uso en KuriBot
Analítica tradicional (SQL, SIG)	Consultas exactas sobre datos estructurados	Interpretar texto libre o razonar con evidencia incierta	Consultas al catastro una vez entendida la intención
Machine learning clásico	Patrones léxicos con pocos datos	Generalizar a paráfrasis y errores ortográficos	Línea base TF-IDF + regresión logística (M0)
Deep learning	Representaciones contextuales del lenguaje	Garantizar políticas, conocer hechos, explicar decisiones	BETO ajustado para clasificar intenciones (M3)
IA simbólica (reglas)	Decisiones auditables y restricciones de seguridad	Cubrir la variabilidad del lenguaje	Motor de reglas R01–R42 y reglas de vuelo V01–V06
IA probabilística	Combinar evidencias inciertas con conocimiento experto	Obtener probabilidades válidas sin datos ni elicitación	Red bayesiana de credibilidad de alertas

Escenario económico

Con la línea base hipotética, si el asistente resolviera sin intervención la mitad de las 132 consultas repetitivas mensuales, liberaría unas 11 horas de analista al mes (132 × 0,5 × 10 min); a un costo cargado supuesto de USD 10 por hora, el ahorro directo sería de unos USD 1 300 al año, modesto frente al costo de integrar y mantener el sistema. El argumento económico principal es otro: reducir los errores de datum en entregas con efectos legales y acortar el tiempo entre un reporte de campo y la decisión de verificarlo. Ninguno de esos beneficios puede cuantificarse con datos sintéticos, de modo que la decisión de inversión debe tomarse después de un piloto.

Estado del arte

ChatBots en la empresa y en la industria

Adamopoulou y Moussiades (2020) clasifican los chatbots por dominio, servicio, objetivo y método de respuesta (reglas, recuperación o generación) y muestran que su calidad depende tanto de la integración con fuentes de conocimiento como del modelo de lenguaje. Caldarini et al. (2022) documentan el paso hacia arquitecturas de *deep learning*, junto con la falta de criterios de evaluación comparables y el predominio de recursos en inglés, límites que este proyecto enfrenta con métricas explícitas y un corpus propio en español. Følstad et al. (2021) proponen una agenda centrada en el usuario, el impacto, la ética y la confianza, y Adam et al. (2021) advierten que las señales antropomórficas aumentan la complacencia del usuario, razón por la que KuriBot se presenta siempre como asistente automatizado. En la industria, Colabianchi et al. (2023) proponen una arquitectura de cinco módulos para agentes conversacionales industriales y Wellsandt et al. (2022) describen asistentes de mantenimiento con inteligencia híbrida aumentada, que acercan la información al operador sin retirarle la decisión. No se identificaron estudios sobre asistentes conversacionales para catastro minero o geomática en español; esa es la brecha aplicada que atiende este trabajo.

Clasificación de intenciones con modelos preentrenados

Larson et al. (2019) mostraron, con 150 intenciones, que los clasificadores aciertan dentro del dominio pero fallan con consultas fuera de alcance, y que entrenar con ejemplos de esa clase mejora su detección; por eso KuriBot tiene una clase «fuera de alcance» y un umbral de abstención. Casanueva et al. (2020) encontraron que los codificadores preentrenados detectan intenciones con pocas decenas de ejemplos por clase, el escenario de este proyecto. La arquitectura *transformer* (Vaswani et al., 2017) y el preentrenamiento bidireccional de BERT (Devlin et al., 2019) son la base de BETO, el BERT entrenado con un corpus grande en español (Cañete et al., 2020). Como alternativa liviana se evaluaron vectores fastText con información de subpalabras (Bojanowski et al., 2017) y una red LSTM (Hochreiter y Schmidhuber, 1997). Guo et al. (2017) demostraron que las redes modernas son sobreconfiadas y que un escalado de temperatura corrige gran parte del problema, algo indispensable cuando la confianza decide si el bot responde, pregunta o se abstiene.

Razonamiento híbrido y minería ilegal

Una red bayesiana representa una distribución conjunta mediante un grafo acíclico dirigido y tablas de probabilidad condicional, y permite inferencia exacta bajo incertidumbre (Koller y Friedman, 2009; Pearl, 1988). En minería, Mishra et al. (2021) combinaron juicio experto y datos de sitio en una red bayesiana de riesgo geotécnico, precedente directo de parámetros elicitados que se actualizan con datos. Garcez y Lamb (2023) describen la integración de aprendizaje y razonamiento como la tercera ola de la IA; KuriBot aplica esa idea de forma modesta: la red neuronal interpreta el lenguaje y los componentes simbólicos y probabilísticos deciden. En el Ecuador, Mestanza-Ramón et al. (2022) señalan la débil coordinación entre Estado, comunidades y mineros y la insuficiente fiscalización de la minería aurífera, y Chuizaca-Espinoza et al. (2026) documentaron con Sentinel-2 y *random forest* la expansión de la minería ilegal en el río Nangaritza. Ese tipo de alerta satelital es una de las evidencias que recibe la red bayesiana, con una condición: la detección remota orienta la verificación, no la sustituye.

Metodología

Diseño, corpus y particiones

El trabajo sigue un ciclo de ciencia del diseño adaptado de CRISP-DM y alineado con las cuatro fases de la actividad: negocio y arquitectura; datos y modelado; implementación y pruebas en dos iteraciones; evaluación, ética y defensa. Es un estudio de ingeniería con datos sintéticos, no un experimento con usuarios.

El equipo redactó 406 plantillas para las 14 intenciones, en español de uso laboral ecuatoriano y con registros formales e informales. Las plantillas con marcadores (concesión, coordenadas, ticket, producto, fecha) se instanciaron tres veces con valores del catastro sintético, lo que produjo 744 expresiones de entrenamiento y validación. Aparte, y con otra formulación, se redactaron 18 frases de prueba por intención (252). Para medir robustez, cada frase de prueba se perturbó con una semilla fija (minúsculas, sin tildes, abreviaturas de chat como «q» o «porfa» y un error de tecleo con probabilidad 0,7), lo que dio la **prueba con ruido** de 252 frases. Ninguna frase de prueba coincide con una de entrenamiento, y todas las variantes de una plantilla comparten un identificador de grupo para que la validación cruzada nunca las separe (StratifiedGroupKFold). El desbalance entre clases (de 24 a 90 ejemplos) se compensa con pesos de clase.

Antes de clasificar, cada mensaje pasa por dos pasos deterministas comunes a todos los modelos: el enmascarado de datos personales (cédulas con dígito verificador válido, teléfonos y correos) y el de entidades, que sustituye coordenadas, tickets y códigos por su tipo. Así el clasificador aprende la intención por el contexto y el extractor se encarga de los valores.

Modelos comparados

- **M0, TF-IDF + regresión logística:** unigramas y bigramas de palabras más n-gramas de caracteres de 2 a 5, con pesos balanceados; es la línea base de *machine learning* clásico.
- **M1, fastText + BiLSTM:** vectores de 300 dimensiones del modelo es_core_news_lg de spaCy (Honnibal et al., 2020), una LSTM bidireccional de 128 unidades, agregación por atención y por máximo, y un perceptrón multicapa.
- **M2, BETO congelado + MLP:** media estandarizada de la última capa de BETO y un perceptrón de 256 unidades ocultas.
- **M3, BETO ajustado:** el *transformer* completo se ajusta de extremo a extremo con una capa lineal de salida.

Los pesos de BETO se obtuvieron del pipeline oficial es_dep_news_trf 3.8.0 de spaCy, publicado en GitHub, porque el entorno de desarrollo no tenía acceso a otros repositorios de modelos. Ese pipeline contiene BETO ajustado por spaCy para análisis morfosintáctico, de modo que no es idéntico al original; la conversión se verificó contra la implementación de origen, con una diferencia máxima de $4{,}1 \times 10^{-6}$.

Protocolo de evaluación

Los modelos se comparan con validación cruzada de cinco pliegues estratificada por intención y agrupada por plantilla, con una parte de cada pliegue reservada para la parada temprana. Después, cada modelo se entrena con el 85 % del corpus y se valida con el 15 % restante, que también fija la temperatura de calibración y el umbral de confianza; ninguna decisión se toma con el conjunto de prueba. Se reportan exactitud con intervalo de Wilson, F1 macro con intervalo *bootstrap* de 2 000 remuestreos, métricas por clase, matriz de confusión, error de calibración esperado, latencia y parámetros entrenables, y las diferencias se contrastan con la prueba exacta de McNemar (Dietterich, 1998). Se usaron scikit-learn (Pedregosa et al., 2011), PyTorch (Paszke et al., 2019), Transformers (Wolf et al., 2020) y pgmpy

(Ankan y Panda, 2015), con semillas fijas y CPU de dos hilos; las tablas y figuras se generan desde los archivos de resultados, sin transcripción manual.

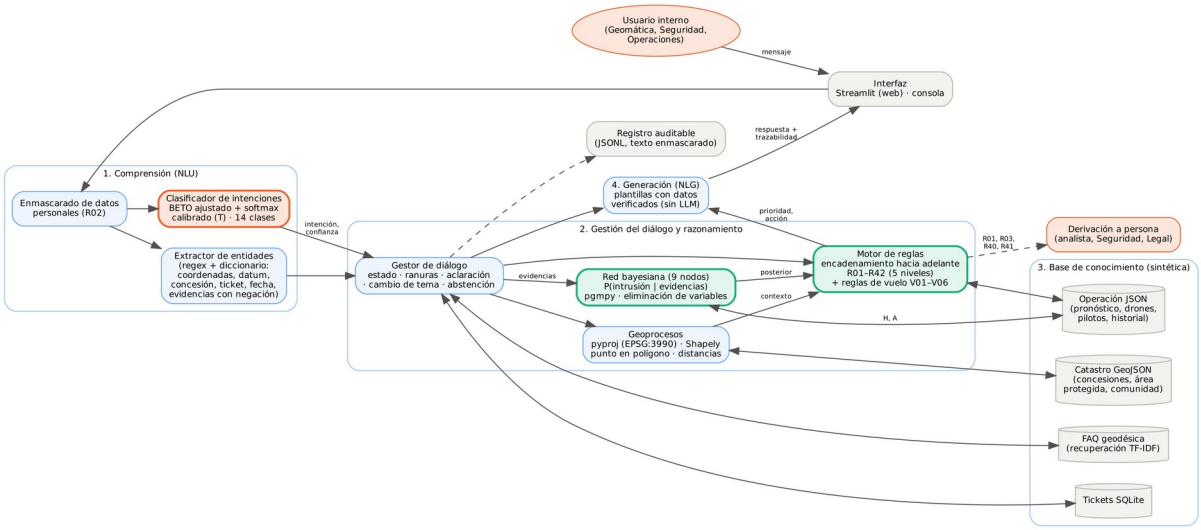
La evaluación funcional usa una suite de 34 escenarios (43 turnos) con criterios fijados antes de ejecutarla: intención esperada, acción o regla esperada, fragmentos obligatorios en la respuesta y latencia inferior a 1 500 ms. Se ejecuta en dos iteraciones, la primera para revelar fallos y la segunda para verificar los ajustes, y se complementa con 34 pruebas unitarias y con ocho escenarios de confirmación redactados al final y ejecutados una sola vez.

Arquitectura del sistema

La arquitectura sigue la separación clásica entre comprensión, gestión del diálogo y generación (Adamopoulou y Moussiades, 2020; Zambrano Martínez, 2026), con dos decisiones propias: la red neuronal solo decide la **intención**, nunca los hechos ni las acciones, y toda respuesta se construye con plantillas a partir de datos verificados, sin un modelo generativo que pueda inventar cifras (Figura 1).

Figura 1.

Arquitectura de componentes de KuriBot



Nota. Las flechas discontinuas indican registro y derivación a personas. Elaboración propia.

Cada mensaje sigue siete pasos: (1) se enmascaran los datos personales; (2) una red de seguridad busca señales de emergencia; (3) el extractor obtiene coordenadas, datum, concesión, ticket, fecha, producto y evidencias, con detección de negaciones; (4) si había una acción pendiente, se intenta completar la ranura que faltaba; (5) si no, el clasificador predice la intención y su confianza calibrada; (6) la política de la intención consulta el catastro, ejecuta geoprocesos e invoca la red bayesiana y el motor de reglas; (7) la respuesta se genera con plantillas y se acompaña de una traza auditable (intención, confianza, reglas, posterior bayesiano y latencia). La base de conocimiento es sintética: un catastro en GeoJSON (seis concesiones propias, una de un tercero, un bosque protector y una comunidad), un archivo de operación (pronóstico de cinco días, drones, pilotos, historial de alertas y actividades autorizadas), una FAQ geodésica de 19 entradas y una base SQLite de tickets.

Modelado del componente inteligente

Clasificador de intenciones

Para un mensaje con piezas $x_1, \ldots, x_n$, BETO produce estados contextuales $h_i \in R^{768}$; la representación del mensaje es su media y la distribución sobre las 14 intenciones se obtiene con una capa lineal y la temperatura T:

$$\acute{h} = \frac{1}{n}\sum_{i=1}^{n} h_i, \hat{p} = softmax\left(\frac{W\acute{h} + b}{T}\right) \quad (1)$$

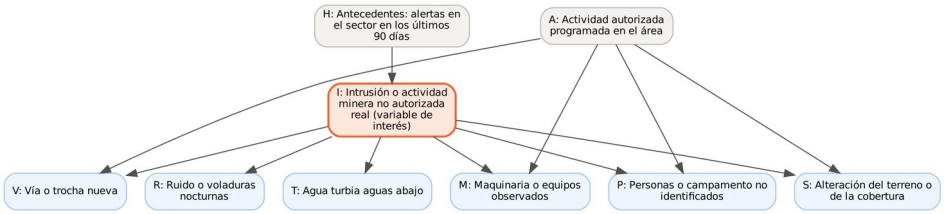
El ajuste minimiza la entropía cruzada ponderada por clase, con suavizado de etiquetas de 0,05, AdamW (tasa 3×10^{-5}), lotes de 16, hasta 48 piezas por mensaje y parada temprana sobre el F1 macro de validación. T se estima después del entrenamiento minimizando la log-verosimilitud negativa en validación (Guo et al., 2017). La decisión usa la confianza calibrada p_1 y la segunda probabilidad p_2, pues la probabilidad máxima del softmax es una base sencilla y eficaz para detectar errores y entradas ajenas al entrenamiento (Hendrycks y Gimpel, 2017): si $p_1 \geq$ máx(0,50; τ), el bot **acepta** la intención; si $p_1 \geq 0{,}30$ y $p_1 + p_2 \geq 0{,}70$, **pide aclaración** entre las dos opciones; en otro caso **se abstiene**, explica lo que puede hacer y, al segundo fallo seguido, ofrece una persona. El umbral τ se elige en validación.

Red bayesiana de credibilidad de alertas

El nodo de interés I representa una actividad minera no autorizada real (Figura 2). Su causa son los antecedentes del sector (H, alertas en los últimos 90 días). La actividad autorizada programada (A) es una causa alternativa de varias evidencias: maquinaria (M), personas o campamento (P), alteración del terreno (S) y vía nueva (V); el ruido nocturno (R) y la turbidez (T) dependen solo de I. H y A se observan en la base de conocimiento; las evidencias se extraen del mensaje, incluidas las negadas («no hay personas» fija $P = 0$), y las no mencionadas se marginalizan. Las tablas de probabilidad condicional de la Tabla 3 son juicios expertos del equipo, no frecuencias estimadas.

Figura 2.

Grafo de la red bayesiana de credibilidad de alertas



Nota. Elaboración propia, generado desde el código.

Tabla 3.

Tablas de probabilidad condicional elicitadas, P(X = 1 | padres)

Nodo	Padres	I=0, A=0	I=0, A=1	I=1, A=0	I=1, A=1
M: maquinaria	I, A	0,03	0,60	0,70	0,85
P: personas o campamento	I, A	0,10	0,50	0,75	0,85
S: alteración del terreno	I, A	0,05	0,45	0,65	0,80
V: vía o trocha nueva	I, A	0,03	0,20	0,40	0,45
R: ruido o voladuras nocturnas	I	0,05	—	0,40	—
T: agua turbia	I	0,15	—	0,40	—

Nota. Para el nodo de interés, P(I = 1 | H = 0) = 0,10 y P(I = 1 | H = 1) = 0,30. Las probabilidades a priori P(H = 1) = 0,30 y P(A = 1) = 0,40 solo se usan cuando H o A no se observan en la base de conocimiento.

La inferencia es exacta por eliminación de variables (pgmpy): para una evidencia e, el posterior normaliza la distribución conjunta sobre las variables no observadas z, y se verificó por enumeración completa con diferencias menores que 10^{-9}.

$$P(I=1\mid e)=\frac{\sum_{z} P(I=1,e,z)}{\sum_{i\in\{0,1\}}\sum_{z} P(I=i,e,z)},\quad P(x)=\prod_{k} P\left(x_k \mid pa\left(x_k\right)\right) \tag{2}$$

La estructura captura la explicación alternativa (*explaining away*): maquinaria y desbroce en Peña Blanca, sin antecedentes, dan un posterior de 0,219 porque hay un frente de explotación autorizado; la misma evidencia en un área sin actividad programada lo eleva a 0,971. Un análisis de sensibilidad (±0,10 en cada parámetro) mostró que el posterior depende mucho más del prior $P(I=1\mid H=0)$ (rango 0,02–0,39) que de cualquier verosimilitud (rangos menores de 0,08); por eso lo primero que debe calibrarse con verificaciones de campo es la tasa base de intrusiones por sector.

Motor de reglas y umbrales de decisión

El motor es un sistema de producción con encadenamiento hacia adelante: en cada ciclo arma la agenda con las reglas aplicables, dispara la de menor nivel y mayor prioridad, agrega sus conclusiones a la memoria de trabajo y no la repite (refracción). Las prioridades son excluyentes y una prueba automatizada comprueba que barajar el orden de las reglas no cambia la conclusión (Tabla 4). R12 y R20 impiden tratar como intrusión lo que ocurre en la concesión de un tercero, y R40 obliga a la revisión humana cerca de comunidades o áreas protegidas, criterios de equidad heredados de la semana 2.

Tabla 4.

Base de reglas del motor de decisiones (resumen)

Nivel	Reglas	Condición → conclusión
0. Seguridad	R01–R03	Emergencia con riesgo para personas → CRÍTICA, protocolo, sin despliegue y derivación humana; datos personales → enmascarar; pedido de una persona → derivar
1. Contexto geoespacial	R10–R15	Concesión propia; franja de 500 m; concesión de tercero; área protegida; comunidad a 1 km o menos; fuera del área
2. Priorización	R20–R25	Derecho de tercero → REVISIÓN LEGAL; P(I) ≥ 0,80 CRÍTICA; ≥ 0,50 ALTA; ≥ 0,15 MEDIA; resto BAJA; fuera del área → INFORMATIVA
3. Acción	R30–R36	Vuelo prioritario si hay ventana en 24 h, si no satélite y patrullaje; vuelo en 72 h; revisión satelital; registro; derivación a Legal
4. Salvaguardas	R40–R42	Zona sensible → revisión humana; CRÍTICA o ALTA → aprobación del supervisor; toda alerta → ticket
Vuelos	V01–V06	Viento > 10 m/s, lluvia, visibilidad < 5 km, sin dron, sin piloto o menos de un día de anticipación → no apto; búsqueda de la primera ventana apta

Nota. Los umbrales de 500 m, 1 km, 10 m/s y 5 km son supuestos de diseño del equipo.

El umbral inferior de acción se deriva de una razón de costos: si verificar con dron cuesta C y no detectar a tiempo una intrusión real cuesta L, conviene verificar cuando $P(I\mid e)L\geq C$, es decir, cuando $P(I\mid e)\geq C/L$. Con el supuesto $L/C\approx 6{,}7$, el umbral queda en 0,15, que separa BAJA de MEDIA; 0,50 y 0,80 escalan el plazo de verificación. Con costos asimétricos, la métrica y el umbral son decisiones de negocio (Zambrano Martínez, 2026).

El gestor de diálogo guarda la acción pendiente, sus ranuras, la última concesión mencionada y los fallos seguidos. Si falta un dato (concesión, coordenadas, datum, fecha, producto o evidencias), lo pide y

completa la acción cuando llega; si llega otra intención con confianza suficiente, abandona la pendiente; tras dos intentos fallidos ofrece una persona, y «cancelar» anula la acción en cualquier momento.

Resultados

Implementación

KuriBot está escrito en Python y organizado en módulos con responsabilidades separadas: entidades, geodesia, base de conocimiento, modelos e inferencia de intenciones, razonamiento y gestor de diálogo (Anexo D). Tiene dos interfaces sobre el mismo motor. La web, construida con Streamlit, muestra la conversación y, bajo cada respuesta, un panel de trazabilidad con la intención, las probabilidades, las entidades, las reglas disparadas, el posterior bayesiano, las ventanas de vuelo y un mapa del catastro; la de consola imprime la misma información en una línea técnica (Anexo A). El modelo de producción ocupa 219 MB en media precisión y corre en CPU sin servicios externos; si faltan sus pesos, la aplicación usa la línea base TF-IDF y lo advierte.

Desempeño del componente NLP

En la validación cruzada agrupada por plantilla, la prueba más exigente porque cada pliegue evalúa formulaciones nuevas, BETO ajustado (M3) superó con claridad a los demás: F1 macro de 0,848 frente a 0,762 de la línea base (Tabla 5 y Figura 3). En la prueba limpia, en cambio, M3 y la línea base TF-IDF quedaron prácticamente empatados (0,917 y 0,922; McNemar p = 0,86), y con ruido ortográfico la línea base fue mejor (0,881 frente a 0,805; p = 0,005): los n-gramas de caracteres toleran una letra cambiada, mientras que BETO divide una palabra con errata en piezas que nunca vio juntas. La BiLSTM con fastText (cobertura del vocabulario de 96,9 %) perdió 13 puntos con ruido porque trabaja con palabras completas, y BETO congelado fue el peor (0,764), coherente con que sus pesos se ajustaron en spaCy para sintaxis y no para semántica.

Tabla 5.

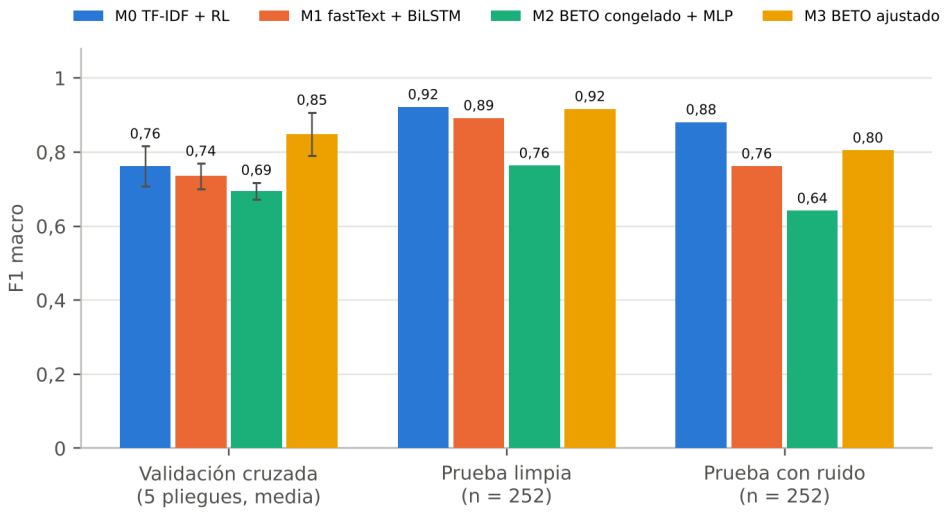
Comparación de modelos de intenciones, iteración 1

Modelo	F1 macro VC (media ± DE)	F1 prueba limpia [IC 95 %]	F1 prueba con ruido [IC 95 %]	ECE calibrado (limpia)	Parámetros entrenables	Latencia p95 (ms)
M0 TF-IDF + RL	0,762 ± 0,054	0,922 [0,89–0,95]	0,881 [0,84–0,92]	0,037	109 886	1,3
M1 fastText + BiLSTM	0,735 ± 0,034	0,892 [0,85–0,93]	0,762 [0,70–0,81]	0,057	683 547	2,3
M2 BETO congelado + MLP	0,695 ± 0,023	0,764 [0,70–0,81]	0,642 [0,58–0,70]	0,048	200 462	52,5
M3 BETO ajustado (fine-tuning)	0,848 ± 0,059	0,917 [0,88–0,95]	0,805 [0,75–0,85]	0,048	109 271 054	45,2

Nota. VC = validación cruzada de cinco pliegues agrupada por plantilla; IC = intervalo *bootstrap* de 2 000 remuestreos; la latencia es por mensaje en CPU de dos hilos.

Figura 3.

F1 macro de los cuatro modelos en validación cruzada y en los conjuntos de prueba, iteración 1



Nota. Las barras de error muestran una desviación estándar entre pliegues.

La decisión no podía tomarse solo por el F1 en limpio. M3 generalizó mejor a formulaciones nuevas, detectó mucho mejor los mensajes fuera de alcance (exhaustividad de 0,833 frente a 0,611 en la prueba limpia), que es la clase donde un error produce una respuesta impropia (Larson et al., 2019), y su debilidad, los errores de tecleo, tenía un remedio conocido. Por eso se eligió como modelo de producción y se planteó una segunda iteración para mejorar su robustez.

Iteración 2: ajustes del modelo

Casi todos los errores de M3 con ruido ocurrían en palabras con errata o sin tilde («agenda drrone para San Gregorio» se clasificó como consulta de concesión). Se aplicaron tres ajustes, todos con datos de entrenamiento y validación: (1) una copia perturbada de cada frase de entrenamiento, con una semilla distinta de la de prueba, siguiendo las técnicas de aumento simple para texto (Wei y Zou, 2019); (2) ocho plantillas nuevas de alertas con coordenadas, motivadas por el fallo A17 de la suite de aceptación; y (3) hasta diez épocas con paciencia de tres, porque el F1 de validación seguía subiendo en la sexta.

Tabla 6.

Efecto de los ajustes de la iteración 2 sobre la línea base y el modelo de producción

Versión	Modelo	F1 limpia	F1 ruido	Exactitud limpia	ECE calibrado	Épocas entrenadas	τ
v1	M0 TF-IDF + RL	0,922	0,881	0,925	0,037	—	0,20
v1	M3 BETO ajustado (fine-tuning)	0,917	0,805	0,917	0,048	6	0,20
v2	M0 TF-IDF + RL	0,906	0,861	0,909	0,036	—	0,35
v2	M3 BETO ajustado (fine-tuning)	0,929	0,874	0,929	0,042	7	0,40

Nota. v1: corpus original y hasta 6 épocas; v2: plantillas nuevas, una copia con ruido por frase y hasta 10 épocas. τ es el umbral elegido en validación; el bot acepta con confianza ≥ máx(0,50; τ).

El aumento benefició al *transformer* y no a la línea base (Tabla 6). M3 pasó de 0,805 a 0,874 con ruido, una mejora significativa (25 aciertos nuevos frente a 9 perdidos; McNemar p = 0,009), y de 0,917 a 0,929 en limpio, mientras M0 bajó levemente en ambos conjuntos. Frente a la mejor línea base de la iteración 1, el modelo final ya no difiere de forma significativa (p = 1,00 en limpio y p = 0,74 con ruido), pero conserva la ventaja en detección fuera de alcance (0,833 y 0,778 de exhaustividad frente a 0,556 y

0,444 de M0). La parada temprana se activó en la séptima época, con el mejor F1 de validación en la cuarta.

Desempeño por intención y calibración

Las clases más difíciles son «fuera de alcance» (F1 de 0,769), que agrupa temas heterogéneos, y «solicitar vuelo» (0,848), que se confunde con cartografía cuando se pide una ortofoto, que en efecto exige volar (Tabla 7). La matriz de confusión (Figura 4) muestra que los errores se concentran en pares semánticamente cercanos: verificar frente a convertir coordenadas, y cartografía frente a vuelo o concesión.

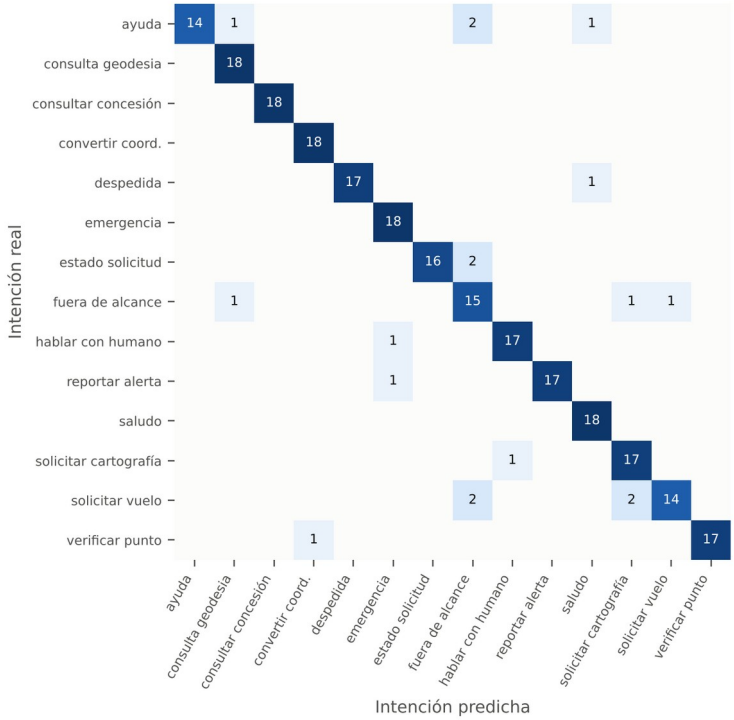
Tabla 7.

Precisión, exhaustividad y F1 por intención del modelo de producción (M3, iteración 2)

Intención	Precisión	Exhaustividad	F1 (limpia)	F1 (ruido)	Soporte
ayuda capacidades	1,000	0,778	0,875	0,824	18
consulta geodesia	0,900	1,000	0,947	0,865	18
consultar concesion	1,000	1,000	1,000	0,872	18
convertir coordenadas	0,947	1,000	0,973	0,947	18
despedida	1,000	0,944	0,971	0,941	18
emergencia seguridad	0,900	1,000	0,947	0,857	18
estado solicitud	1,000	0,889	0,941	0,941	18
fuera alcance	0,714	0,833	0,769	0,718	18
hablar humano	0,944	0,944	0,944	0,857	18
reportar alerta	1,000	0,944	0,971	0,944	18
saludo	0,900	1,000	0,947	0,947	18
solicitar cartografia	0,850	0,944	0,895	0,789	18
solicitar vuelo	0,933	0,778	0,848	0,788	18
verificar punto	1,000	0,944	0,971	0,941	18
Promedio macro	**0,935**	**0,929**	**0,929**	**0,874**	252

Figura 4.

Matriz de confusión del modelo de producción en la prueba limpia (n = 252)



El escalado de temperatura (T = 0,81) redujo el error de calibración esperado de 0,066 a 0,042 en la prueba limpia y de 0,085 a 0,058 con ruido. Con el umbral elegido en validación (τ = 0,40), el bot acepta el 98,4 % de los mensajes con 93,1 % de exactitud; exigir 0,80 elevaría esa exactitud a 98,2 % a costa de pedir aclaración en el 12 % de los casos. Se optó por un umbral de aceptación de 0,50 y por la aclaración entre las dos intenciones más probables, porque preguntar es barato y equivocarse en una alerta no lo es. La latencia del clasificador fue de 53,6 ms en el percentil 95.

Pruebas funcionales documentadas

La suite de aceptación ejecuta 34 escenarios (43 turnos) que cubren las 14 intenciones, el llenado de ranuras en dos y tres turnos, los geoprocesos, las seis ramas de priorización de alertas, la emergencia, la protección de datos, los mensajes ambiguos y fuera de alcance, y la regla de no inventar datos; el Anexo C documenta cada escenario con su entrada, la decisión del sistema y su resultado en ambas iteraciones. La primera iteración aprobó 32/34 escenarios (94,1 %) y la segunda 34/34 (100,0 %), con una latencia de extremo a extremo de 38,6 ms en promedio y 50,5 ms en el percentil 95.

Los escenarios de alertas muestran el razonamiento completo (Tabla 8). La misma combinación de maquinaria y desbroce produce prioridad MEDIA en Peña Blanca, donde hay un frente de explotación autorizado (A15), mientras que maquinaria y personas en Cerro Azul, suspendida y con tres alertas recientes, llevan el posterior a 0,987 y a prioridad CRÍTICA (A14). Cuando el reporte niega una evidencia («pero no hay personas», A19), el posterior cae a 0,045, pero R40 exige revisión humana porque La Ventana colinda con el bosque protector y la comunidad. En A17 el posterior es 0,359, pero el punto está en la concesión de un tercero y R20 lo deriva a Legal sin calificarlo como intrusión.

Tabla 8.

Decisiones del componente de razonamiento en los escenarios de alerta (iteración 2)

ID	Reporte	P(I | e)	Prioridad	Reglas disparadas
A14	hay una retroexcavadora y gente extraña trabajando en Cerro	0,987	CRÍTICA	R10, R21, R30, R41, R42
A15	se ve desbroce nuevo y maquinaria en Peña Blanca	0,219	MEDIA	R10, R23, R33, R42
A16	el agua baja turbia	0,533	ALTA	R10, R13, R22, R32, R40, R41, R42
A17	vimos maquinaria en E 642000 N 9663600 WGS84	0,359	REVISIÓN LEGAL	R12, R20, R35, R42
A18	reporto voladuras de noche y una trocha nueva en El Roble	0,979	CRÍTICA	R10, R21, R30, R41, R42
A19	hay maquinaria en La Ventana, pero no hay personas	0,045	BAJA	R10, R13, R14, R24, R34, R40, R42

Nota. P(I | e) = probabilidad posterior de actividad no autorizada dada la evidencia, calculada con la red bayesiana.

Análisis de fallos y ajustes

La primera ejecución falló en dos escenarios. En **A17** («vimos maquinaria en E 642000 N 9663600 WGS84») el modelo predijo «verificar punto»: el corpus original no tenía alertas con coordenadas y la red aprendió una correlación espuria entre el marcador de coordenadas y la verificación de puntos; se agregaron ocho plantillas de alertas con coordenadas. En **A31** («mi cédula es [CÉDULA], ¿cuál es el estado de CM-104?») el modelo confundió la consulta con el estado de un ticket, porque el marcador de dato personal no existía en el entrenamiento; se retiraron esos marcadores del texto que recibe el clasificador, ya que no aportan información sobre la intención. Ambos pasaron en la iteración 2 sin afectar a los demás. Un tercer problema apareció al reproducir el proyecto en un directorio limpio: el primer turno tardó una vez 2,2 s por el arranque en frío de PyTorch y A01 falló por latencia; el clasificador ahora ejecuta una inferencia de calentamiento al cargarse, y la suite repetida con ese cambio no varió ninguna intención, regla ni respuesta.

Ajustar el sistema hasta que la suite pase no garantiza que mejore de verdad. Para controlarlo, después de cerrar la iteración 2 se redactaron ocho escenarios de confirmación nuevos (C01–C08), ejecutados una sola vez y sin ajustes posteriores: aprobaron siete (87,5 %). El fallo, C07 («¿va a llover mañana en Machala?»), se clasificó como solicitud de vuelo por la presencia de clima y fecha; la respuesta fue inocua, porque el bot preguntó por la concesión del vuelo, pero confirma que la frontera entre el clima como condición de vuelo y el clima como charla no está bien aprendida. Las 34 pruebas unitarias (Anexo C) verifican, entre otras cosas, la ida y vuelta PSAD56–WGS 84 con error inferior a 1 cm, la coincidencia entre pgmpy y la enumeración exacta, la prevalencia de la regla de emergencia, el determinismo del motor y la negativa a inventar tickets.

Evaluación integral contra las métricas de éxito

El sistema cumplió todas las metas de sistema y las cinco técnicas en la prueba limpia (Tabla 9). Con ruido ortográfico, el F1 macro superó su meta, pero la exhaustividad fuera de alcance (0,778) y el error de calibración (0,058) quedaron ligeramente por debajo de lo exigido. Las métricas de negocio no pueden evaluarse con datos sintéticos.

Tabla 9.

Evaluación del sistema frente a las métricas de éxito

Indicador	Meta	Resultado	Estado
F1 macro, prueba limpia	≥ 0,90	0,929 [IC 0,89–0,96]	Cumplida
F1 macro, prueba con ruido	≥ 0,85	0,874 [IC 0,83–0,91]	Cumplida tras la iteración 2

Indicador	Meta	Resultado	Estado
Exhaustividad fuera de alcance	≥ 0,80	0,833 limpia; 0,778 con ruido	Cumplida solo en limpia
ECE calibrado	≤ 0,05	0,042 limpia; 0,058 con ruido	Cumplida solo en limpia
Latencia del clasificador, p95	≤ 300 ms	53,6 ms	Cumplida
Escenarios de aceptación	≥ 90 %	100,0 % (iteración 2); 87,5 % en confirmación	Cumplida; confirmación levemente por debajo
Emergencias derivadas a una persona	100 %	2 de 2 escenarios (A20, C03) y prueba unitaria	Cumplida
Alertas en terceros calificadas como intrusión	0	0 (A17 y prueba unitaria)	Cumplida
Acciones de campo sin aprobación humana	0	0: R41 se dispara en toda prioridad CRÍTICA o ALTA	Cumplida
Datos inventados	0	0: tickets y concesiones inexistentes se declaran no encontrados	Cumplida
Latencia de extremo a extremo, p95	≤ 1 500 ms	50,5 ms	Cumplida
Contención, tiempo a decisión, CSAT	Ver Tabla 1	No medibles con datos sintéticos	Pendiente de piloto

Dos resultados merecen leerse con cautela. La ventaja del *deep learning* sobre una buena línea base de n-gramas es real en generalización y en detección fuera de alcance, pero no en todos los conjuntos: en la prueba limpia ambos modelos rinden igual. Y la suite fue escrita por el mismo equipo que construyó el sistema; la confirmación con escenarios nuevos (87,5 %) es una estimación más honesta del desempeño ante consultas no previstas.

Análisis crítico: ética, sesgos y regulación

Un asistente que ayuda a vigilar concesiones mineras no es éticamente neutro: opera sobre territorios donde conviven la empresa, comunidades, mineros artesanales e informales y, a veces, grupos armados, en un contexto de fiscalización débil (Mestanza-Ramón et al., 2022). Tres implicaciones orientaron el diseño.

Del lugar a la persona. La red bayesiana evalúa alertas sobre *lugares*, no sobre individuos; el sistema no guarda nombres ni rostros y enmascara cédulas, teléfonos y correos antes de clasificar y registrar (R02). Aun así, la palabra «intrusión» puede estigmatizar a población vulnerable; por eso R12 y R20 impiden calificar como intrusión lo que ocurre en la concesión de un tercero, y R40 exige revisión humana y protocolo comunitario cerca de comunidades o áreas protegidas.

Seguridad y sesgo de automatización. Una probabilidad de 98 % dicha con fluidez se lee como certeza. KuriBot aclara que es una estimación con supuestos expertos, muestra sus evidencias y no ejecuta nada: propone y exige la aprobación del supervisor (R41). Ante señales de violencia, R01 prevalece sobre cualquier probabilidad. Como las señales humanizadoras aumentan la complacencia del usuario (Adam et al., 2021), el asistente se identifica como automatizado desde el primer mensaje.

Rendición de cuentas. La responsabilidad no se transfiere al modelo: cada respuesta conserva su traza y el Anexo B documenta el clasificador con una ficha de modelo (Mitchell et al., 2019).

Sesgos identificados

1. **Autoría y selección.** Tres personas redactaron el corpus en español escrito; no representa el kichwa ni el shuar, la voz ni la baja alfabetización digital. El F1 macro cae de 0,929 en texto limpio a 0,874 con errores de tecleo: quien escribe de forma no normativa recibe peor servicio (Mehrabi et al., 2021).

2. **Medición.** «Fuera de alcance» agrupa temas heterogéneos y el léxico de emergencias no capta amenazas implícitas; se prefirió errar por falsa alarma y no por omisión.
3. **Priores y retroalimentación.** $P(I \mid H)$ hace más sospechosos a los sectores con antecedentes. Si H se alimenta con alertas y no con verificaciones, se forma un ciclo (más alertas, más vuelos, más detecciones) que puede estigmatizar sectores; la mitigación es recalibrar H solo con casos verificados y auditar las falsas alarmas por sector.
4. **Modelo base y evaluación.** La versión de BETO usada fue ajustada con texto periodístico y subrepresenta la terminología minera; además, el mismo equipo escribió el sistema y sus criterios de aceptación.

Marco regulatorio

El Ecuador no tiene una ley general de IA: la Asamblea archivó en primer debate el proyecto de Ley Orgánica de Regulación y Promoción de la IA el 15 de septiembre de 2026 (Asamblea Nacional del Ecuador, 2026). Rigen la Ley Orgánica de Protección de Datos Personales (Asamblea Nacional del Ecuador, 2021) y la norma de la Superintendencia de Protección de Datos para sistemas de IA, que pide gestionar riesgos antes del despliegue, informar el carácter automatizado del tratamiento y asegurar revisión humana de decisiones con efectos significativos (Superintendencia de Protección de Datos Personales [SPDP], 2026a); su reforma de 2026 delimita las obligaciones de los encargados y exige evaluación de impacto cuando corresponda (SPDP, 2026b). Un despliegue real registraría datos de empleados y, quizá, de terceros, por lo que esa evaluación sería prudente.

Como referencia, el Reglamento europeo de IA clasifica los sistemas por riesgo y, desde el 2 de agosto de 2026, obliga a informar a quien interactúa con una IA (Parlamento Europeo y Consejo de la Unión Europea, 2024); el Ómnibus digital aplazó las obligaciones de alto riesgo, no esa transparencia (Parlamento Europeo y Consejo de la Unión Europea, 2026). El uso previsto de KuriBot, apoyo interno con decisión humana, no es de alto riesgo, clasificación que habría que revisar si sus salidas llegaran a autoridades. El marco del NIST (National Institute of Standards and Technology [NIST], 2023) y la recomendación de la UNESCO (UNESCO, 2022) convergen con los principios más repetidos en las guías éticas: transparencia, justicia, no maleficencia, responsabilidad y privacidad (Jobin et al., 2019).

Lineamientos de uso responsable

1. **Propósito limitado:** consultas internas y triaje de alertas sobre lugares; prohibido identificar o perfilar personas.
2. **Transparencia y decisión humana:** aviso de IA en cada sesión, prioridad explicada y aprobación humana de toda acción de campo.
3. **Datos mínimos:** no pedir cédulas ni teléfonos; enmascarar, retener lo indispensable y eliminar con plazo.
4. **Seguridad primero:** ante riesgo para personas, protocolo y ECU 911; nunca enviar equipos a ciegas.
5. **Calibración, auditoría y trazabilidad:** recalibrar la red con verificaciones de campo, revisar falsas alarmas por sector cada trimestre y registrar versión, reglas y responsable de cada decisión.

Limitaciones y trabajo futuro

Los resultados tienen límites claros. El corpus, el catastro y la operación son sintéticos, y no se midieron usuarios, contención real ni impacto económico. Con 18 frases de prueba por intención, un solo

error cambia el F1 de una clase en varios puntos, y las frases de prueba se redactaron aparte, pero por las mismas personas. BETO sigue siendo sensible a los errores de tecleo: el aumento de datos cerró casi toda la brecha con la mejor línea base de n-gramas (0,874 frente a 0,881, diferencia no significativa), pero la caída respecto del texto limpio persiste. Se usó la versión de BETO incluida en spaCy, distribuida con licencia GPL-3.0, sin compararla con el BETO original ni con modelos multilingües. Las tablas de la red bayesiana son juicios expertos y la independencia condicional de las evidencias dado I y A puede no cumplirse (maquinaria y vía nueva suelen aparecer juntas). El extractor reconoce los formatos de coordenadas previstos, pero no todos, y el diálogo no maneja varias intenciones en un mismo mensaje. La FAQ se recupera por similitud de n-gramas de caracteres: en una prueba manual posterior a la evaluación, «¿qué es el datum PSAD56?» devolvió la definición general de datum y no la ficha de PSAD56, porque la consulta comparte más fragmentos con la pregunta general. Por último, no hay autenticación ni integración con el catastro oficial o servicios meteorológicos.

El siguiente paso válido es un piloto de cuatro a seis semanas con un grupo pequeño de usuarios internos, conversaciones registradas con consentimiento y seudonimización, y medición de contención, tiempo de primera respuesta y satisfacción. Con esos datos conviene reentrenar el clasificador, comparar el BETO original, modelos multilingües y ensambles con la línea base de caracteres, estimar las tablas de la red con el histórico de verificaciones (por ejemplo, con priors de Dirichlet que conserven el juicio experto) y contrastar su estructura con pruebas de independencia condicional. En ingeniería quedan la integración con el catastro oficial y un servicio meteorológico, la autenticación por roles, la recuperación semántica de la FAQ y la ingesta automática de alertas satelitales, que se conectaría con el sistema de monitoreo en ArcGIS propuesto en la práctica profesional del equipo.

Conclusiones

KuriBot muestra que la mesa de servicios de Geomática de una operación minera puede abordarse como un sistema de IA y no como un clasificador aislado. La red neuronal interpreta un lenguaje variable y lleno de códigos; el extractor convierte el mensaje en datos; la red bayesiana combina evidencias inciertas con conocimiento experto y explica por qué una misma observación merece prioridades distintas según el contexto; y el motor de reglas garantiza lo que no puede quedar a merced de una probabilidad: la seguridad del personal, los derechos de terceros y la supervisión humana. De la evaluación se desprenden tres conclusiones:

- El modelo neuronal ajustado superó a la línea base clásica cuando debió generalizar a formulaciones nuevas (F1 macro de 0,848 frente a 0,762 en validación cruzada) y al detectar consultas fuera de alcance, pero no en todos los conjuntos: con texto limpio ambos rindieron igual. La sofisticación de un modelo no garantiza su superioridad; por eso la elección se sustentó en pruebas estadísticas, en el tipo de errores y en un ajuste dirigido que llevó el F1 con ruido de 0,805 a 0,874.
- El sistema completo aprobó los 34 escenarios de aceptación después de corregir dos fallos con causa identificada, respondió en menos de 60 ms en el percentil 95 y en ningún caso inventó datos, calificó como intrusión lo ocurrido en una concesión de terceros ni propuso acciones de campo sin aprobación humana; con escenarios nuevos aprobó siete de ocho.
- La conclusión empresarial es condicionada: hay factibilidad técnica y un diseño de gobernanza razonable, pero el valor real depende de la contención que se logre con usuarios reales, de la calidad del catastro integrado y de calibrar la red con verificaciones de campo. No se recomienda pasar a producción sin un piloto controlado.

Referencias

Adam, M., Wessel, M., & Benlian, A. (2021). AI-based chatbots in customer service and their effects on user compliance. *Electronic Markets, 31*(2), 427–445. https://doi.org/10.1007/s12525-020-00414-7

Adamopoulou, E., & Moussiades, L. (2020). Chatbots: History, technology, and applications. *Machine Learning with Applications, 2*, Artículo 100006. https://doi.org/10.1016/j.mlwa.2020.100006

Ankan, A., & Panda, A. (2015). pgmpy: Probabilistic graphical models using Python. En K. Huff & J. Bergstra (Eds.), *Proceedings of the 14th Python in Science Conference* (SciPy 2015). https://doi.org/10.25080/Majora-7b98e3ed-001

Asamblea Nacional del Ecuador. (2021, 26 de mayo). *Ley Orgánica de Protección de Datos Personales*. Registro Oficial, Quinto Suplemento n.º 459.

Asamblea Nacional del Ecuador. (2026, 15 de septiembre). *Pleno archiva Proyecto de Ley de Inteligencia Artificial* [Comunicado de prensa]. https://www.asambleanacional.gob.ec/es/noticia/119354-pleno-archiva-proyecto-de-ley-de-inteligencia

Bojanowski, P., Grave, E., Joulin, A., & Mikolov, T. (2017). Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics, 5*, 135–146. https://doi.org/10.1162/tacl_a_00051

Caldarini, G., Jaf, S., & McGarry, K. (2022). A literature survey of recent advances in chatbots. *Information, 13*(1), Artículo 41. https://doi.org/10.3390/info13010041

Cañete, J., Chaperon, G., Fuentes, R., Ho, J.-H., Kang, H., & Pérez, J. (2020). Spanish pre-trained BERT model and evaluation data. En *PML4DC at ICLR 2020*. https://arxiv.org/abs/2308.02976

Casanueva, I., Temčinas, T., Gerz, D., Henderson, M., & Vulić, I. (2020). Efficient intent detection with dual sentence encoders. En *Proceedings of the 2nd Workshop on Natural Language Processing for Conversational AI* (pp. 38–45). Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.nlp4convai-1.5

Chuizaca-Espinoza, I. A., Romero-Bermeo, D., da Silva, L. das D. de J., & Velastegui-Montoya, A. (2026). Fluvial dynamics changes driven by illegal gold mining: A land use/land cover analysis in the Ecuadorian Amazon. *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences, XLIX-B3-2026*, 573–580. https://doi.org/10.5194/isprs-archives-XLIX-B3-2026-573-2026

Colabianchi, S., Tedeschi, A., & Costantino, F. (2023). Human-technology integration with industrial conversational agents: A conceptual architecture and a taxonomy for manufacturing. *Journal of Industrial Information Integration, 35*, Artículo 100510. https://doi.org/10.1016/j.jii.2023.100510

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. En *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (Vol. 1, pp. 4171–4186). Association for Computational Linguistics. https://doi.org/10.18653/v1/N19-1423

Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation, 10*(7), 1895–1923. https://doi.org/10.1162/089976698300017197

Følstad, A., Araujo, T., Law, E. L.-C., Brandtzaeg, P. B., Papadopoulos, S., Reis, L., Baez, M., Laban, G., McAllister, P., Ischen, C., Wald, R., Catania, F., Meyer von Wolff, R., Hobert, S., & Luger, E. (2021). Future directions for chatbot research: An interdisciplinary research agenda. *Computing, 103*(12), 2915–2942. https://doi.org/10.1007/s00607-021-01016-7

Garcez, A. d'A., & Lamb, L. C. (2023). Neurosymbolic AI: The 3rd wave. *Artificial Intelligence Review, 56*(11), 12387–12406. https://doi.org/10.1007/s10462-023-10448-w

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. En D. Precup & Y. W. Teh (Eds.), *Proceedings of the 34th International Conference on Machine Learning* (Vol. 70, pp. 1321–1330). PMLR. https://proceedings.mlr.press/v70/guo17a.html

Hendrycks, D., & Gimpel, K. (2017). A baseline for detecting misclassified and out-of-distribution examples in neural networks. En *5th International Conference on Learning Representations (ICLR 2017)*. https://arxiv.org/abs/1610.02136

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

Honnibal, M., Montani, I., Van Landeghem, S., & Boyd, A. (2020). *spaCy: Industrial-strength natural language processing in Python* [Software]. Zenodo. https://doi.org/10.5281/zenodo.1212303

Jobin, A., Ienca, M., & Vayena, E. (2019). The global landscape of AI ethics guidelines. *Nature Machine Intelligence, 1*(9), 389–399. https://doi.org/10.1038/s42256-019-0088-2

Koller, D., & Friedman, N. (2009). *Probabilistic graphical models: Principles and techniques*. MIT Press.

Larson, S., Mahendran, A., Peper, J. J., Clarke, C., Lee, A., Hill, P., Kummerfeld, J. K., Leach, K., Laurenzano, M. A., Tang, L., & Mars, J. (2019). An evaluation dataset for intent classification and out-of-scope prediction. En *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)* (pp. 1311–1316). Association for Computational Linguistics. https://doi.org/10.18653/v1/D19-1131

Mehrabi, N., Morstatter, F., Saxena, N., Lerman, K., & Galstyan, A. (2021). A survey on bias and fairness in machine learning. *ACM Computing Surveys, 54*(6), 1–35. https://doi.org/10.1145/3457607

Mestanza-Ramón, C., Cuenca-Cumbicus, J., D'Orio, G., Flores-Toala, J., Segovia-Cáceres, S., Bonilla-Bonilla, A., & Straface, S. (2022). Gold mining in the Amazon region of Ecuador: History and a review of its socio-environmental impacts. *Land, 11*(2), Artículo 221. https://doi.org/10.3390/land11020221

Mishra, R., Uotinen, L., & Rinne, M. (2021). A Bayesian network approach for geotechnical risk assessment in underground mines. *Journal of the Southern African Institute of Mining and Metallurgy, 121*(6), 287–294. https://doi.org/10.17159/2411-9717/581/2021

Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., & Gebru, T. (2019). Model cards for model reporting. En *Proceedings of the Conference on Fairness, Accountability, and Transparency* (pp. 220–229). Association for Computing Machinery. https://doi.org/10.1145/3287560.3287596

Mitchell, T. M. (1997). *Machine learning*. McGraw-Hill.

National Institute of Standards and Technology. (2023). *Artificial intelligence risk management framework (AI RMF 1.0)* (NIST AI 100-1). U.S. Department of Commerce. https://doi.org/10.6028/NIST.AI.100-1

Parlamento Europeo y Consejo de la Unión Europea. (2024). *Reglamento (UE) 2024/1689 por el que se establecen normas armonizadas en materia de inteligencia artificial (Reglamento de Inteligencia Artificial)*. Diario Oficial de la Unión Europea, L, 12 de julio de 2024. Versión consolidada del 27 de julio de 2026. https://eur-lex.europa.eu/eli/reg/2024/1689/oj

Parlamento Europeo y Consejo de la Unión Europea. (2026). *Reglamento (UE) 2026/1744 que modifica el Reglamento (UE) 2024/1689 (Ómnibus digital sobre IA)*. Diario Oficial de la Unión Europea, L, 24 de julio de 2026.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., … Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. En *Advances in Neural Information Processing Systems 32* (pp. 8024–8035). Curran Associates.

Pearl, J. (1988). *Probabilistic reasoning in intelligent systems: Networks of plausible inference*. Morgan Kaufmann.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Russell, S., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4.ª ed.). Pearson.

Superintendencia de Protección de Datos Personales. (2026a). *Resolución n.º SPDP-SPD-2026-0009-R: Norma general para la garantía del derecho de protección de datos personales en el uso de sistemas de inteligencia artificial*. Registro Oficial n.º 240, 10 de marzo de 2026.

Superintendencia de Protección de Datos Personales. (2026b). *Resolución n.º SPDP-SPD-2026-0037-R: Reforma a la Norma general para la garantía del derecho de protección de datos personales en el uso de sistemas de inteligencia artificial*.

UNESCO. (2022). *Recomendación sobre la ética de la inteligencia artificial* (adoptada el 23 de noviembre de 2021). https://unesdoc.unesco.org/ark:/48223/pf0000381137_spa

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. En *Advances in Neural Information Processing Systems 30* (pp. 5998–6008). Curran Associates.

Wei, J., & Zou, K. (2019). EDA: Easy data augmentation techniques for boosting performance on text classification tasks. En *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)* (pp. 6382–6388). Association for Computational Linguistics. https://doi.org/10.18653/v1/D19-1670

Wellsandt, S., Klein, K., Hribernik, K., Lewandowski, M., Bousdekis, A., Mentzas, G., & Thoben, K.-D. (2022). Hybrid-augmented intelligence in predictive maintenance with digital intelligent assistants. *Annual Reviews in Control, 53*, 382–390. https://doi.org/10.1016/j.arcontrol.2022.04.001

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., … Rush, A. (2020). Transformers: State-of-the-art natural language processing. En *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations* (pp. 38–45). Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.emnlp-demos.6

Zambrano Martínez, J. L. (2026). *Inteligencia artificial vs. machine learning y ChatBots* [Documento académico de apoyo, Semana 4, asignatura Inteligencia Artificial]. Maestría en Inteligencia de Negocios y Ciencia de Datos, Universidad Espíritu Santo.

Anexos

A Manual básico de usuario

Instalación e inicio

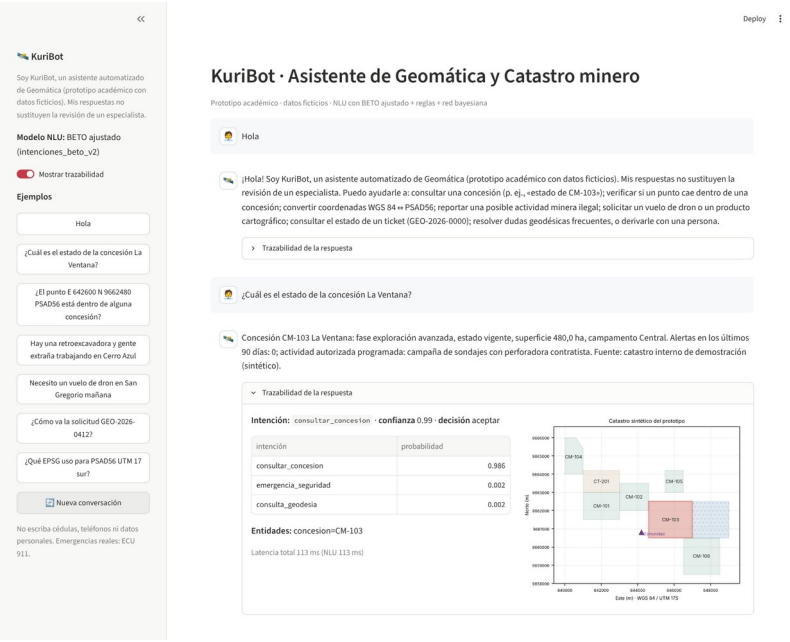
1. Instale Python 3.10 o superior (en Windows, marque «Add Python to PATH»).
2. Descargue el proyecto desde el repositorio (botón *Code → Download ZIP*, o `git clone`).
3. En una terminal abierta en la carpeta del proyecto, ejecute `python -m pip install -r requirements.txt` y, una sola vez, `python scripts/unir_modelo.py`. Este script une las cinco partes del modelo guardadas en `modelos/partes_beto_v2/`, porque GitHub no admite archivos de más de 100 MB, verifica sus sumas SHA-256 y crea `modelos/intenciones_beto_v2/`. Sin el modelo, la aplicación funciona con la línea base TF-IDF y lo indica en la barra lateral.
4. Interfaz web: `streamlit run app_streamlit.py` (se abre `http://localhost:8501`). Consola: `python cli.py`; escriba `salir` para terminar.

Uso de la interfaz web

La pantalla tiene tres zonas (Figura 5). La barra lateral muestra el aviso de asistente automatizado, el modelo cargado, el interruptor de trazabilidad, mensajes de ejemplo que se envían con un clic y el botón «Nueva conversación». La zona central contiene la conversación y el cuadro de mensajes. Bajo cada respuesta, el desplegable «Trazabilidad de la respuesta» muestra la intención y su confianza, las tres intenciones más probables, las entidades, las reglas disparadas, la red bayesiana (posterior y evidencias), las ventanas de vuelo y, cuando corresponde, un mapa del catastro sintético (Figura 6 a Figura 9). La consola imprime la misma información en una línea técnica (Figura 10).

Figura 5.

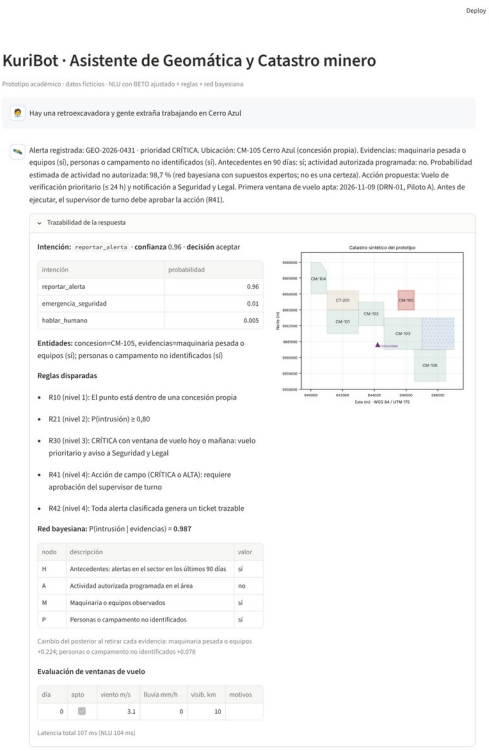
Interfaz web: saludo, aviso de transparencia y consulta de una concesión



Nota. Captura de la ejecución real del prototipo.

Figura 6.

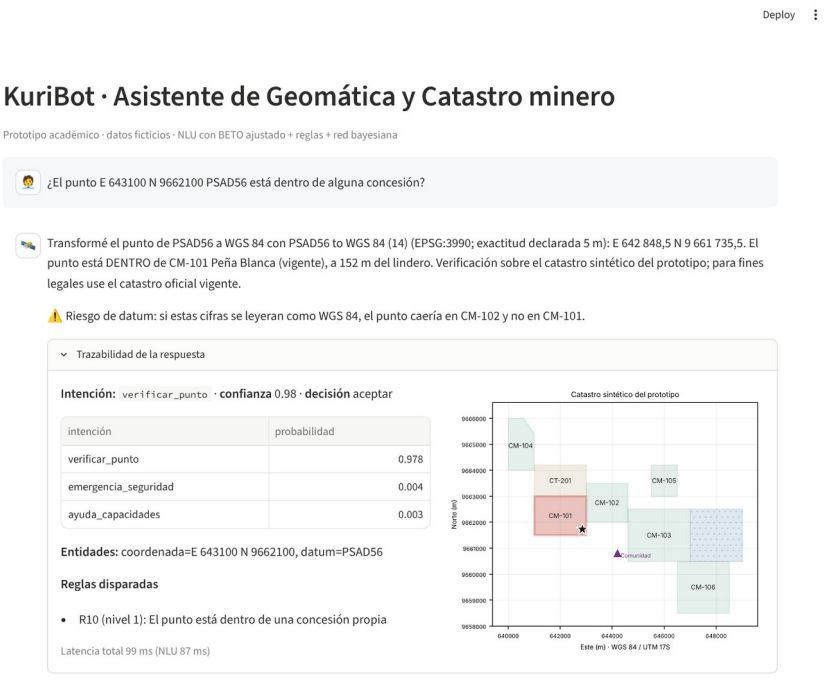
Reporte de alerta con prioridad CRÍTICA: posterior bayesiano, evidencias, reglas disparadas y mapa



Nota. Captura de la ejecución real del prototipo.

Figura 7.

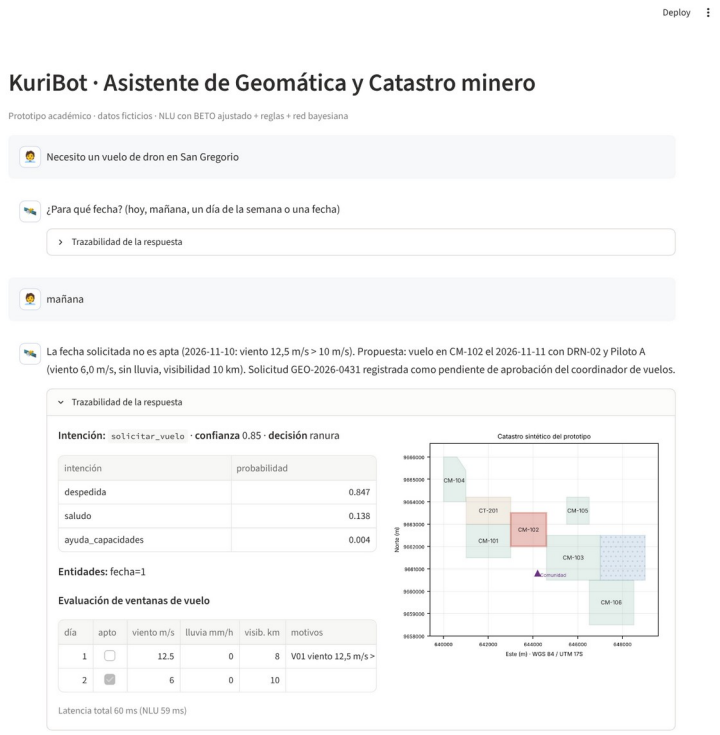
Verificación de un punto en PSAD56 con advertencia de riesgo de datum



Nota. Leídas como WGS 84, las mismas cifras caerían en otra concesión. Captura de la ejecución real del prototipo.

Figura 8.

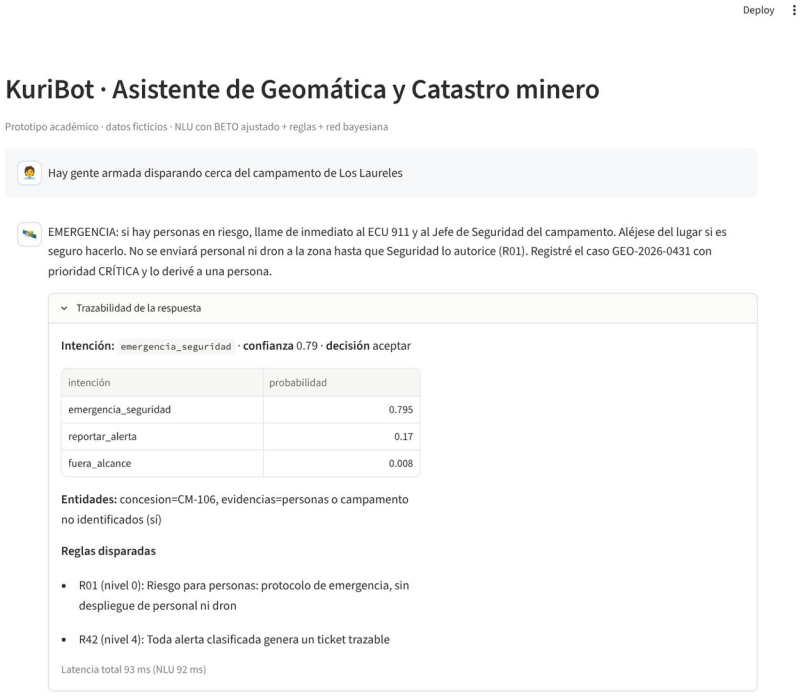
Solicitud de vuelo en dos turnos: el bot pide la fecha, descarta el día con viento y propone la primera ventana apta



Nota. Captura de la ejecución real del prototipo.

Figura 9.

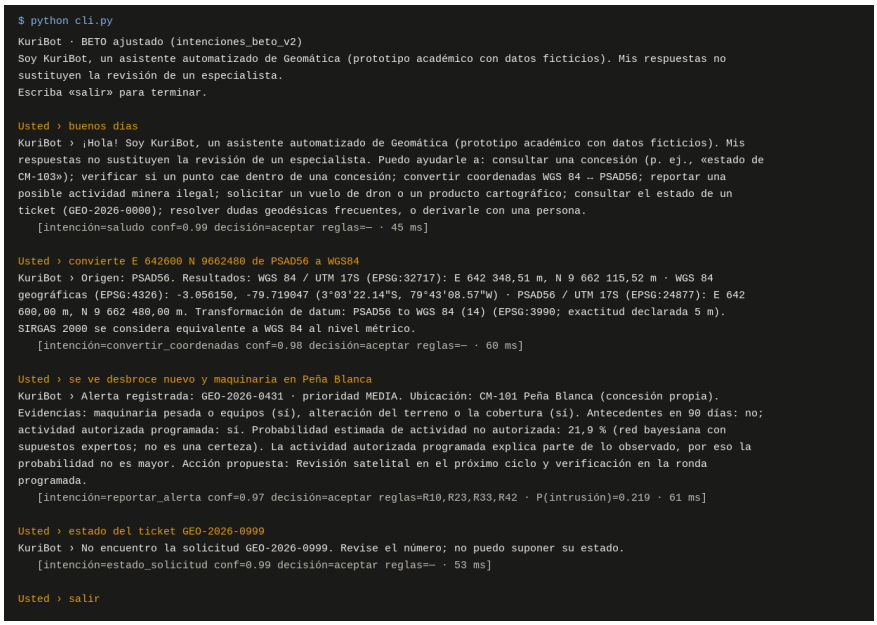
Emergencia: la regla R01 prevalece, no se despacha personal ni dron y el caso se deriva a una persona



Nota. Captura de la ejecución real del prototipo.

Figura 10.

Interfaz de consola con la línea técnica de cada respuesta



Nota. Captura de la ejecución real del prototipo.

Datos de demostración y buenas prácticas

Tabla 10.

Concesiones, coordenadas y tickets sintéticos para la demostración

Elemento	Valor	Qué permite probar
CM-101 Peña Blanca	Explotación, vigente, actividad autorizada programada	Explicación alternativa: maquinaria y desbroce → prioridad MEDIA
CM-103 La Ventana	Exploración avanzada; colinda con bosque protector y comunidad	Regla R40 de revisión humana
CM-104 El Roble	Renovación en trámite; 2 alertas en 90 días	Alerta CRÍTICA con vuelo al día siguiente
CM-105 Cerro Azul	Suspendida; 3 alertas en 90 días	Alerta CRÍTICA con vuelo el mismo día
CM-106 Los Laureles	Exploración inicial; 1 alerta en 90 días	Turbidez → prioridad ALTA
CT-201 Minas del Valle	Concesión de un tercero	Regla R20: revisión legal, no intrusión
E 642600 N 9662480 PSAD56	Se transforma a WGS 84 (≈ 443 m)	Conversión y verificación de datum
E 641100 N 9664950 WGS84	100 m fuera de CM-104	Franja de amortiguamiento (R11)
GEO-2026-0412 / 0415 / 0419 / 0423 / 0430	Tickets semilla	Consulta de estado; GEO-2026-0999 no existe

- Indique siempre el datum de las coordenadas (WGS 84 o PSAD56); si lo omite, el asistente lo pedirá.
- No escriba cédulas, teléfonos ni datos de terceros; si lo hace, el sistema los enmascara y no los registra.
- Ante riesgo para personas, escriba «emergencia» y llame al ECU 911: el asistente no reemplaza los canales de emergencia.
- Escriba «cancelar» para abandonar una solicitud y «quiero hablar con una persona» para derivar.
- Si `streamlit` no se reconoce como comando, ejecute `python -m streamlit run app_streamlit.py`.

B Ficha del modelo, reproducción y correspondencia con la rúbrica

Tabla 11.

Ficha del modelo de intenciones (formato adaptado de Mitchell et al., 2019)

Campo	Descripción
Modelo	BETO (bert-base-spanish-wwm-cased, versión del pipeline es_dep_news_trf 3.8.0 de spaCy) ajustado de extremo a extremo, con media de la última capa y capa lineal; 109,3 millones de parámetros (intenciones_beto_v2, float16, 219 MB)
Uso previsto	Clasificar mensajes internos de la mesa de Geomática en 14 intenciones para un ChatBot con reglas y red bayesiana
Usos no previstos	Decisiones sobre personas, atención al público, idiomas distintos del español, emergencias sin canal humano
Datos de entrenamiento	Corpus sintético en español: 639 frases de entrenamiento más una copia con ruido por frase (1278 en total) y 129 de validación, de 414 plantillas y 14 intenciones
Datos de evaluación	252 frases de prueba redactadas aparte y su versión con ruido ortográfico (252)
Métricas	F1 macro 0,929 (limpia) y 0,874 (ruido); exactitud 0,929 [IC Wilson 0,89–0,95]; ECE 0,042; latencia p95 53,6 ms en CPU
Calibración	Escalado de temperatura en validación (T = 0,81); umbral de aceptación 0,50
Factores no evaluados	Variantes dialectales, kichwa y shuar, voz, usuarios reales, deriva temporal
Licencia del modelo base	Pipeline es_dep_news_trf de spaCy (GPL-3.0); BETO original con su propia licencia

Reproducción. El código, los datos sintéticos, los resultados y el cuaderno para Google Colab están en https://github.com/gaboas2703/kuribot-chatbot-geomatica. La secuencia es: extraer BETO (`scripts/extraer_beto.py`), construir la base y el corpus (`datos/`), entrenar las iteraciones 1 y 2 (`experimentos/entrenar_nlp.py`), ejecutar las pruebas (`pruebas/`) y generar las figuras (`experimentos/figuras.py`). El archivo `README.md` contiene los comandos exactos, y `cuaderno/KuriBot_Colab.ipynb` ejecuta la misma secuencia en Colab.

Tabla 12.

Correspondencia con los criterios de evaluación

Criterio (puntos)	Dónde se evidencia
Caso de negocio y diferenciación IA vs. ML (2)	Caso de negocio: diagnóstico, actores, Tabla 1, Tabla 2 y escenario económico; arquitectura en la Figura 1
Modelado técnico: NLP y razonamiento (4)	Metodología; clasificador, red bayesiana (Figura 2 y Tabla 3), motor de reglas (Tabla 4); comparación de cuatro modelos y ajustes (Tabla 5 a Tabla 7)
Funcionalidad demostrable (3,5)	Interfaces web y de consola (Anexo A); 34 escenarios y 43 turnos documentados, 8 de confirmación y 34 pruebas unitarias (Anexo C); análisis de fallos y ajustes
Ética, regulación y sesgos (2)	Análisis crítico: ética, sesgos, marco regulatorio y lineamientos de uso responsable
Rigor metodológico (2)	Particiones sin fuga, validación cruzada agrupada, intervalos de confianza, McNemar, calibración, dos iteraciones y confirmación; limitaciones declaradas
Normas APA 7 (1,5)	Citas autor-fecha en el texto y 43 referencias verificadas

C Evidencia de pruebas

Tabla 13.

Suite de aceptación: última entrada de cada escenario, decisión del sistema y resultado por iteración

ID	Escenario	Última entrada	Intención	Acción / reglas	v1	v2	ms
A01	Saludo y aviso de transparencia	buenos días, ¿con quién hablo?	saludo	responder	Sí	Sí	48

ID	Escenario	Última entrada	Intención	Acción / reglas	v1	v2	ms
A02	Capacidades	¿qué cosas puedes resolver?	ayuda capacidades	responder	Sí	Sí	36
A03	Ficha de concesión por nombre	dame la ficha de la concesión La Ventana	consultar concesion	responder	Sí	Sí	40
A04	Estado de concesión suspendida	¿cerro azul está activa?	consultar concesion	responder	Sí	Sí	34
A05	Ranura faltante de concesión	CM-106	consultar concesion	responder	Sí	Sí	27
A06	Punto WGS84 dentro de concesión	¿el punto E 642350 N 9662120 WGS84 está dentro…	verificar punto	geoproceso · R10	Sí	Sí	43
A07	Punto sin datum: se pide y luego cerca de comunidad	wgs84	verificar punto	geoproceso · R10, R14, R40	Sí	Sí	28
A08	Punto PSAD56 transformado antes del cruce	¿E 642600 N 9662480 PSAD56 en qué concesión qu…	verificar punto	geoproceso · R10	Sí	Sí	53
A09	Punto en franja de amortiguamiento	chequea 641100 9664950 wgs84 contra el catastro	verificar punto	geoproceso · R11	Sí	Sí	40
A10	Punto en concesión de tercero	¿la coordenada 642000 9663600 wgs84 es nuestra?	verificar punto	geoproceso · R12	Sí	Sí	40
A11	Conversión PSAD56 -> WGS84	convierte E 642600 N 9662480 de PSAD56 a WGS84	convertir coordenadas	geoproceso	Sí	Sí	41
A12	Conversión geográficas -> UTM	pasa -3.0512, -79.7301 a utm, está en wgs84	convertir coordenadas	geoproceso	Sí	Sí	41
A13	Conversión con dos ranuras	psad56	convertir coordenadas	geoproceso	Sí	Sí	28
A14	Alerta con maquinaria y personas en concesión suspendida	hay una retroexcavadora y gente extraña trabaj…	reportar alerta	Vuelo de verificación prioritari… · R10, R21, R30, R41, R42	Sí	Sí	44
A15	Alerta explicada por actividad autorizada (explaining away)	se ve desbroce nuevo y maquinaria en Peña Blanca	reportar alerta	Revisión satelital en el próximo… · R10, R23, R33, R42	Sí	Sí	47
A16	Alerta en tres turnos con ranuras	el agua baja turbia	reportar alerta	Programar vuelo de verificación … · R10, R13, R22, R32, R40, R41	Sí	Sí	33
A17	Alerta en concesión de tercero	vimos maquinaria en E 642000 N 9663600 WGS84	reportar alerta	Derivar a Legal y Catastro para … · R12, R20, R35, R42	No	Sí	54
A18	Alerta crítica con ventana de vuelo al día siguiente	reporto voladuras de noche y una trocha nueva …	reportar alerta	Vuelo de verificación prioritari… · R10, R21, R30, R41, R42	Sí	Sí	42
A19	Alerta con negación y zona sensible	hay maquinaria en La Ventana, pero no hay pers…	reportar alerta	Registrar y mantener monitoreo · R10, R13, R14, R24, R34, R40	Sí	Sí	50
A20	Emergencia con armas	hay gente armada disparando cerca del campamento	emergencia seguridad	PROTOCOLO DE EMERGENCIA · R01, R42	Sí	Sí	38
A21	Vuelo con fecha no apta por viento	mañana	solicitar vuelo	programar vuelo	Sí	Sí	18
A22	Vuelo el mismo día sin urgencia (anticipación)	programa un vuelo en Los Laureles para hoy	solicitar vuelo	programar vuelo	Sí	Sí	42
A23	Solicitud de KMZ	necesito el KMZ de los linderos de CM-102	solicitar cartografia	crear ticket	Sí	Sí	51
A24	Cartografía con ranuras	de El Roble	solicitar cartografia	crear ticket	Sí	Sí	29

ID	Escenario	Última entrada	Intención	Acción / reglas	v1	v2	ms
A25	Estado de ticket existente	¿cómo va la solicitud GEO-2026-0412?	estado solicitud	consultar ticket	Sí	Sí	43
A26	Ticket inexistente (no inventar)	estado del ticket GEO-2026-0999	estado solicitud	no encontrado	Sí	Sí	47
A27	FAQ de códigos EPSG	¿qué EPSG corresponde a PSAD56 en UTM 17 sur?	consulta geodesia	responder faq	Sí	Sí	43
A28	FAQ de desplazamiento entre datums	¿cuántos metros se mueve un punto entre PSAD56…	consulta geodesia	responder faq	Sí	Sí	44
A29	Derivación a humano	quiero hablar con un analista de verdad	hablar humano	derivar humano · R03	Sí	Sí	42
A30	Fuera de alcance	¿quién ganó la Copa Libertadores?	fuera alcance	fuera alcance	Sí	Sí	40
A31	Dato personal enmascarado	mi cédula es 3000000004, ¿cuál es el estado de…	consultar concesion	responder · R02	No	Sí	44
A32	Mensaje con faltas ortográficas	ola, nesesito saver el estado de la consecion …	consultar concesion	responder	Sí	Sí	41
A33	Despedida	gracias, eso es todo	despedida	cerrar	Sí	Sí	36
A34	Coordenadas sin verbo (mensaje ambiguo)	642350 9662120	convertir coordenadas	solicitar dato:datum	Sí	Sí	34

Nota. Sí = cumple todos los criterios (intención, acción o regla, contenido obligatorio y latencia < 1 500 ms). Las respuestas completas de cada turno están en los archivos JSON de la carpeta resultados del repositorio.

Tabla 14.

Resumen de las ejecuciones de la suite funcional

Ejecución	Modelo NLU	Escenarios	Aprobados	Tasa	Latencia media (ms)	Latencia p95 (ms)
Aceptación, iteración 1	BETO ajustado (intenciones_beto_v1)	34	32	94,1 %	40,5	51,8
Aceptación, iteración 2	BETO ajustado (intenciones_beto_v2)	34	34	100,0 %	38,6	50,5
Confirmación (escenarios nuevos)	BETO ajustado (intenciones_beto_v2)	8	7	87,5 %	38,9	43,0

Pruebas unitarias (`python -m pytest pruebas/test_unitarias.py -v`):

```
============================= test session starts ==============================
platform linux -- Python 3.13.18, pytest-9.1.1, pluggy-1.6.0 -- /usr/local/bin/python3
cachedir: .pytest_cache
rootdir: /home/claude/kuribot
plugins: platformdirs-4.13.5, anyio-4.15.1
collecting ... collected 34 items

pruebas/test_unitarias.py::test_ida_y_vuelta_psad56_wgs84 PASSED         [  2%]
pruebas/test_unitarias.py::test_desplazamiento_psad56_en_zona_del_proyecto PASSED [  5%]
pruebas/test_unitarias.py::test_geograficas_a_utm PASSED                 [  8%]
pruebas/test_unitarias.py::test_operacion_epsg_3990_disponible PASSED    [ 11%]
pruebas/test_unitarias.py::test_extraer_coordenadas[E 642350 N 9662120-esperado0] PASSED [ 14%]
pruebas/test_unitarias.py::test_extraer_coordenadas[642350, 9662120-esperado1] PASSED [ 17%]
pruebas/test_unitarias.py::test_extraer_coordenadas[x=644800 y=9661500-esperado2] PASSED [ 20%]
pruebas/test_unitarias.py::test_extraer_coordenadas[-3.0512, -79.7301-esperado3] PASSED [ 23%]
pruebas/test_unitarias.py::test_extraer_coordenadas[3\xb003'04"S 79\xb043'48"W-esperado4] PASSED [ 26%]
pruebas/test_unitarias.py::test_datum_detectado PASSED                   [ 29%]
pruebas/test_unitarias.py::test_concesion_por_codigo_nombre_y_errata PASSED [ 32%]
pruebas/test_unitarias.py::test_evidencias_con_negacion PASSED           [ 35%]
pruebas/test_unitarias.py::test_fechas_relativas PASSED                  [ 38%]
pruebas/test_unitarias.py::test_cedula_y_enmascarado PASSED              [ 41%]
pruebas/test_unitarias.py::test_emergencia_palabras_clave PASSED         [ 44%]
pruebas/test_unitarias.py::test_pgmpy_coincide_con_enumeracion PASSED    [ 47%]
pruebas/test_unitarias.py::test_prior_sin_evidencia PASSED               [ 50%]
pruebas/test_unitarias.py::test_explicacion_alternativa_reduce_posterior PASSED [ 52%]
pruebas/test_unitarias.py::test_monotonia_evidencias PASSED              [ 55%]
pruebas/test_unitarias.py::test_sensibilidad_ordenada PASSED             [ 58%]
pruebas/test_unitarias.py::test_emergencia_prevalece PASSED              [ 61%]
pruebas/test_unitarias.py::test_tercero_no_se_califica_como_intrusion PASSED [ 64%]
pruebas/test_unitarias.py::test_umbral_prioridad[0.9-CR\xcdTICA] PASSED  [ 67%]
pruebas/test_unitarias.py::test_umbral_prioridad[0.6-ALTA] PASSED        [ 70%]
pruebas/test_unitarias.py::test_umbral_prioridad[0.3-MEDIA] PASSED       [ 73%]
pruebas/test_unitarias.py::test_umbral_prioridad[0.05-BAJA] PASSED       [ 76%]
pruebas/test_unitarias.py::test_prioridad_unica_y_determinista PASSED    [ 79%]
pruebas/test_unitarias.py::test_supervision_humana_en_zona_sensible PASSED [ 82%]
```

```
pruebas/test_unitarias.py::test_ventana_vuelo_evita_viento PASSED        [ 85%]
pruebas/test_unitarias.py::test_anticipacion_minima PASSED               [ 88%]
pruebas/test_unitarias.py::test_analisis_espacial PASSED                 [ 91%]
pruebas/test_unitarias.py::test_ticket_inexistente_no_se_inventa PASSED  [ 94%]
pruebas/test_unitarias.py::test_areas_derivadas_de_geometria PASSED      [ 97%]
pruebas/test_unitarias.py::test_riesgo_de_datum_cambia_la_concesion PASSED [100%]

============================== 34 passed in 2.49s ==============================
```

D Código fuente completo

Se reproducen los módulos del ChatBot, la construcción de datos, el entrenamiento y evaluación, las pruebas y la extracción de BETO. El corpus fuente (`datos/corpus_fuente.py`), los datos sintéticos y los scripts auxiliares de figuras y capturas están en el repositorio.

D.1 Extracción de entidades y datos personales (kuribot/entidades.py)

```
"""Extracción de entidades basada en reglas (expresiones regulares + diccionario geográfico).

La intención la decide la red neuronal; los valores concretos (coordenadas, códigos, fechas,
evidencias) los extrae este módulo de forma determinista y auditable. También detecta datos
personales para enmascararlos antes de registrar la conversación.
"""
from __future__ import annotations

import difflib
import re
import unicodedata
from dataclasses import dataclass, field

from .geodesia import Punto

# ---------------------------------------------------------------------- utilidades


def plano(texto: str) -> str:
    """Minúsculas y sin tildes (conserva la ñ) para comparar con diccionarios."""
    t = unicodedata.normalize("NFD", texto.lower())
    t = "".join(c for c in t if unicodedata.category(c) != "Mn" or c == "̃")
    return unicodedata.normalize("NFC", t)


def _num(s: str) -> float:
    return float(s.replace(",", "."))


# ---------------------------------------------------------------------- coordenadas

_UTM_ETIQ = re.compile(r"(?:\b[ex]|este)\s*[:=]?\s*(\d{6}(?:\.\d+)?)\s*[,;]?\s*(?:\b[ny]|norte)\s*[:=]?\s*(\d{7}(?:\.\d+)?)",
re.I)
_UTM_PAR = re.compile(r"(?<![\d.])(\d{6}(?:\.\d+)?)\s*[,;]?\s+(\d{7}(?:\.\d+)?)(?![\d])|(?<![\d.])(\d{6}(?:\.\d+)?),(\d{7}(?:\.\
d+)?)(?![\d])")
_DEC = re.compile(r"(?:lat(?:itud)?\s*[:=]?\s*)?(-?\d{1,2}\.\d{3,})\s*[,;]?\s+(?:lon(?:gitud)?\s*[:=]?\s*)?(-?\d{2,3}\.\d{3,})|"
                  r"(-?\d{1,2}\.\d{3,}),\s*(-?\d{2,3}\.\d{3,})", re.I)
_DMS = re.compile(r"(\d{1,2})\s*°\s*(\d{1,2})\s*['′]\s*(\d{1,2}(?:[.,]\d+)?)\s*(?:\"|″|'')?\s*([NS])\s*[,;]?\s*"
                  r"(\d{2,3})\s*°\s*(\d{1,2})\s*['′]\s*(\d{1,2}(?:[.,]\d+)?)\s*(?:\"|″|'')?\s*([EWO])", re.I)
_DATUM = [("PSAD56", re.compile(r"psad\s*-?\s*56|psad\b", re.I)),
          ("WGS84", re.compile(r"wgs\s*-?\s*84|wgs\b", re.I)),
          ("SIRGAS", re.compile(r"sirgas", re.I))]


@dataclass
class Coordenada:
    punto: Punto
    texto: str
    inicio: int
    fin: int
    avisos: list[str] = field(default_factory=list)


def extraer_datum(texto: str) -> str | None:
    """Devuelve el datum mencionado. Si hay dos, toma el primero (origen) — el destino lo decide el diálogo."""
    hallados = sorted(((m.start(), nombre) for nombre, rx in _DATUM for m in rx.finditer(texto)))
    return hallados[0][1] if hallados else None


def extraer_coordenada(texto: str) -> Coordenada | None:
    datum = extraer_datum(texto)
    m = _UTM_ETIQ.search(texto)
    if m:
        return Coordenada(Punto(_num(m.group(1)), _num(m.group(2)), "utm", datum), m.group(0), m.start(), m.end())
    m = _DMS.search(texto)
    if m:
        lat = int(m.group(1)) + int(m.group(2)) / 60 + _num(m.group(3)) / 3600
        lon = int(m.group(5)) + int(m.group(6)) / 60 + _num(m.group(7)) / 3600
        lat = -lat if m.group(4).upper() == "S" else lat
        lon = -lon if m.group(8).upper() in ("W", "O") else lon
        return Coordenada(Punto(lon, lat, "geo", datum), m.group(0), m.start(), m.end())
    m = _DEC.search(texto)
    if m:
        a, b = (m.group(1), m.group(2)) if m.group(1) else (m.group(3), m.group(4))
        lat, lon = _num(a), _num(b)
```

```
        avisos = []
        if lon > 0 and 75 <= lon <= 92:
            lon = -lon
            avisos.append("Longitud sin signo: se asumió Oeste.")
        return Coordenada(Punto(lon, lat, "geo", datum), m.group(0), m.start(), m.end(), avisos)
    m = _UTM_PAR.search(texto)
    if m:
        e, n = (m.group(1), m.group(2)) if m.group(1) else (m.group(3), m.group(4))
        return Coordenada(Punto(_num(e), _num(n), "utm", datum), m.group(0), m.start(), m.end())
    return None


# ---------------------------------------------------------------- tickets, concesiones, fechas, productos

_TICKET = re.compile(r"\bgeo[-\s]?(\d{4})[-\s]?(\d{4})\b", re.I)
_CODIGO = re.compile(r"\b(cm|ct)\s*-?\s*(\d{3})\b", re.I)


def extraer_ticket(texto: str) -> str | None:
    m = _TICKET.search(texto)
    return f"GEO-{m.group(1)}-{m.group(2)}" if m else None


def extraer_concesion(texto: str, nombres: dict[str, str]) -> str | None:
    """nombres: {nombre_plano: codigo}. Acepta código (CM-101) o nombre con tolerancia a erratas."""
    m = _CODIGO.search(texto)
    if m:
        return f"{m.group(1).upper()}-{m.group(2)}"
    t = plano(texto)
    for nombre, codigo in nombres.items():
        if re.search(rf"\b{re.escape(nombre)}\b", t):
            return codigo
    palabras = re.findall(r"[a-zñ]+", t)
    mejor, puntaje = None, 0.0
    for nombre, codigo in nombres.items():
        k = len(nombre.split())
        for i in range(len(palabras) - k + 1):
            r = difflib.SequenceMatcher(None, " ".join(palabras[i:i + k]), nombre).ratio()
            if r > puntaje:
                mejor, puntaje = codigo, r
    return mejor if puntaje >= 0.86 else None


_DIAS = {"lunes": 0, "martes": 1, "miercoles": 2, "jueves": 3, "viernes": 4, "sabado": 5, "domingo": 6}
_MESES = {m: i + 1 for i, m in enumerate(["enero", "febrero", "marzo", "abril", "mayo", "junio", "julio", "agosto",
                                          "septiembre", "octubre", "noviembre", "diciembre"])}


def extraer_fecha(texto: str, hoy) -> int | None:
    """Devuelve el desfase en días respecto de 'hoy' (date) o None."""
    import datetime as dt
    t = plano(texto)
    if "pasado mañana" in t or "pasado manana" in t:
        return 2
    if re.search(r"\bmañana\b|\bmanana\b", t):
        return 1
    if re.search(r"\bhoy\b|\bahora\b|\burgente\b", t):
        return 0
    for nombre, wd in _DIAS.items():
        if re.search(rf"\b{nombre}\b", t):
            return (wd - hoy.weekday()) % 7 or 7
    m = re.search(r"\b(\d{1,2})\s+de\s+(" + "|".join(_MESES) + r")\b", t)
    if m:
        try:
            f = dt.date(hoy.year, _MESES[m.group(2)], int(m.group(1)))
            return (f - hoy).days
        except ValueError:
            return None
    m = re.search(r"\b(\d{1,2})/(\d{1,2})(?:/(\d{2,4}))?\b", t)
    if m:
        try:
            f = dt.date(hoy.year, int(m.group(2)), int(m.group(1)))
            return (f - hoy).days
        except ValueError:
            return None
    if "esta semana" in t:
        return 1
    return None


_PRODUCTOS = [("ortofoto", r"ortofoto|orto\b"), ("modelo digital de elevación", r"modelo digital|\bmde\b|\bmdt\b|\bdem\b|curvas
de nivel"),
              ("KMZ/KML", r"\bkmz\b|\bkml\b"), ("shapefile", r"shape|shp\b|geopackage|\bgpkg\b|capa"),
              ("plano topográfico", r"topograf|plano"), ("mapa temático", r"tematic|cobertura|pendiente"),
              ("mapa de ubicación", r"mapa")]


def extraer_producto(texto: str) -> str | None:
    t = plano(texto)
    for nombre, rx in _PRODUCTOS:
        if re.search(rx, t):
            return nombre
    return None


# ---------------------------------------------------------------- evidencias de alerta (para la red bayesiana)
```

```
EVIDENCIAS = {
    "M": ("maquinaria pesada o equipos", r"maquinaria|excavadora|retroexcavadora|volqueta|clasificadora|draga|bomba|"
                                         r"motobomba|manguera|tractor|perforadora|maquina"),
    "S": ("alteración del terreno o la cobertura", r"desbroce|deforest|\btala\b|claro nuevo|cambio de cobertura|"
                                                   r"movimiento de tierra|hueco|\bexcava(?!dor)|labores|bocamina|socavon|imagen|
sentinel|satelit"),
    "P": ("personas o campamento no identificados", r"gente|personas|mineros|informales|campamento|carpas"),
    "R": ("ruido o voladuras nocturnas", r"ruido|voladura|explosion|dinamita|de noche|madrugada|nocturn"),
    "T": ("agua turbia aguas abajo", r"turbi|lodo|agua sucia|sediment"),
    "V": ("vía o trocha nueva", r"trocha|camino nuevo|camino abierto|aparecio un camino|via nueva|acceso nuevo|sendero nuevo"),
}
_NEGACION = re.compile(r"\b(no|sin|ningun|ninguna|ninguno|nada de|tampoco|ni)\b")


def extraer_evidencias(texto: str) -> dict[str, bool]:
    """Evidencias observadas: True si se afirma, False si se niega explícitamente. Ausentes = no observadas."""
    t = plano(texto)
    salida: dict[str, bool] = {}
    for clave, (_, rx) in EVIDENCIAS.items():
        for m in re.finditer(rx, t):
            previo = " ".join(t[:m.start()].split()[-3:])
            salida[clave] = not bool(_NEGACION.search(previo))
            break
    return salida


# ---------------------------------------------------------------------- seguridad y datos personales

_EMERGENCIA = re.compile(r"armad[oa]s?\b|\barmas\b|disparo|dispararon|\btiros\b|balacera|herid[oa]|lesionad|secuestr|"
                         r"retuvieron|retenid|amenaz|atrapad|incendio|agred|agresi|asalt|desaparecid|violencia|"
                         r"enfrentamiento|atacaron|auxilio|encapuchad")


def hay_emergencia(texto: str) -> bool:
    return bool(_EMERGENCIA.search(plano(texto)))


def cedula_valida(d: str) -> bool:
    """Validación del dígito verificador de la cédula ecuatoriana (módulo 10)."""
    if len(d) != 10 or not d.isdigit():
        return False
    prov, tercero = int(d[:2]), int(d[2])
    if not (1 <= prov <= 24 or prov == 30) or tercero >= 6:
        return False
    suma = 0
    for i, c in enumerate(d[:9]):
        v = int(c) * (2 if i % 2 == 0 else 1)
        suma += v - 9 if v > 9 else v
    return (10 - suma % 10) % 10 == int(d[9])


_DIEZ = re.compile(r"(?<!\d)(\d{10})(?!\d)")
_TELEFONO = re.compile(r"(?<!\d)(?:\+?593\s?|0)9\d{8}(?!\d)")
_CORREO = re.compile(r"[\w.+-]+@[\w-]+\.[\w.]+")


def enmascarar_pii(texto: str) -> tuple[str, list[str]]:
    tipos: list[str] = []

    def _ced(m):
        if cedula_valida(m.group(1)):
            tipos.append("cédula")
            return "[CÉDULA]"
        return m.group(0)

    t = _DIEZ.sub(_ced, texto)
    t = _TELEFONO.sub(lambda m: (tipos.append("teléfono"), "[TELÉFONO]")[1], t)
    t = _CORREO.sub(lambda m: (tipos.append("correo"), "[CORREO]")[1], t)
    return t, tipos


# ---------------------------------------------------------------------- enmascarado para el clasificador


def enmascarar_entidades(texto: str) -> str:
    """Sustituye valores por su tipo antes de clasificar la intención (mismo paso para todos los modelos)."""
    t = texto
    c = extraer_coordenada(t)
    while c is not None:
        t = t[:c.inicio] + " COORDENADA " + t[c.fin:]
        c = extraer_coordenada(t)
    t = _TICKET.sub(" TICKET ", t)
    t = _CODIGO.sub(" CODIGO ", t)
    return re.sub(r"\s+", " ", t).strip()
```

D.2 Transformaciones geodésicas y utilidades UTM (kuribot/geodesia.py)

```
"""Operaciones geodésicas y espaciales de KuriBot (pyproj + Shapely).

Convenciones
- Sistema de trabajo: WGS 84 / UTM zona 17S (EPSG:32717), en metros.
- PSAD56 / UTM 17S (EPSG:24877) se transforma con la operación EPSG:3990
  «PSAD56 to WGS 84 (14)», Helmert de 7 parámetros (convención coordinate frame) válida
  para Ecuador continental, con exactitud declarada de 5 m en la base EPSG. En producción
  deben confirmarse los parámetros oficiales vigentes del IGM antes de usar los resultados
  con fines legales o catastrales.
```

```
- SIRGAS 2000 / UTM 17S (EPSG:31977) se trata como equivalente a WGS 84 al nivel métrico
  (transformación nula de la base EPSG); la diferencia real depende de la época de referencia.
"""
from __future__ import annotations

import math
from dataclasses import dataclass
from functools import lru_cache

from pyproj import CRS, Transformer
from pyproj.transformer import TransformerGroup

EPSG = {
    "WGS84_GEO": 4326,
    "WGS84_UTM17S": 32717,
    "PSAD56_GEO": 4248,
    "PSAD56_UTM17S": 24877,
    "SIRGAS2000_UTM17S": 31977,
}
OPERACION_PSAD56_WGS84 = 3990  # PSAD56 to WGS 84 (14), Ecuador continental


@dataclass(frozen=True)
class Punto:
    """Coordenada de entrada tal como la expresó el usuario."""
    x: float              # Este (m) o longitud (°)
    y: float              # Norte (m) o latitud (°)
    tipo: str             # "utm" o "geo"
    datum: str | None     # "WGS84", "PSAD56", "SIRGAS" o None si no se indicó


@lru_cache(maxsize=None)
def _transformador(origen: int, destino: int) -> Transformer:
    return Transformer.from_crs(origen, destino, always_xy=True)


@lru_cache(maxsize=None)
def _psad56_a_wgs84_geo() -> Transformer:
    """Transformación PSAD56 (geográficas) -> WGS 84 (geográficas) fijada en EPSG:3990."""
    grupo = TransformerGroup(EPSG["PSAD56_GEO"], EPSG["WGS84_GEO"], always_xy=True)
    for t in grupo.transformers:
        ident = t.to_json_dict().get("id", {})
        if ident.get("code") == OPERACION_PSAD56_WGS84:
            return t
    raise RuntimeError("La base EPSG local no contiene la operación 3990.")


def descripcion_transformacion() -> str:
    t = _psad56_a_wgs84_geo()
    nombre = t.description.split(" (with axis")[0]
    return f"{nombre} (EPSG:{OPERACION_PSAD56_WGS84}; exactitud declarada {t.accuracy:.0f} m)"


def psad56_utm_a_wgs84_utm(e: float, n: float) -> tuple[float, float]:
    lon, lat = _transformador(EPSG["PSAD56_UTM17S"], EPSG["PSAD56_GEO"]).transform(e, n)
    lon2, lat2 = _psad56_a_wgs84_geo().transform(lon, lat)
    return _transformador(EPSG["WGS84_GEO"], EPSG["WGS84_UTM17S"]).transform(lon2, lat2)


def wgs84_utm_a_psad56_utm(e: float, n: float) -> tuple[float, float]:
    lon, lat = _transformador(EPSG["WGS84_UTM17S"], EPSG["WGS84_GEO"]).transform(e, n)
    lon2, lat2 = _psad56_a_wgs84_geo().transform(lon, lat, direction="INVERSE")
    return _transformador(EPSG["PSAD56_GEO"], EPSG["PSAD56_UTM17S"]).transform(lon2, lat2)


def a_wgs84_utm(p: Punto) -> tuple[float, float]:
    """Lleva cualquier entrada soportada a WGS 84 / UTM 17S."""
    datum = (p.datum or "WGS84").upper()
    if p.tipo == "geo":
        if datum == "PSAD56":
            lon, lat = _psad56_a_wgs84_geo().transform(p.x, p.y)
        else:
            lon, lat = p.x, p.y
        return _transformador(EPSG["WGS84_GEO"], EPSG["WGS84_UTM17S"]).transform(lon, lat)
    if datum == "PSAD56":
        return psad56_utm_a_wgs84_utm(p.x, p.y)
    return p.x, p.y  # WGS84 o SIRGAS (equivalencia métrica)


def todas_las_representaciones(p: Punto) -> dict[str, tuple[float, float]]:
    e, n = a_wgs84_utm(p)
    lon, lat = _transformador(EPSG["WGS84_UTM17S"], EPSG["WGS84_GEO"]).transform(e, n)
    pe, pn = wgs84_utm_a_psad56_utm(e, n)
    return {"WGS84_UTM17S": (e, n), "WGS84_GEO": (lat, lon), "PSAD56_UTM17S": (pe, pn)}


def dms(valor: float, eje: str) -> str:
    hemis = ("N" if valor >= 0 else "S") if eje == "lat" else ("E" if valor >= 0 else "W")
    v = abs(valor)
    g = int(v)
    m = int((v - g) * 60)
    s = (v - g - m / 60) * 3600
    return f"{g}°{m:02d}'{s:05.2f}\"{hemis}"


def dentro_de_ecuador_utm17s(e: float, n: float) -> bool:
    """Control de rango aproximado para Ecuador continental expresado en UTM 17S."""
```

```
    return 450_000 <= e <= 1_100_000 and 9_440_000 <= n <= 10_170_000


def distancia(a: tuple[float, float], b: tuple[float, float]) -> float:
    return math.hypot(a[0] - b[0], a[1] - b[1])


def nombre_crs(epsg: int) -> str:
    return CRS.from_epsg(epsg).name
```

D.3 Base de conocimiento: catastro, operación, FAQ y tickets (kuribot/conocimiento.py)

```
"""Base de conocimiento de KuriBot: catastro sintético, operación, FAQ y tickets (SQLite)."""
from __future__ import annotations

import datetime as dt
import json
import sqlite3
from dataclasses import dataclass
from pathlib import Path

from shapely.geometry import Point, shape
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

from .entidades import plano

RAIZ = Path(__file__).resolve().parents[1]
DATOS = RAIZ / "datos"
FRANJA_M = 500.0      # franja de amortiguamiento (supuesto de diseño heredado de SEPRIM)
RADIO_COMUNIDAD_M = 1000.0


@dataclass
class AnalisisPunto:
    e: float
    n: float
    ubicacion: str                 # propia | amortiguamiento | tercero | fuera
    concesion: str | None          # código de la concesión que contiene el punto
    cercana: str | None            # concesión propia más cercana
    distancia_lindero_m: float     # distancia al lindero de la concesión que lo contiene o de la más cercana
    area_protegida: str | None
    comunidad: str | None
    distancia_comunidad_m: float | None


class BaseConocimiento:
    def __init__(self, ruta_datos: Path = DATOS, db: str = ":memory:"):
        geo = json.loads((ruta_datos / "concesiones.geojson").read_text(encoding="utf-8"))
        self.concesiones, self.areas, self.comunidades = {}, {}, {}
        for f in geo["features"]:
            p, g = f["properties"], shape(f["geometry"])
            if p["capa"] == "concesion":
                self.concesiones[p["codigo"]] = {**p, "geom": g, "area_ha": round(g.area / 1e4, 1)}
            elif p["capa"] == "area_protegida":
                self.areas[p["codigo"]] = {**p, "geom": g}
            else:
                self.comunidades[p["codigo"]] = {**p, "geom": g}
        self.op = json.loads((ruta_datos / "operacion.json").read_text(encoding="utf-8"))
        self.hoy = dt.date.fromisoformat(self.op["fecha_referencia"])
        self.faq = json.loads((ruta_datos / "faq.json").read_text(encoding="utf-8"))
        self.nombres = {plano(c["nombre"]): cod for cod, c in self.concesiones.items()}
        self._indexar_faq()
        self.db = sqlite3.connect(db, check_same_thread=False)
        self._crear_tickets(json.loads((ruta_datos / "tickets_semilla.json").read_text(encoding="utf-8")))

    # ------------------------------------------------------------- catastro
    def ficha(self, codigo: str) -> dict | None:
        c = self.concesiones.get(codigo)
        if not c:
            return None
        return {k: c[k] for k in ("codigo", "nombre", "fase", "estado", "campamento", "titular", "area_ha")} | {
            "alertas_90d": self.op["historial_alertas_90d"].get(codigo, 0),
            "actividad_programada": self.op["actividad_autorizada_programada"].get(codigo)}

    def analizar_punto(self, e: float, n: float) -> AnalisisPunto:
        pt = Point(e, n)
        contiene = [c for c in self.concesiones.values() if c["geom"].covers(pt)]
        propias = [c for c in self.concesiones.values() if c["titular"] == "propia"]
        cercana = min(propias, key=lambda c: c["geom"].distance(pt))
        if contiene:
            c = sorted(contiene, key=lambda c: c["titular"] != "propia")[0]
            ubic = "propia" if c["titular"] == "propia" else "tercero"
            cod, dist = c["codigo"], c["geom"].exterior.distance(pt)
        else:
            dist = cercana["geom"].distance(pt)
            ubic, cod = ("amortiguamiento" if dist <= FRANJA_M else "fuera"), None
        area = next((a["nombre"] for a in self.areas.values() if a["geom"].covers(pt)), None)
        com = min(self.comunidades.values(), key=lambda c: c["geom"].distance(pt)) if self.comunidades else None
        dcom = com["geom"].distance(pt) if com else None
        return AnalisisPunto(e, n, ubic, cod, cercana["codigo"], round(dist, 1), area,
                             com["nombre"] if com and dcom <= RADIO_COMUNIDAD_M else None,
                             round(dcom, 1) if dcom is not None else None)

    def campamento(self, codigo: str | None) -> str:
        c = self.concesiones.get(codigo or "")
        return c["campamento"] if c and c["campamento"] in self.op["pronostico"] else "Central"
```

```
    # ------------------------------------------------------------- FAQ (recuperación por similitud TF-IDF)
    def _indexar_faq(self):
        self._faq_textos, self._faq_ids = [], []
        for f in self.faq:
            for q in f["preguntas"]:
                self._faq_textos.append(q)
                self._faq_ids.append(f["id"])
        self._vec = TfidfVectorizer(analyzer="char_wb", ngram_range=(3, 5), lowercase=True, strip_accents="unicode")
        self._mat = self._vec.fit_transform(self._faq_textos)

    def buscar_faq(self, consulta: str) -> tuple[dict | None, float]:
        s = cosine_similarity(self._vec.transform([consulta]), self._mat)[0]
        i = int(s.argmax())
        f = next(x for x in self.faq if x["id"] == self._faq_ids[i])
        return f, float(s[i])

    # ------------------------------------------------------------- tickets
    def _crear_tickets(self, semilla):
        cur = self.db.cursor()
        cur.execute("CREATE TABLE IF NOT EXISTS tickets (id TEXT PRIMARY KEY, tipo TEXT, detalle TEXT, estado TEXT, "
                    "asignado TEXT, compromiso TEXT, resolucion TEXT, prioridad TEXT, creado TEXT)")
        for t in semilla:
            cur.execute("INSERT OR REPLACE INTO tickets VALUES (?,?,?,?,?,?,?,?,?)",
                        (t["id"], t["tipo"], t["detalle"], t["estado"], t["asignado"], t["compromiso"],
                         t.get("resolucion"), t.get("prioridad"), self.op["fecha_referencia"]))
        self.db.commit()

    def ticket(self, ident: str) -> dict | None:
        cur = self.db.execute("SELECT * FROM tickets WHERE id = ?", (ident,))
        fila = cur.fetchone()
        if not fila:
            return None
        return dict(zip([c[0] for c in cur.description], fila))

    def nuevo_ticket(self, tipo: str, detalle: str, asignado: str, compromiso: str | None, prioridad: str | None = None) -> str:
        ultimos = [int(r[0].split("-")[-1]) for r in self.db.execute("SELECT id FROM tickets")]
        ident = f"GEO-2026-{max(ultimos) + 1:04d}"
        self.db.execute("INSERT INTO tickets VALUES (?,?,?,?,?,?,?,?,?)",
                        (ident, tipo, detalle, "Registrado", asignado, compromiso, None, prioridad, self.op["fecha_referencia"]))
        self.db.commit()
        return ident

    def fecha(self, desfase: int) -> str:
        return (self.hoy + dt.timedelta(days=desfase)).isoformat()
```

D.4 Modelos de intenciones M0–M3, calibración y métricas (kuribot/nlu_modelos.py)

```
"""Modelos de clasificación de intenciones comparados en el proyecto.

M0  TF-IDF (palabras 1-2 + caracteres 2-5) + regresión logística   -> ML clásico (línea base)
M1  Embeddings fastText preentrenados (spaCy es_core_news_lg) + BiLSTM + atención + MLP
M2  BETO congelado (media de la última capa) + MLP
M3  BETO ajustado de extremo a extremo (fine-tuning) + capa lineal   -> modelo de producción

Todos reciben el mismo texto preprocesado (entidades enmascaradas) y las mismas particiones.
"""
from __future__ import annotations

import copy
import math
import random
import re
import time
from dataclasses import dataclass
from pathlib import Path

import numpy as np
import torch
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score
from sklearn.pipeline import FeatureUnion, Pipeline
from torch import nn

from .entidades import enmascarar_entidades

TOKEN = re.compile(r"[a-záéíóúüñ]+|\d+|[A-ZÁÉÍÓÚÑ]+", re.I)


def fijar_semilla(s: int) -> None:
    random.seed(s)
    np.random.seed(s)
    torch.manual_seed(s)


def preprocesar(textos: list[str]) -> list[str]:
    return [enmascarar_entidades(t) for t in textos]


def pesos_clase(y: np.ndarray, n: int) -> torch.Tensor:
    conteo = np.bincount(y, minlength=n).astype(float)
    w = conteo.sum() / (n * np.maximum(conteo, 1))
    return torch.tensor(w, dtype=torch.float32)
```

```
# ============================================================================ M0


def crear_m0(C: float = 10.0) -> Pipeline:
    feats = FeatureUnion([
        ("palabras", TfidfVectorizer(lowercase=True, strip_accents="unicode", ngram_range=(1, 2), sublinear_tf=True)),
        ("caracteres", TfidfVectorizer(lowercase=True, strip_accents="unicode", analyzer="char_wb",
                                       ngram_range=(2, 5), sublinear_tf=True)),
    ])
    return Pipeline([("tfidf", feats), ("lr", LogisticRegression(C=C, max_iter=4000, class_weight="balanced"))])


class ModeloM0:
    nombre = "M0 TF-IDF + RL"

    def __init__(self, n_clases: int, semilla: int = 0):
        self.n = n_clases
        self.pipe = crear_m0()

    def entrenar(self, xtr, ytr, xdev=None, ydev=None):
        self.pipe.fit(preprocesar(xtr), ytr)
        return {}

    def logits(self, textos):
        # log-probabilidades como logits: el softmax recupera las probabilidades de la RL
        return np.log(np.clip(self.pipe.predict_proba(preprocesar(textos)), 1e-12, 1))


# ============================================================================ utilidades de entrenamiento neuronal


def _entrenar_torch(modelo, lotes_tr, evaluar_dev, epocas, lr, pesos, paciencia, wd=0.01, calentamiento=0.0,
                    params=None):
    params = params if params is not None else [p for p in modelo.parameters() if p.requires_grad]
    opt = torch.optim.AdamW(params, lr=lr, weight_decay=wd)
    pasos_total = epocas * len(lotes_tr())
    pasos_cal = int(calentamiento * pasos_total)
    sched = torch.optim.lr_scheduler.LambdaLR(
        opt, lambda s: (s + 1) / max(1, pasos_cal) if s < pasos_cal else max(0.0, (pasos_total - s) / max(1, pasos_total -
pasos_cal)))
    perdida = nn.CrossEntropyLoss(weight=pesos, label_smoothing=0.05)
    mejor, mejor_estado, sin_mejora, historia = -1.0, None, 0, []
    for ep in range(epocas):
        modelo.train()
        total, n = 0.0, 0
        for entrada, y in lotes_tr():
            opt.zero_grad()
            salida = modelo(*entrada)
            l = perdida(salida, y)
            l.backward()
            torch.nn.utils.clip_grad_norm_(params, 1.0)
            opt.step()
            sched.step()
            total += float(l.detach()) * len(y)
            n += len(y)
        f1 = evaluar_dev() if evaluar_dev else float("nan")
        historia.append({"epoca": ep + 1, "perdida_entrenamiento": total / n, "f1_macro_validacion": f1})
        if evaluar_dev is None:
            continue
        if f1 > mejor + 1e-4:
            mejor, mejor_estado, sin_mejora = f1, copy.deepcopy(modelo.state_dict()), 0
        else:
            sin_mejora += 1
            if sin_mejora >= paciencia:
                break
    if mejor_estado is not None:
        modelo.load_state_dict(mejor_estado)
    return historia


# ============================================================================ M1


def tokenizar(texto: str) -> list[str]:
    return [t.lower() for t in TOKEN.findall(texto)]


class RedBiLSTM(nn.Module):
    def __init__(self, emb: np.ndarray, n_clases: int, oculto: int = 128, dropout: float = 0.3):
        super().__init__()
        self.embedding = nn.Embedding.from_pretrained(torch.tensor(emb, dtype=torch.float32), freeze=False, padding_idx=0)
        self.drop = nn.Dropout(dropout)
        self.lstm = nn.LSTM(emb.shape[1], oculto, batch_first=True, bidirectional=True)
        self.atencion = nn.Linear(2 * oculto, 1)
        self.mlp = nn.Sequential(nn.Linear(4 * oculto, 128), nn.ReLU(), nn.Dropout(dropout), nn.Linear(128, n_clases))

    def forward(self, ids, mascara):
        h, _ = self.lstm(self.drop(self.embedding(ids)))
        puntaje = self.atencion(h).squeeze(-1).masked_fill(~mascara, -1e9)
        alfa = torch.softmax(puntaje, dim=1).unsqueeze(-1)
        atendido = (alfa * h).sum(1)
        maximo = h.masked_fill(~mascara.unsqueeze(-1), -1e9).max(1).values
        return self.mlp(torch.cat([atendido, maximo], dim=1))


class ModeloM1:
    nombre = "M1 fastText + BiLSTM"
```

```
    def __init__(self, n_clases: int, semilla: int = 0, vectores=None, max_len: int = 32):
        self.n, self.semilla, self.max_len = n_clases, semilla, max_len
        self.vectores = vectores  # función token -> vector (np.ndarray) o None
        self.vocab: dict[str, int] = {}
        self.red: RedBiLSTM | None = None
        self.cobertura = float("nan")

    def _ids(self, textos):
        filas = [[self.vocab.get(t, 1) for t in tokenizar(x)][: self.max_len] or [1] for x in preprocesar(textos)]
        L = max(len(f) for f in filas)
        ids = torch.zeros(len(filas), L, dtype=torch.long)
        for i, f in enumerate(filas):
            ids[i, : len(f)] = torch.tensor(f)
        return ids, ids != 0

    def entrenar(self, xtr, ytr, xdev, ydev):
        fijar_semilla(self.semilla)
        toks = sorted({t for x in preprocesar(xtr) for t in tokenizar(x)})
        self.vocab = {"<pad>": 0, "<unk>": 1, **{t: i + 2 for i, t in enumerate(toks)}}
        emb = np.random.default_rng(self.semilla).normal(0, 0.1, (len(self.vocab), 300)).astype(np.float32)
        emb[0] = 0
        cubiertos = 0
        if self.vectores is not None:
            for t, i in self.vocab.items():
                v = self.vectores(t) if i > 1 else None
                if v is not None:
                    emb[i], cubiertos = v, cubiertos + 1
        self.cobertura = cubiertos / max(1, len(self.vocab) - 2)
        self.red = RedBiLSTM(emb, self.n)
        ytr_t = torch.tensor(ytr)
        idx = np.arange(len(xtr))

        epoca = [0]

        def lotes_lista():
            rng = np.random.default_rng(self.semilla + epoca[0])
            epoca[0] += 1
            orden = rng.permutation(idx)
            salida = []
            for i in range(0, len(orden), 32):
                b = orden[i:i + 32]
                ids, m = self._ids([xtr[j] for j in b])
                salida.append(((ids, m), ytr_t[b]))
            return salida

        def dev():
            return f1_score(ydev, self.logits(xdev).argmax(1), average="macro")

        hist = _entrenar_torch(self.red, lotes_lista, dev, epocas=40, lr=2e-3, pesos=pesos_clase(ytr, self.n), paciencia=6)
        return {"historia": hist, "cobertura_embeddings": self.cobertura, "vocabulario": len(self.vocab)}

    @torch.no_grad()
    def logits(self, textos):
        self.red.eval()
        salida = []
        for i in range(0, len(textos), 64):
            ids, m = self._ids(textos[i:i + 64])
            salida.append(self.red(ids, m).numpy())
        return np.concatenate(salida)


# ============================================================================ BETO (M2 y M3)


class CodificadorBETO(nn.Module):
    def __init__(self, ruta: str | Path | None = None, config=None):
        super().__init__()
        from transformers import BertModel
        # Con 'config' se crea la arquitectura vacía (los pesos se cargan después desde el modelo exportado).
        self.bert = (BertModel(config, add_pooling_layer=False) if config is not None
                     else BertModel.from_pretrained(str(ruta), add_pooling_layer=False))

    def forward(self, input_ids, attention_mask, token_type_ids=None):
        h = self.bert(input_ids=input_ids, attention_mask=attention_mask).last_hidden_state
        m = attention_mask.unsqueeze(-1).float()
        return (h * m).sum(1) / m.sum(1).clamp_min(1.0)


class ClasificadorBETO(nn.Module):
    def __init__(self, ruta, n_clases: int, dropout: float = 0.1, config=None):
        super().__init__()
        self.cod = CodificadorBETO(ruta, config)
        self.drop = nn.Dropout(dropout)
        self.salida = nn.Linear(768, n_clases)

    def forward(self, input_ids, attention_mask):
        return self.salida(self.drop(self.cod(input_ids, attention_mask)))


def _tok(ruta):
    from transformers import BertTokenizerFast
    return BertTokenizerFast.from_pretrained(str(ruta))


class ModeloM2:
    nombre = "M2 BETO congelado + MLP"
```

```
    _cache: dict = {}

    def __init__(self, n_clases: int, semilla: int = 0, ruta_beto: str = "modelos/beto_base"):
        self.n, self.semilla, self.ruta = n_clases, semilla, ruta_beto
        self.tok = _tok(ruta_beto)
        if ruta_beto not in ModeloM2._cache:
            ModeloM2._cache[ruta_beto] = (CodificadorBETO(ruta_beto).eval(), {})
        self.cod, self.memo = ModeloM2._cache[ruta_beto]
        self.mlp = None

    @torch.no_grad()
    def _vec(self, textos):
        pre = preprocesar(textos)
        faltan = [t for t in dict.fromkeys(pre) if t not in self.memo]
        for i in range(0, len(faltan), 32):
            enc = self.tok(faltan[i:i + 32], padding=True, truncation=True, max_length=48, return_tensors="pt")
            v = self.cod(enc["input_ids"], enc["attention_mask"])
            for t, fila in zip(faltan[i:i + 32], v):
                self.memo[t] = fila
        return torch.stack([self.memo[t] for t in pre])

    def entrenar(self, xtr, ytr, xdev, ydev):
        fijar_semilla(self.semilla)
        Xtr = self._vec(xtr)
        # Estandarización con media y desviación de entrenamiento: las salidas de BERT son muy anisótropas
        # (coseno medio ≈ 0,9 entre frases), lo que impide aprender a una MLP sin normalizar.
        self.mu, self.sd = Xtr.mean(0), Xtr.std(0).clamp_min(1e-4)
        Xtr = (Xtr - self.mu) / self.sd
        Xdev = (self._vec(xdev) - self.mu) / self.sd
        self.mlp = nn.Sequential(nn.Dropout(0.1), nn.Linear(768, 256), nn.ReLU(), nn.Dropout(0.2), nn.Linear(256, self.n))
        ytr_t = torch.tensor(ytr)

        def lotes():
            orden = torch.randperm(len(ytr))
            return [((Xtr[orden[i:i + 32]],), ytr_t[orden[i:i + 32]]) for i in range(0, len(ytr), 32)]

        def dev():
            self.mlp.eval()
            with torch.no_grad():
                return f1_score(ydev, self.mlp(Xdev).argmax(1).numpy(), average="macro")

        hist = _entrenar_torch(self.mlp, lotes, dev, epocas=150, lr=1e-3, pesos=pesos_clase(ytr, self.n), paciencia=15)
        return {"historia": hist}

    @torch.no_grad()
    def logits(self, textos):
        self.mlp.eval()
        return self.mlp((self._vec(textos) - self.mu) / self.sd).numpy()


class ModeloM3:
    nombre = "M3 BETO ajustado (fine-tuning)"

    def __init__(self, n_clases: int, semilla: int = 0, ruta_beto: str = "modelos/beto_base", epocas: int = 6,
                 lr: float = 3e-5, lote: int = 16, paciencia: int = 2):
        self.n, self.semilla, self.ruta = n_clases, semilla, ruta_beto
        self.epocas, self.lr, self.lote, self.paciencia = epocas, lr, lote, paciencia
        self.tok = _tok(ruta_beto)
        self.red = None

    def _enc(self, textos):
        return self.tok(preprocesar(textos), padding=True, truncation=True, max_length=48, return_tensors="pt")

    def entrenar(self, xtr, ytr, xdev, ydev):
        fijar_semilla(self.semilla)
        self.red = ClasificadorBETO(self.ruta, self.n)
        ytr_t = torch.tensor(ytr)

        def lotes():
            orden = torch.randperm(len(ytr)).tolist()
            out = []
            for i in range(0, len(orden), self.lote):
                b = orden[i:i + self.lote]
                enc = self._enc([xtr[j] for j in b])
                out.append(((enc["input_ids"], enc["attention_mask"]), ytr_t[b]))
            return out

        def dev():
            return f1_score(ydev, self.logits(xdev).argmax(1), average="macro") if xdev is not None else float("nan")

        hist = _entrenar_torch(self.red, lotes, dev if xdev is not None else None, epocas=self.epocas, lr=self.lr,
                               pesos=pesos_clase(ytr, self.n), paciencia=self.paciencia, wd=0.01, calentamiento=0.1)
        return {"historia": hist}

    @torch.no_grad()
    def logits(self, textos):
        self.red.eval()
        salida = []
        for i in range(0, len(textos), 32):
            enc = self._enc(textos[i:i + 32])
            salida.append(self.red(enc["input_ids"], enc["attention_mask"]).numpy())
        return np.concatenate(salida)


# ============================================================================ calibración y métricas
```

```
def softmax(z: np.ndarray, T: float = 1.0) -> np.ndarray:
    z = z / T
    z = z - z.max(1, keepdims=True)
    e = np.exp(z)
    return e / e.sum(1, keepdims=True)


def ajustar_temperatura(logits_dev: np.ndarray, y_dev: np.ndarray) -> float:
    """Escalado de temperatura (Guo et al., 2017): minimiza la log-verosimilitud negativa en validación."""
    z = torch.tensor(logits_dev, dtype=torch.float64)
    y = torch.tensor(y_dev)
    logT = torch.zeros(1, dtype=torch.float64, requires_grad=True)
    opt = torch.optim.LBFGS([logT], lr=0.1, max_iter=200)

    def cierre():
        opt.zero_grad()
        l = nn.functional.cross_entropy(z / logT.exp(), y)
        l.backward()
        return l

    opt.step(cierre)
    return float(logT.detach().exp().clamp(0.05, 20))


def ece(probs: np.ndarray, y: np.ndarray, bins: int = 10) -> float:
    conf, pred = probs.max(1), probs.argmax(1)
    bordes = np.linspace(0, 1, bins + 1)
    total = 0.0
    for a, b in zip(bordes[:-1], bordes[1:]):
        m = (conf > a) & (conf <= b)
        if m.any():
            total += m.mean() * abs((pred[m] == y[m]).mean() - conf[m].mean())
    return float(total)


def latencia_ms(modelo, textos: list[str], repeticiones: int = 1) -> dict:
    tiempos = []
    for _ in range(repeticiones):
        for t in textos:
            t0 = time.perf_counter()
            modelo.logits([t])
            tiempos.append((time.perf_counter() - t0) * 1000)
    a = np.array(tiempos)
    return {"media_ms": float(a.mean()), "p50_ms": float(np.percentile(a, 50)), "p95_ms": float(np.percentile(a, 95))}


def n_parametros(modelo) -> int:
    if isinstance(modelo, ModeloM0):
        lr = modelo.pipe.named_steps["lr"]
        return int(lr.coef_.size + lr.intercept_.size)
    if isinstance(modelo, ModeloM1):
        return sum(p.numel() for p in modelo.red.parameters())
    if isinstance(modelo, ModeloM2):
        return sum(p.numel() for p in modelo.mlp.parameters())
    if isinstance(modelo, ModeloM3):
        return sum(p.numel() for p in modelo.red.parameters())
    return -1


def wilson(k: int, n: int, z: float = 1.96) -> tuple[float, float]:
    if n == 0:
        return (math.nan, math.nan)
    p = k / n
    d = 1 + z * z / n
    c = p + z * z / (2 * n)
    r = z * math.sqrt(p * (1 - p) / n + z * z / (4 * n * n))
    return ((c - r) / d, (c + r) / d)
```

D.5 Inferencia de intenciones y decisión aceptar, aclarar o abstenerse (kuribot/nlu.py)

```
"""Inferencia de intenciones para el ChatBot (carga el modelo exportado por experimentos/entrenar_nlp.py)."""
from __future__ import annotations

import json
import time
from dataclasses import dataclass
from pathlib import Path

import numpy as np

from .entidades import enmascarar_entidades

RAIZ = Path(__file__).resolve().parents[1]
MODELOS = RAIZ / "modelos"


@dataclass
class Prediccion:
    intencion: str
    confianza: float
    top: list[tuple[str, float]]
    decision: str          # aceptar | aclarar | abstener
    modelo: str
    latencia_ms: float
```

```
def _softmax(z: np.ndarray, T: float) -> np.ndarray:
    z = z / T
    z = z - z.max()
    e = np.exp(z)
    return e / e.sum()


class ClasificadorIntenciones:
    """Envuelve el BETO ajustado (preferido) o la línea base TF-IDF si no hay pesos disponibles."""

    def __init__(self, ruta: str | Path | None = None, umbral_aclarar: float | None = None, umbral_abstener: float = 0.30):
        ruta = Path(ruta) if ruta else self._ultimo("intenciones_beto_v*")
        if ruta is not None and (ruta / "clasificador.safetensors").exists():
            self._cargar_beto(ruta)
        else:
            base = self._ultimo("intenciones_tfidf_v*.joblib")
            if base is None:
                raise FileNotFoundError("No hay modelos entrenados en modelos/. Ejecute experimentos/entrenar_nlp.py")
            self._cargar_tfidf(base)
        self.umbral_aclarar = umbral_aclarar if umbral_aclarar is not None else max(0.5, self.tau)
        self.umbral_abstener = umbral_abstener
        self._logits("hola")  # calentamiento: la primera inferencia de PyTorch tras cargar el modelo es lenta

    @staticmethod
    def _ultimo(patron: str) -> Path | None:
        c = sorted(MODELOS.glob(patron))
        return c[-1] if c else None

    def _cargar_beto(self, ruta: Path):
        import torch
        from safetensors.torch import load_file
        from transformers import BertConfig, BertTokenizerFast

        from .nlu_modelos import ClasificadorBETO

        meta = json.loads((ruta / "kuribot_nlu.json").read_text(encoding="utf-8"))
        self.etiquetas, self.T, self.tau = meta["etiquetas"], meta["temperatura"], meta["umbral_confianza"]
        self.max_len = meta.get("max_len", 48)
        cfg = BertConfig.from_json_file(str(ruta / "bert_config.json"))
        red = ClasificadorBETO(None, len(self.etiquetas), config=cfg)
        estado = {k: v.float() for k, v in load_file(str(ruta / "clasificador.safetensors")).items()}
        red.load_state_dict(estado)
        red.eval()
        self.red, self.tok, self.torch = red, BertTokenizerFast.from_pretrained(str(ruta)), torch
        self.nombre = f"BETO ajustado ({ruta.name})"
        self._logits = self._logits_beto

    def _cargar_tfidf(self, ruta: Path):
        import joblib
        d = joblib.load(ruta)
        self.pipe, self.etiquetas, self.T, self.tau = d["pipe"], d["etiquetas"], d["temperatura"], 0.5
        self.nombre = f"TF-IDF + RL ({ruta.name}, modo de respaldo)"
        self._logits = lambda t: np.log(np.clip(self.pipe.predict_proba([t])[0], 1e-12, 1))

    def _logits_beto(self, texto: str) -> np.ndarray:
        with self.torch.no_grad():
            enc = self.tok([texto], truncation=True, max_length=self.max_len, return_tensors="pt")
            return self.red(enc["input_ids"], enc["attention_mask"])[0].numpy()

    def predecir(self, texto: str) -> Prediccion:
        t0 = time.perf_counter()
        p = _softmax(self._logits(enmascarar_entidades(texto)), self.T)
        orden = np.argsort(-p)
        top = [(self.etiquetas[i], float(p[i])) for i in orden[:3]]
        conf = top[0][1]
        if conf >= self.umbral_aclarar:
            decision = "aceptar"
        elif conf >= self.umbral_abstener and top[0][1] + top[1][1] >= 0.70:
            decision = "aclarar"
        else:
            decision = "abstener"
        return Prediccion(top[0][0], conf, top, decision, self.nombre, (time.perf_counter() - t0) * 1000)
```

D.6 Motor de reglas, red bayesiana y ventanas de vuelo (kuribot/razonamiento.py)

```
"""Componente de razonamiento de KuriBot (contenidos de las semanas 2 y 3).

1. MotorReglas: sistema de producción con encadenamiento hacia adelante, agenda por nivel y
   prioridad, refracción (cada regla dispara una vez) y traza explicativa. Es la continuación
   del sistema experto SEPRIM de la semana 2, ahora alimentado desde la conversación.
2. RedAlertas: red bayesiana discreta (9 nodos) que estima P(intrusión real | evidencias) con
   eliminación de variables exacta (pgmpy). Las tablas de probabilidad condicional (TPC) son
   juicios expertos del equipo, no frecuencias estimadas: deben recalibrarse con el histórico
   de verificaciones de campo.
3. Factibilidad de vuelo: reglas operativas + búsqueda de la primera ventana apta.
"""
from __future__ import annotations

import itertools
from dataclasses import dataclass, field
from typing import Callable

import numpy as np

# ============================================================================ motor de reglas
```

```
@dataclass
class Regla:
    id: str
    nivel: int
    prioridad: int
    descripcion: str
    si: Callable[[dict], bool]
    entonces: Callable[[dict], dict]
    texto_si: str = ""
    texto_entonces: str = ""


@dataclass
class ResultadoInferencia:
    hechos: dict
    disparadas: list[str] = field(default_factory=list)
    traza: list[str] = field(default_factory=list)


class MotorReglas:
    def __init__(self, reglas: list[Regla]):
        ids = [r.id for r in reglas]
        if len(ids) != len(set(ids)):
            raise ValueError("Identificadores de regla duplicados")
        self.reglas = sorted(reglas, key=lambda r: (r.nivel, -r.prioridad, r.id))

    def inferir(self, hechos: dict, max_ciclos: int = 100) -> ResultadoInferencia:
        wm = dict(hechos)
        res = ResultadoInferencia(wm)
        disparadas: set[str] = set()
        for _ in range(max_ciclos):
            agenda = [r for r in self.reglas if r.id not in disparadas and _seguro(r.si, wm)]
            if not agenda:
                break
            r = agenda[0]  # resolución de conflictos: menor nivel, mayor prioridad, id
            nuevos = r.entonces(wm)
            wm.update(nuevos)
            disparadas.add(r.id)
            res.disparadas.append(r.id)
            res.traza.append(f"{r.id} (nivel {r.nivel}): {r.descripcion}")
        res.hechos = wm
        return res


def _seguro(cond, wm) -> bool:
    try:
        return bool(cond(wm))
    except (KeyError, TypeError):
        return False


def _sin_prioridad(f):
    return "prioridad" not in f


UMBRALES = {"critica": 0.80, "alta": 0.50, "media": 0.15}  # supuestos de diseño (ver informe, sección 5.4)

REGLAS_ALERTA: list[Regla] = [
    # ---------------- nivel 0: seguridad, privacidad, autonomía
    Regla("R01", 0, 100, "Riesgo para personas: protocolo de emergencia, sin despliegue de personal ni dron",
          lambda f: f.get("emergencia"), lambda f: {"prioridad": "CRÍTICA", "accion": "PROTOCOLO DE EMERGENCIA",
                                                    "prohibir_despliegue": True, "derivar_humano": True},
          "emergencia = verdadero", "prioridad = CRÍTICA; acción = protocolo; derivar a humano"),
    Regla("R02", 0, 90, "Datos personales detectados: se enmascaran en el registro",
          lambda f: f.get("pii"), lambda f: {"enmascarar_registro": True}, "pii = verdadero", "enmascarar registro"),
    Regla("R03", 0, 80, "Solicitud explícita de atención humana",
          lambda f: f.get("intencion") == "hablar_humano", lambda f: {"derivar_humano": True},
          "intención = hablar_humano", "derivar a humano"),
    # ---------------- nivel 1: contexto geoespacial
    Regla("R10", 1, 50, "El punto está dentro de una concesión propia",
          lambda f: f.get("ubicacion") == "propia", lambda f: {"zona": "concesión propia"}, "ubicación = propia", "zona =
concesión propia"),
    Regla("R11", 1, 50, "El punto está en la franja de 500 m alrededor de una concesión propia",
          lambda f: f.get("ubicacion") == "amortiguamiento", lambda f: {"zona": "franja de amortiguamiento"},
          "ubicación = amortiguamiento", "zona = amortiguamiento"),
    Regla("R12", 1, 60, "El punto está en una concesión de un tercero con derecho vigente",
          lambda f: f.get("ubicacion") == "tercero", lambda f: {"derecho_tercero": True}, "ubicación = tercero",
"derecho_tercero"),
    Regla("R13", 1, 40, "El punto está dentro de un área protegida",
          lambda f: f.get("area_protegida"), lambda f: {"area_sensible": True}, "área protegida", "área sensible"),
    Regla("R14", 1, 40, "Hay una comunidad a 1 km o menos",
          lambda f: f.get("comunidad_cercana"), lambda f: {"sensibilidad_social": True}, "comunidad ≤ 1 km", "sensibilidad
social"),
    Regla("R15", 1, 30, "El punto está fuera del área de responsabilidad de la empresa",
          lambda f: f.get("ubicacion") == "fuera", lambda f: {"fuera_jurisdiccion": True}, "ubicación = fuera", "fuera de
jurisdicción"),
    # ---------------- nivel 2: priorización con el posterior bayesiano
    Regla("R20", 2, 100, "Derecho vigente de un tercero: no se califica como intrusión; revisión legal",
          lambda f: _sin_prioridad(f) and f.get("derecho_tercero"), lambda f: {"prioridad": "REVISIÓN LEGAL"},
          "derecho_tercero y sin prioridad", "prioridad = REVISIÓN LEGAL"),
    Regla("R25", 2, 90, "Fuera del área de la empresa: registro informativo",
          lambda f: _sin_prioridad(f) and f.get("fuera_jurisdiccion"), lambda f: {"prioridad": "INFORMATIVA"},
          "fuera_jurisdicción y sin prioridad", "prioridad = INFORMATIVA"),
    Regla("R21", 2, 80, "P(intrusión) ≥ 0,80",
          lambda f: _sin_prioridad(f) and f["p_intrusion"] >= UMBRALES["critica"], lambda f: {"prioridad": "CRÍTICA"},
          "P ≥ 0,80 y sin prioridad", "prioridad = CRÍTICA"),
    Regla("R22", 2, 70, "0,50 ≤ P(intrusión) < 0,80",
          lambda f: _sin_prioridad(f) and f["p_intrusion"] >= UMBRALES["alta"], lambda f: {"prioridad": "ALTA"},
```

```
          "P ≥ 0,50 y sin prioridad", "prioridad = ALTA"),
    Regla("R23", 2, 60, "0,15 ≤ P(intrusión) < 0,50",
          lambda f: _sin_prioridad(f) and f["p_intrusion"] >= UMBRALES["media"], lambda f: {"prioridad": "MEDIA"},
          "P ≥ 0,15 y sin prioridad", "prioridad = MEDIA"),
    Regla("R24", 2, 50, "P(intrusión) < 0,15",
          lambda f: _sin_prioridad(f) and "p_intrusion" in f, lambda f: {"prioridad": "BAJA"},
          "sin prioridad (regla por defecto)", "prioridad = BAJA"),
    # ---------------- nivel 3: acción operativa
    Regla("R30", 3, 60, "CRÍTICA con ventana de vuelo hoy o mañana: vuelo prioritario y aviso a Seguridad y Legal",
          lambda f: f.get("prioridad") == "CRÍTICA" and not f.get("emergencia") and f.get("vuelo_dias") is not None and
f["vuelo_dias"] <= 1,
          lambda f: {"accion": "Vuelo de verificación prioritario (≤ 24 h) y notificación a Seguridad y Legal"},
          "CRÍTICA y ventana ≤ 1 día", "vuelo prioritario"),
    Regla("R31", 3, 55, "CRÍTICA sin ventana de vuelo inmediata: verificación satelital y patrullaje con seguridad",
          lambda f: f.get("prioridad") == "CRÍTICA" and not f.get("emergencia") and "accion" not in f,
          lambda f: {"accion": "Revisión satelital inmediata y patrullaje con Seguridad; vuelo en la primera ventana apta"},
          "CRÍTICA y sin ventana ≤ 1 día", "satélite + patrullaje"),
    Regla("R32", 3, 50, "ALTA: vuelo en la primera ventana apta (≤ 72 h)",
          lambda f: f.get("prioridad") == "ALTA", lambda f: {"accion": "Programar vuelo de verificación en la primera ventana
apta (≤ 72 h)"},
          "ALTA", "vuelo ≤ 72 h"),
    Regla("R33", 3, 40, "MEDIA: revisión satelital y verificación en ronda programada",
          lambda f: f.get("prioridad") == "MEDIA", lambda f: {"accion": "Revisión satelital en el próximo ciclo y verificación en
la ronda programada"},
          "MEDIA", "revisión satelital"),
    Regla("R34", 3, 30, "BAJA: registrar y monitorear",
          lambda f: f.get("prioridad") == "BAJA", lambda f: {"accion": "Registrar y mantener monitoreo"}, "BAJA", "registro"),
    Regla("R35", 3, 70, "REVISIÓN LEGAL: Legal y Catastro cotejan derechos del titular vecino",
          lambda f: f.get("prioridad") == "REVISIÓN LEGAL",
          lambda f: {"accion": "Derivar a Legal y Catastro para cotejar los derechos del titular vecino"}, "REVISIÓN LEGAL",
"derivar a Legal"),
    Regla("R36", 3, 65, "INFORMATIVA: Legal evalúa si corresponde informar a la autoridad",
          lambda f: f.get("prioridad") == "INFORMATIVA",
          lambda f: {"accion": "Registrar; Legal evalúa si corresponde informar a la autoridad competente"}, "INFORMATIVA",
"registro + Legal"),
    # ---------------- nivel 4: salvaguardas con supervisión humana
    Regla("R40", 4, 50, "Área protegida o comunidad cercana: revisión humana y protocolo comunitario antes de actuar",
          lambda f: f.get("area_sensible") or f.get("sensibilidad_social"),
          lambda f: {"revision_humana": True, "protocolo_comunitario": True}, "área sensible o comunidad", "revisión humana"),
    Regla("R41", 4, 40, "Acción de campo (CRÍTICA o ALTA): requiere aprobación del supervisor de turno",
          lambda f: f.get("prioridad") in ("CRÍTICA", "ALTA") and not f.get("emergencia"),
          lambda f: {"aprobacion_supervisor": True}, "CRÍTICA o ALTA", "aprobación humana"),
    Regla("R42", 4, 30, "Toda alerta clasificada genera un ticket trazable",
          lambda f: "prioridad" in f, lambda f: {"crear_ticket": True}, "prioridad asignada", "crear ticket"),
]


def motor_alertas() -> MotorReglas:
    return MotorReglas(REGLAS_ALERTA)


def motor_contexto() -> MotorReglas:
    """Solo reglas de contexto geoespacial y salvaguarda (para verificar puntos, sin priorizar alertas)."""
    return MotorReglas([r for r in REGLAS_ALERTA if r.nivel == 1 or r.id == "R40"])


# ============================================================================ red bayesiana

NODOS = ["H", "A", "I", "M", "P", "S", "R", "T", "V"]
DESCRIPCION_NODOS = {
    "H": "Antecedentes: alertas en el sector en los últimos 90 días",
    "A": "Actividad autorizada programada en el área",
    "I": "Intrusión o actividad minera no autorizada real (variable de interés)",
    "M": "Maquinaria o equipos observados",
    "P": "Personas o campamento no identificados",
    "S": "Alteración del terreno o de la cobertura",
    "R": "Ruido o voladuras nocturnas",
    "T": "Agua turbia aguas abajo",
    "V": "Vía o trocha nueva",
}
ARISTAS = [("H", "I"), ("I", "M"), ("A", "M"), ("I", "P"), ("A", "P"), ("I", "S"), ("A", "S"), ("I", "V"), ("A", "V"),
           ("I", "R"), ("I", "T")]

# TPC elicitadas: P(X = 1 | padres). Para nodos con padres (I, A) el orden es (I=0,A=0), (0,1), (1,0), (1,1).
TPC = {
    "H": 0.30,
    "A": 0.40,
    "I": {(0,): 0.10, (1,): 0.30},
    "M": {(0, 0): 0.03, (0, 1): 0.60, (1, 0): 0.70, (1, 1): 0.85},
    "P": {(0, 0): 0.10, (0, 1): 0.50, (1, 0): 0.75, (1, 1): 0.85},
    "S": {(0, 0): 0.05, (0, 1): 0.45, (1, 0): 0.65, (1, 1): 0.80},
    "V": {(0, 0): 0.03, (0, 1): 0.20, (1, 0): 0.40, (1, 1): 0.45},
    "R": {(0,): 0.05, (1,): 0.40},
    "T": {(0,): 0.15, (1,): 0.40},
}
PADRES = {"H": (), "A": (), "I": ("H",), "M": ("I", "A"), "P": ("I", "A"), "S": ("I", "A"), "V": ("I", "A"),
          "R": ("I",), "T": ("I",)}


class RedAlertas:
    """Red bayesiana de credibilidad de alertas, con inferencia exacta en pgmpy."""

    def __init__(self, tpc: dict | None = None):
        self.tpc = tpc or TPC
        self.modelo = self._construir()
        from pgmpy.inference import VariableElimination
        self.inf = VariableElimination(self.modelo)
```

```
    def _construir(self):
        try:
            from pgmpy.models import DiscreteBayesianNetwork as BN
        except ImportError:  # pgmpy < 1.0
            from pgmpy.models import BayesianNetwork as BN
        from pgmpy.factors.discrete import TabularCPD
        m = BN(ARISTAS)
        for nodo in NODOS:
            pad = PADRES[nodo]
            if not pad:
                p1 = self.tpc[nodo]
                cpd = TabularCPD(nodo, 2, [[1 - p1], [p1]])
            else:
                combos = list(itertools.product([0, 1], repeat=len(pad)))
                fila1 = [self.tpc[nodo][c] for c in combos]
                cpd = TabularCPD(nodo, 2, [[1 - v for v in fila1], fila1], evidence=list(pad), evidence_card=[2] * len(pad))
            m.add_cpds(cpd)
        if not m.check_model():
            raise ValueError("Red bayesiana inválida")
        return m

    def posterior(self, evidencia: dict[str, int]) -> float:
        ev = {k: int(v) for k, v in evidencia.items() if k in NODOS and k != "I"}
        q = self.inf.query(["I"], evidence=ev, show_progress=False)
        return float(q.values[1])

    def posterior_enumeracion(self, evidencia: dict[str, int]) -> float:
        """Verificación independiente por enumeración completa de la distribución conjunta."""
        return posterior_enumeracion(self.tpc, evidencia)

    def contribuciones(self, evidencia: dict[str, int]) -> list[tuple[str, float]]:
        """Cambio del posterior al retirar cada evidencia (explicación local tipo «dejar uno fuera»)."""
        base = self.posterior(evidencia)
        salida = []
        for k in evidencia:
            if k in ("H", "A"):
                continue
            resto = {kk: vv for kk, vv in evidencia.items() if kk != k}
            salida.append((k, base - self.posterior(resto)))
        return sorted(salida, key=lambda t: -abs(t[1]))


def posterior_enumeracion(tpc: dict, evidencia: dict[str, int]) -> float:
    num = den = 0.0
    libres = [n for n in NODOS if n not in evidencia]
    for valores in itertools.product([0, 1], repeat=len(libres)):
        asig = dict(evidencia)
        asig.update(dict(zip(libres, valores)))
        p = 1.0
        for nodo in NODOS:
            pad = PADRES[nodo]
            p1 = tpc[nodo] if not pad else tpc[nodo][tuple(asig[x] for x in pad)]
            p *= p1 if asig[nodo] == 1 else 1 - p1
        den += p
        if asig["I"] == 1:
            num += p
    return num / den


def sensibilidad(evidencia: dict[str, int], delta: float = 0.10) -> list[dict]:
    """Análisis de sensibilidad univariante: perturba ±delta cada parámetro de las TPC."""
    base = posterior_enumeracion(TPC, evidencia)
    filas = []
    for nodo, val in TPC.items():
        entradas = [((), val)] if not isinstance(val, dict) else list(val.items())
        for clave, p in entradas:
            extremos = []
            for signo in (-1, 1):
                tpc = {k: (dict(v) if isinstance(v, dict) else v) for k, v in TPC.items()}
                nuevo = float(np.clip(p + signo * delta, 0.01, 0.99))
                if isinstance(val, dict):
                    tpc[nodo][clave] = nuevo
                else:
                    tpc[nodo] = nuevo
                extremos.append(posterior_enumeracion(tpc, evidencia))
            bajo, alto = extremos
            cond = ",".join(f"{pa}={v}" for pa, v in zip(PADRES[nodo], clave))
            filas.append({"parametro": f"P({nodo}=1 | {cond})" if clave else f"P({nodo}=1)",
                          "bajo": bajo, "alto": alto, "rango": abs(alto - bajo), "base": base})
    return sorted(filas, key=lambda f: -f["rango"])


# ============================================================================ factibilidad de vuelo


def evaluar_dia(dia: int, campamento: str, op: dict, urgente: bool) -> tuple[bool, list[str], dict]:
    pol = op["politica_vuelo"]
    viento, lluvia, vis = op["pronostico"][campamento][str(dia)]
    drones = [d for d, v in op["drones"].items() if str(dia) not in v["no_disponible"]]
    pilotos = [p for p, v in op["pilotos"].items() if str(dia) not in v["no_disponible"]]
    motivos = []
    if viento > pol["viento_max_ms"]:
        motivos.append(f"V01 viento {viento:.1f} m/s > {pol['viento_max_ms']:.0f} m/s".replace(".", ","))
    if lluvia > pol["lluvia_max_mmh"]:
        motivos.append(f"V02 lluvia {lluvia:.1f} mm/h".replace(".", ","))
    if vis < pol["visibilidad_min_km"]:
        motivos.append(f"V03 visibilidad {vis:.0f} km < {pol['visibilidad_min_km']:.0f} km")
    if not drones:
```

```
        motivos.append("V04 sin dron disponible")
    if not pilotos:
        motivos.append("V05 sin piloto disponible")
    if not urgente and dia < pol["anticipacion_min_dias"]:
        motivos.append("V06 anticipación menor a 1 día para solicitud no urgente")
    return (not motivos), motivos, {"viento": viento, "lluvia": lluvia, "visibilidad": vis,
                                    "dron": drones[0] if drones else None, "piloto": pilotos[0] if pilotos else None}


def primera_ventana(desde: int, campamento: str, op: dict, urgente: bool = False) -> dict:
    """Búsqueda secuencial de la primera ventana apta dentro del horizonte de pronóstico."""
    evaluados = []
    for d in range(max(0, desde), op["politica_vuelo"]["horizonte_dias"] + 1):
        apto, motivos, datos = evaluar_dia(d, campamento, op, urgente)
        evaluados.append({"dia": d, "apto": apto, "motivos": motivos, **datos})
        if apto:
            return {"dia": d, "evaluados": evaluados, **datos}
    return {"dia": None, "evaluados": evaluados}
```

D.7 Gestor de diálogo y generación de respuestas (kuribot/dialogo.py)

```
"""Gestor de diálogo de KuriBot: estado, llenado de ranuras, políticas y generación de respuestas.

Flujo por mensaje: (1) enmascarar datos personales; (2) red de seguridad de emergencias;
(3) extraer entidades; (4) completar una acción pendiente o (5) clasificar la intención;
(6) aplicar la política de la intención (consultas, geoprocesos, red bayesiana, reglas);
(7) responder con plantillas y devolver una traza auditable.
"""
from __future__ import annotations

import json
import re
import time
import warnings
from dataclasses import asdict, dataclass, field
from pathlib import Path

from . import entidades as E
from . import geodesia as G
from .conocimiento import BaseConocimiento
from .nlu import ClasificadorIntenciones
from .razonamiento import RedAlertas, motor_alertas, motor_contexto, primera_ventana

warnings.filterwarnings("ignore", category=FutureWarning)

NOMBRES_INTENCION = {
    "saludo": "saludar", "despedida": "despedirse", "ayuda_capacidades": "conocer qué puedo hacer",
    "consultar_concesion": "consultar los datos de una concesión", "verificar_punto": "verificar si un punto cae en una
concesión",
    "convertir_coordenadas": "convertir coordenadas entre sistemas", "reportar_alerta": "reportar una posible actividad ilegal",
    "solicitar_vuelo": "solicitar un vuelo de dron", "solicitar_cartografia": "pedir un producto cartográfico",
    "estado_solicitud": "consultar el estado de una solicitud", "consulta_geodesia": "resolver una duda geodésica",
    "hablar_humano": "hablar con una persona", "emergencia_seguridad": "reportar una emergencia",
    "fuera_alcance": "un tema fuera de mi alcance",
}
CAPACIDADES = ("Puedo ayudarle a: consultar una concesión (p. ej., «estado de CM-103»); verificar si un punto cae dentro "
               "de una concesión; convertir coordenadas WGS 84 ↔ PSAD56; reportar una posible actividad minera ilegal; "
               "solicitar un vuelo de dron o un producto cartográfico; consultar el estado de un ticket (GEO-2026-0000); "
               "resolver dudas geodésicas frecuentes, o derivarle con una persona.")
AVISO = ("Soy KuriBot, un asistente automatizado de Geomática (prototipo académico con datos ficticios). "
         "Mis respuestas no sustituyen la revisión de un especialista.")
UBICACION = {"propia": "concesión propia", "tercero": "concesión de un tercero", "amortiguamiento": "franja de 500 m",
             "fuera": "fuera del área de la empresa"}
NEGATIVOS = re.compile(r"^\s*(no s[eé]|nada|ninguna|ninguno|no vi nada|no|desconozco|no tengo)\b", re.I)
CANCELAR = re.compile(r"\b(cancelar|cancela|olv[ií]dalo|dejalo|déjalo)\b", re.I)


_PII_FRASE = re.compile(r"\b(mi|su|el|la)?\s*(c[eé]dula|tel[eé]fono|celular|n[uú]mero|correo)\s*(es|:)?\s*\[(C[ÉE]DULA|
TEL[ÉE]FONO|CORREO)\]\s*,?", re.I)
_PII_MARCA = re.compile(r"\[(C[ÉE]DULA|TEL[ÉE]FONO|CORREO)\]")


def sin_pii(t: str) -> str:
    """Texto para el clasificador: los datos personales enmascarados no aportan información de intención (ajuste v2)."""
    t = _PII_MARCA.sub(" ", _PII_FRASE.sub(" ", t))
    return re.sub(r"\s+", " ", t).strip(" ,")


def pct(p: float) -> str:
    return f"{100 * p:.1f} %".replace(".", ",")


def num(x: float, dec: int = 1) -> str:
    s = f"{x:,.{dec}f}"
    return s.replace(",", " ").replace(".", ",")


@dataclass
class Respuesta:
    texto: str
    intencion: str | None = None
    confianza: float | None = None
    decision: str | None = None
    top: list = field(default_factory=list)
    accion: str | None = None
    reglas: list = field(default_factory=list)
    traza_reglas: list = field(default_factory=list)
```

```
    bayes: dict | None = None
    entidades: dict = field(default_factory=dict)
    ticket: str | None = None
    mapa: dict | None = None
    vuelo: dict | None = None
    latencia_ms: float = 0.0
    latencia_nlu_ms: float = 0.0
    avisos: list = field(default_factory=list)


@dataclass
class Estado:
    pendiente: dict | None = None
    aclaracion: dict | None = None
    fallos: int = 0
    intentos_ranura: int = 0
    ultima_concesion: str | None = None
    turnos: int = 0


class KuriBot:
    def __init__(self, nlu: ClasificadorIntenciones | None = None, kb: BaseConocimiento | None = None,
                 registro: str | Path | None = None):
        self.nlu = nlu or ClasificadorIntenciones()
        self.kb = kb or BaseConocimiento()
        self.red = RedAlertas()
        self.motor = motor_alertas()
        self.motor_ctx = motor_contexto()
        self.estado = Estado()
        self.registro = Path(registro) if registro else None

    def reiniciar(self):
        self.estado = Estado()

    # ============================================================ entrada principal
    def responder(self, texto: str) -> Respuesta:
        t0 = time.perf_counter()
        self.estado.turnos += 1
        seguro, pii = E.enmascarar_pii(texto)
        ent = self._entidades(seguro)
        r = self._despachar(seguro, ent)
        if pii:
            r.avisos.append("R02: detecté datos personales (" + ", ".join(sorted(set(pii))) + "); no los registro ni los
necesito.")
            r.reglas = ["R02"] + r.reglas
        r.entidades = {k: v for k, v in ent.items() if v not in (None, {}, [])}
        r.latencia_ms = (time.perf_counter() - t0) * 1000
        self._registrar(seguro, r)
        return r

    def _entidades(self, t: str) -> dict:
        c = E.extraer_coordenada(t)
        return {"coord": c, "datum": E.extraer_datum(t), "concesion": E.extraer_concesion(t, self.kb.nombres),
                "ticket": E.extraer_ticket(t), "fecha": E.extraer_fecha(t, self.kb.hoy), "producto": E.extraer_producto(t),
                "evidencias": E.extraer_evidencias(t), "urgente": bool(re.search(r"urgent", E.plano(t)))}

    def _despachar(self, t: str, ent: dict) -> Respuesta:
        st = self.estado
        if CANCELAR.search(t) and (st.pendiente or st.aclaracion):
            st.pendiente = st.aclaracion = None
            return Respuesta("De acuerdo, cancelé la solicitud en curso. ¿En qué más puedo ayudarle?", "cancelar",
decision="regla")
        pred = self.nlu.predecir(sin_pii(t))
        base = {"confianza": pred.confianza, "top": pred.top, "latencia_nlu_ms": pred.latencia_ms}
        # (2) red de seguridad: emergencias por palabras clave o por la red neuronal
        if E.hay_emergencia(t) or (pred.intencion == "emergencia_seguridad" and pred.confianza >= self.nlu.umbral_abstener):
            st.pendiente = st.aclaracion = None
            return self._con(self._emergencia(t, ent), pred, "emergencia_seguridad", base)
        # (3) respuesta a una aclaración
        if st.aclaracion:
            elegido = self._resolver_aclaracion(t)
            if elegido:
                original = st.aclaracion["texto"]
                st.aclaracion = None
                ent2 = self._entidades(original)
                for k, v in ent.items():
                    if v and not ent2.get(k):
                        ent2[k] = v
                return self._con(self._politica(elegido, original, ent2), pred, elegido, base, decision="aclarada")
            st.aclaracion = None
        # (4) completar acción pendiente
        if st.pendiente:
            p = st.pendiente
            self._fusionar(p["slots"], ent)
            if self._aporta(p["falta"], ent, t) or (pred.intencion == p["intencion"] and pred.decision == "aceptar"):
                st.pendiente = None
                if p["falta"] == "evidencias" and not ent["evidencias"]:
                    p["slots"]["evidencias_sin_dato"] = True
                return self._con(self._politica(p["intencion"], t, p["slots"]), pred, p["intencion"], base, decision="ranura")
            if pred.decision == "aceptar" and pred.intencion not in ("fuera_alcance",):
                st.pendiente = None  # cambio de tema: se abandona la acción pendiente
            else:
                st.intentos_ranura += 1
                if st.intentos_ranura >= 2:
                    st.pendiente = None
                    return self._con(Respuesta("No logré completar el dato requerido. Puedo derivarle con un analista "
                                               "(escriba «quiero hablar con una persona») o puede empezar de nuevo."),
                                     pred, p["intencion"], base, decision="ranura_fallida")
```

```
                return self._con(Respuesta(p["pregunta"] + " (o escriba «cancelar»)."), pred, p["intencion"], base,
decision="ranura")
        # (5) clasificación de la intención
        if pred.decision == "aclarar":
            a, b = pred.top[0][0], pred.top[1][0]
            st.aclaracion = {"opciones": [a, b], "texto": t}
            return self._con(Respuesta(f"No estoy seguro de haber entendido. ¿Quiere 1) {NOMBRES_INTENCION[a]} o "
                                       f"2) {NOMBRES_INTENCION[b]}? Responda 1 o 2."), pred, a, base, decision="aclarar")
        if pred.decision == "abstener":
            st.fallos += 1
            extra = (" Si lo prefiere, puedo derivarle con un analista: escriba «quiero hablar con una persona»."
                     if st.fallos >= 2 else "")
            return self._con(Respuesta("No tengo suficiente seguridad para responder eso. " + CAPACIDADES + extra),
                             pred, "fuera_alcance", base, decision="abstener")
        st.fallos = 0
        return self._con(self._politica(pred.intencion, t, dict(ent)), pred, pred.intencion, base, decision="aceptar")

    @staticmethod
    def _con(r: Respuesta, pred, intencion, base, decision=None) -> Respuesta:
        r.intencion = r.intencion or intencion
        r.confianza, r.top, r.latencia_nlu_ms = base["confianza"], base["top"], base["latencia_nlu_ms"]
        r.decision = r.decision or decision or pred.decision
        return r

    def _resolver_aclaracion(self, t: str) -> str | None:
        ops = self.estado.aclaracion["opciones"]
        tl = E.plano(t).strip()
        if re.fullmatch(r"(1|uno|la primera|primera|opcion 1|si|sí|a)\.?", tl):
            return ops[0]
        if re.fullmatch(r"(2|dos|la segunda|segunda|opcion 2|b)\.?", tl):
            return ops[1]
        return None

    @staticmethod
    def _fusionar(slots: dict, ent: dict):
        for k, v in ent.items():
            if k == "evidencias":
                slots.setdefault("evidencias", {}).update(v)
            elif v not in (None, {}, False):
                slots[k] = v

    @staticmethod
    def _aporta(falta: str, ent: dict, t: str) -> bool:
        if falta == "ubicacion":
            return bool(ent["coord"] or ent["concesion"])
        if falta == "evidencias":
            return bool(ent["evidencias"]) or bool(NEGATIVOS.search(t))
        return ent.get(falta) is not None and ent.get(falta) != {}

    def _pedir(self, intencion: str, falta: str, slots: dict, pregunta: str) -> Respuesta:
        self.estado.pendiente = {"intencion": intencion, "falta": falta, "slots": slots, "pregunta": pregunta}
        self.estado.intentos_ranura = 0
        return Respuesta(pregunta, intencion, accion=f"solicitar_dato:{falta}")

    # ============================================================== políticas por intención
    def _politica(self, intencion: str, t: str, s: dict) -> Respuesta:
        h = getattr(self, f"_i_{intencion}", None)
        return h(t, s) if h else Respuesta(CAPACIDADES, intencion)

    def _i_saludo(self, t, s):
        return Respuesta(f"¡Hola! {AVISO} {CAPACIDADES}", "saludo", accion="responder")

    def _i_despedida(self, t, s):
        self.estado.pendiente = None
        return Respuesta("Gracias por escribir. Si necesita algo más de Geomática, aquí estaré.", "despedida", accion="cerrar")

    def _i_ayuda_capacidades(self, t, s):
        return Respuesta(CAPACIDADES + " Para emergencias con riesgo para personas escriba «emergencia».", accion="responder")

    def _i_fuera_alcance(self, t, s):
        return Respuesta("Ese tema está fuera de mi alcance: solo atiendo consultas de Geomática y Catastro minero. "
                         + CAPACIDADES, accion="fuera_alcance")

    def _i_consultar_concesion(self, t, s):
        cod = s.get("concesion") or (self.estado.ultima_concesion if re.search(r"\b(esa|esta|la misma)\b", E.plano(t)) else None)
        if not cod:
            return self._pedir("consultar_concesion", "concesion", s, "¿De qué concesión? Indique el código (p. ej., CM-103) o el
nombre")
        f = self.kb.ficha(cod)
        if not f:
            return Respuesta(f"No encuentro la concesión {cod} en el catastro de demostración; no puedo inventar sus datos.",
                             accion="no_encontrado")
        self.estado.ultima_concesion = cod
        if f["titular"] != "propia":
            return Respuesta(f"{cod} {f['nombre']} pertenece a un tercero con derecho vigente. Por confidencialidad solo "
                             f"confirmo su existencia y su superficie ({num(f['area_ha'])} ha); para más detalle, consulte a
Legal y Catastro.",
                             accion="responder", mapa=self._mapa(cod))
        act = f"; actividad autorizada programada: {f['actividad_programada']}" if f["actividad_programada"] else ""
        return Respuesta(f"Concesión {cod} {f['nombre']}: fase {f['fase']}, estado {f['estado']}, superficie {num(f['area_ha'])}
ha, "
                         f"campamento {f['campamento']}. Alertas en los últimos 90 días: {f['alertas_90d']}{act}. "
                         "Fuente: catastro interno de demostración (sintético).", accion="responder", mapa=self._mapa(cod))

    def _coordenada_lista(self, intencion: str, s: dict, pregunta_coord: str):
        c = s.get("coord")
        if c is None:
            return None, self._pedir(intencion, "coord", s, pregunta_coord)
```

```
        datum = s.get("datum") or c.punto.datum
        if datum is None:
            return None, self._pedir(intencion, "datum", s, "¿En qué datum están esas coordenadas: WGS 84 o PSAD56? "
                                                             "(en esta zona difieren unos 440 m)")
        p = G.Punto(c.punto.x, c.punto.y, c.punto.tipo, datum)
        e, n = G.a_wgs84_utm(p)
        if not G.dentro_de_ecuador_utm17s(e, n):
            return None, Respuesta("Las coordenadas quedan fuera del rango de Ecuador continental en UTM 17S (G02). "
                                   "Revise si el Este y el Norte están invertidos o si falta un dígito.", intencion,
accion="rechazar")
        return (p, e, n, c.avisos), None

    def _i_verificar_punto(self, t, s):
        listo, r = self._coordenada_lista("verificar_punto", s, "Indique las coordenadas del punto (p. ej., E 642350 N 9662120) y
su datum")
        if r:
            return r
        p, e, n, avisos = listo
        avisos = list(avisos)
        a = self.kb.analizar_punto(e, n)
        f = self.motor_ctx.inferir({"ubicacion": a.ubicacion, "area_protegida": bool(a.area_protegida),
                                    "comunidad_cercana": bool(a.comunidad)})
        partes = []
        if p.datum == "PSAD56":
            partes.append(f"Transformé el punto de PSAD56 a WGS 84 con {G.descripcion_transformacion()}: E {num(e)} N {num(n)}.")
            if p.tipo == "utm":
                crudo = self.kb.analizar_punto(p.x, p.y)  # lectura errónea: mismas cifras tomadas como WGS 84
                if (crudo.concesion or crudo.ubicacion) != (a.concesion or a.ubicacion):
                    avisos.append(f"Riesgo de datum: si estas cifras se leyeran como WGS 84, el punto caería en "
                                  f"{crudo.concesion or 'una zona sin concesión'} y no en {a.concesion or 'una zona sin
concesión'}.")
        if a.ubicacion == "propia":
            c = self.kb.concesiones[a.concesion]
            partes.append(f"El punto está DENTRO de {a.concesion} {c['nombre']} ({c['estado']}), a {num(a.distancia_lindero_m,
0)} m del lindero.")
        elif a.ubicacion == "tercero":
            partes.append(f"El punto está dentro de {a.concesion}, concesión de un tercero con derecho vigente (R12). "
                          "No debe intervenirse sin revisión de Legal.")
        elif a.ubicacion == "amortiguamiento":
            partes.append(f"El punto está FUERA de las concesiones, a {num(a.distancia_lindero_m, 0)} m de {a.cercana} "
                          "(dentro de la franja de 500 m, R11).")
        else:
            partes.append(f"El punto está fuera de las concesiones de la empresa; la más cercana es {a.cercana} a
{num(a.distancia_lindero_m, 0)} m.")
        if a.area_protegida:
            partes.append(f"Atención: cae en {a.area_protegida} (R13).")
        if a.comunidad:
            partes.append(f"Hay una comunidad a {num(a.distancia_comunidad_m, 0)} m: {a.comunidad} (R14).")
        partes.append("Verificación sobre el catastro sintético del prototipo; para fines legales use el catastro oficial
vigente.")
        self.estado.ultima_concesion = a.concesion or a.cercana
        return Respuesta(" ".join(partes), "verificar_punto", accion="geoproceso", reglas=f.disparadas, traza_reglas=f.traza,
                         mapa=self._mapa(a.concesion or a.cercana, (e, n)), avisos=avisos)

    def _i_convertir_coordenadas(self, t, s):
        listo, r = self._coordenada_lista("convertir_coordenadas", s, "Indique las coordenadas a convertir y su datum de origen")
        if r:
            return r
        p, e, n, avisos = listo
        rep = G.todas_las_representaciones(p)
        (we, wn), (lat, lon), (pe, pn) = rep["WGS84_UTM17S"], rep["WGS84_GEO"], rep["PSAD56_UTM17S"]
        texto = (f"Origen: {p.datum}. Resultados: WGS 84 / UTM 17S (EPSG:32717): E {num(we, 2)} m, N {num(wn, 2)} m · "
                 f"WGS 84 geográficas (EPSG:4326): {lat:.6f}, {lon:.6f} ({G.dms(lat, 'lat')}, {G.dms(lon, 'lon')}) · "
                 f"PSAD56 / UTM 17S (EPSG:24877): E {num(pe, 2)} m, N {num(pn, 2)} m. "
                 f"Transformación de datum: {G.descripcion_transformacion()}. SIRGAS 2000 se considera equivalente a WGS 84 "
                 "al nivel métrico.")
        return Respuesta(texto, "convertir_coordenadas", accion="geoproceso", mapa=self._mapa(None, (we, wn)),
avisos=list(avisos))

    def _ubicar_alerta(self, s) -> dict:
        """Contexto geoespacial y nodos raíz (H, A) de la red bayesiana para la alerta."""
        ctx = {"codigo": None, "ubicacion": None, "area_protegida": False, "comunidad_cercana": False, "punto": None}
        ev = {}
        if s.get("coord") is not None:
            datum = s.get("datum") or s["coord"].punto.datum or "WGS84"
            pt = G.Punto(s["coord"].punto.x, s["coord"].punto.y, s["coord"].punto.tipo, datum)
            e, n = G.a_wgs84_utm(pt)
            a = self.kb.analizar_punto(e, n)
            ctx.update(codigo=a.concesion or a.cercana, ubicacion=a.ubicacion, area_protegida=bool(a.area_protegida),
                       comunidad_cercana=bool(a.comunidad), punto=(e, n), datum_supuesto=s.get("datum") is None and
s["coord"].punto.datum is None)
            if a.ubicacion in ("propia", "amortiguamiento"):
                ev["H"] = int(self.kb.op["historial_alertas_90d"].get(ctx["codigo"], 0) > 0)
                ev["A"] = int(a.ubicacion == "propia" and ctx["codigo"] in self.kb.op["actividad_autorizada_programada"])
        else:
            cod = s["concesion"]
            c = self.kb.concesiones.get(cod)
            ctx["codigo"] = cod
            if c:
                ctx["ubicacion"] = "propia" if c["titular"] == "propia" else "tercero"
                g = c["geom"]
                ctx["area_protegida"] = any(g.distance(x["geom"]) <= 1.0 for x in self.kb.areas.values())
                ctx["comunidad_cercana"] = any(g.distance(x["geom"]) <= 1000 for x in self.kb.comunidades.values())
                ev["H"] = int(self.kb.op["historial_alertas_90d"].get(cod, 0) > 0)
                if c["titular"] == "propia":
                    ev["A"] = int(cod in self.kb.op["actividad_autorizada_programada"])
        return {"ctx": ctx, "ev_raiz": ev}
```

```
    def _i_reportar_alerta(self, t, s):
        if not (s.get("coord") or s.get("concesion")):
            return self._pedir("reportar_alerta", "ubicacion", s, "¿Dónde ocurre? Indique la concesión (código o nombre) o las
coordenadas con su datum")
        if s.get("concesion") and not s.get("coord") and s["concesion"] not in self.kb.concesiones:
            return Respuesta(f"No encuentro {s['concesion']} en el catastro; indique coordenadas para ubicar la alerta.",
accion="no_encontrado")
        if not s.get("evidencias") and not s.get("evidencias_sin_dato"):
            return self._pedir("reportar_alerta", "evidencias", s, "¿Qué observó? Por ejemplo: maquinaria, personas o campamento,
"
                                                                    "desbroce o huecos nuevos, agua turbia, ruido o voladuras de
noche, trocha nueva. "
                                                                    "Si no lo sabe, escriba «no sé»")
        u = self._ubicar_alerta(s)
        ctx, ev = u["ctx"], dict(u["ev_raiz"])
        obs = {k: int(v) for k, v in (s.get("evidencias") or {}).items()}
        ev.update(obs)
        p = self.red.posterior(ev)
        contrib = self.red.contribuciones(ev)
        camp = self.kb.campamento(ctx["codigo"])
        ventana = primera_ventana(0, camp, self.kb.op, urgente=True)
        hechos = {"ubicacion": ctx["ubicacion"], "area_protegida": ctx["area_protegida"],
                  "comunidad_cercana": ctx["comunidad_cercana"], "p_intrusion": p, "vuelo_dias": ventana["dia"]}
        inf = self.motor.inferir(hechos)
        f = inf.hechos
        prioridad, accion = f.get("prioridad"), f.get("accion")
        resp = "Seguridad patrimonial" if prioridad not in ("REVISIÓN LEGAL", "INFORMATIVA") else "Legal y Catastro"
        compromiso = self.kb.fecha(ventana["dia"]) if ventana["dia"] is not None and prioridad in ("CRÍTICA", "ALTA") else None
        tk = self.kb.nuevo_ticket("alerta", f"Alerta en {ctx['codigo']} · P={p:.3f}", resp, compromiso, prioridad)
        nombre = self.kb.concesiones.get(ctx["codigo"], {}).get("nombre", "")
        evid = ", ".join(f"{E.EVIDENCIAS[k][0]} ({'sí' if v else 'no'})" for k, v in obs.items()) or "sin evidencias específicas"
        partes = [f"Alerta registrada: {tk} · prioridad {prioridad}.",
                  f"Ubicación: {ctx['codigo']} {nombre} ({UBICACION.get(ctx['ubicacion'], 'sin ubicar')}).", f"Evidencias:
{evid}."]
        if "H" in ev:
            partes.append(f"Antecedentes en 90 días: {'sí' if ev['H'] else 'no'}; actividad autorizada programada: "
                          f"{'sí' if ev.get('A') else ('no' if 'A' in ev else 'desconocida')}.")
        partes.append(f"Probabilidad estimada de actividad no autorizada: {pct(p)} (red bayesiana con supuestos expertos; no es
una certeza).")
        if ev.get("A") == 1 and obs:
            partes.append("La actividad autorizada programada explica parte de lo observado, por eso la probabilidad no es
mayor.")
        partes.append(f"Acción propuesta: {accion}.")
        if ventana["dia"] is not None and prioridad in ("CRÍTICA", "ALTA"):
            partes.append(f"Primera ventana de vuelo apta: {self.kb.fecha(ventana['dia'])} ({ventana['dron']},
{ventana['piloto']}).")
        if f.get("aprobacion_supervisor"):
            partes.append("Antes de ejecutar, el supervisor de turno debe aprobar la acción (R41).")
        if f.get("revision_humana"):
            partes.append("Zona sensible: se requiere revisión humana y protocolo comunitario antes de cualquier acción de campo
(R40).")
        if ctx.get("datum_supuesto"):
            partes.append("Asumí WGS 84 para las coordenadas porque no indicó el datum; confírmelo si no es así.")
        bayes = {"p_intrusion": p, "evidencia": ev, "contribuciones": contrib}
        return Respuesta(" ".join(partes), "reportar_alerta", accion=accion, reglas=inf.disparadas, traza_reglas=inf.traza,
                         bayes=bayes, ticket=tk, vuelo=ventana, mapa=self._mapa(ctx["codigo"], ctx["punto"]))

    def _i_solicitar_vuelo(self, t, s):
        cod = s.get("concesion")
        if not cod:
            return self._pedir("solicitar_vuelo", "concesion", s, "¿Sobre qué concesión necesita el vuelo? (código o nombre)")
        if cod not in self.kb.concesiones:
            return Respuesta(f"No encuentro {cod} en el catastro de demostración.", accion="no_encontrado")
        if s.get("fecha") is None:
            return self._pedir("solicitar_vuelo", "fecha", s, "¿Para qué fecha? (hoy, mañana, un día de la semana o una fecha)")
        camp = self.kb.campamento(cod)
        dia = max(0, s["fecha"])
        if dia > self.kb.op["politica_vuelo"]["horizonte_dias"]:
            return Respuesta(f"Solo tengo pronóstico hasta {self.kb.fecha(self.kb.op['politica_vuelo']['horizonte_dias'])}. "
                             "Registre la solicitud más cerca de la fecha o hable con el coordinador de vuelos.",
accion="fuera_horizonte")
        v = primera_ventana(dia, camp, self.kb.op, urgente=s.get("urgente", False))
        rechazos = [f"{self.kb.fecha(d['dia'])}: " + "; ".join(m.split(' ', 1)[1] for m in d["motivos"]) for d in v["evaluados"]
if not d["apto"]]
        if v["dia"] is None:
            return Respuesta(f"No hay ventana apta en el horizonte de pronóstico para el campamento {camp}. Motivos: "
                             + " | ".join(rechazos) + ". Derivo al coordinador de vuelos.", accion="sin_ventana", vuelo=v)
        tk = self.kb.nuevo_ticket("vuelo", f"Vuelo en {cod}", v["piloto"], self.kb.fecha(v["dia"]))
        cab = "Fecha solicitada apta." if v["dia"] == dia else f"La fecha solicitada no es apta ({' | '.join(rechazos)})."
        return Respuesta(f"{cab} Propuesta: vuelo en {cod} el {self.kb.fecha(v['dia'])} con {v['dron']} y {v['piloto']} "
                         f"(viento {num(v['viento'])} m/s, sin lluvia, visibilidad {v['visibilidad']} km). Solicitud {tk} "
                         "registrada como pendiente de aprobación del coordinador de vuelos.", "solicitar_vuelo",
                         accion="programar_vuelo", ticket=tk, vuelo=v, mapa=self._mapa(cod))

    def _i_solicitar_cartografia(self, t, s):
        prod = s.get("producto")
        if not prod:
            return self._pedir("solicitar_cartografia", "producto", s, "¿Qué producto necesita? (mapa de ubicación, plano
topográfico, "
                                                                       "ortofoto, KMZ, shapefile o modelo digital de
elevación)")
        cod = s.get("concesion") or self.estado.ultima_concesion
        if not cod:
            return self._pedir("solicitar_cartografia", "concesion", s, "¿De qué concesión o área? (código o nombre)")
        sla = self.kb.op["sla_cartografia_dias_habiles"]
        dias = sla["urgente"] if s.get("urgente") else sla["estandar"]
        tk = self.kb.nuevo_ticket("cartografía", f"{prod} de {cod}", "Analista SIG", None)
        return Respuesta(f"Solicitud {tk} registrada: {prod} de {cod}. Plazo interno de referencia: {dias} día(s) hábil(es)"
                         f"{' (urgente: requiere aprobación de la jefatura)' if s.get('urgente') else ''}. "
```

```
                        "El producto se entregará en EPSG:32717 salvo que indique otro sistema.", "solicitar_cartografia",
                        accion="crear_ticket", ticket=tk)

    def _i_estado_solicitud(self, t, s):
        tk = s.get("ticket")
        if not tk:
            return self._pedir("estado_solicitud", "ticket", s, "Indique el número de solicitud (formato GEO-2026-0000)")
        d = self.kb.ticket(tk)
        if not d:
            return Respuesta(f"No encuentro la solicitud {tk}. Revise el número; no puedo suponer su estado.",
accion="no_encontrado")
        extra = f" Resolución: {d['resolucion']}." if d.get("resolucion") else ""
        comp = f", fecha comprometida {d['compromiso']}" if d.get("compromiso") else ""
        return Respuesta(f"{tk} ({d['tipo']}: {d['detalle']}): estado «{d['estado']}», responsable {d['asignado']}{comp}.
{extra}",
                        "estado_solicitud", accion="consultar_ticket", ticket=tk)

    def _i_consulta_geodesia(self, t, s):
        f, sim = self.kb.buscar_faq(t)
        if sim < 0.30:
            return Respuesta("No tengo una respuesta verificada para esa duda. Para no darle información incorrecta, la derivo "
                             "al analista de geodesia (o reformule la pregunta).", accion="derivar_faq",
                             traza_reglas=[f"FAQ: similitud máxima {sim:.2f} < 0,30"])
        return Respuesta(f"{f['respuesta']} (Fuente: {f['fuente']}.)", "consulta_geodesia", accion="responder_faq",
                        traza_reglas=[f"FAQ {f['id']} · similitud {sim:.2f}"])

    def _i_hablar_humano(self, t, s):
        inf = self.motor.inferir({"intencion": "hablar_humano"})
        tk = self.kb.nuevo_ticket("derivación", "Solicitud de atención humana", "Mesa de Geomática", None)
        return Respuesta(f"Entendido. Registré la derivación {tk}; un analista de la Mesa de Geomática le contactará por el canal
"
                        "interno en horario laboral (simulado en este prototipo).", "hablar_humano", accion="derivar_humano",
                        reglas=inf.disparadas, traza_reglas=inf.traza, ticket=tk)

    def _emergencia(self, t, ent):
        u = self._ubicar_alerta(ent) if (ent.get("coord") or ent.get("concesion")) else {"ctx": {"codigo": None}}
        inf = self.motor.inferir({"emergencia": True})
        cod = u["ctx"].get("codigo")
        tk = self.kb.nuevo_ticket("emergencia", f"Emergencia reportada{' en ' + cod if cod else ''}", "Jefe de Seguridad",
self.kb.fecha(0), "CRÍTICA")
        return Respuesta("EMERGENCIA: si hay personas en riesgo, llame de inmediato al ECU 911 y al Jefe de Seguridad del
campamento. "
                        "Aléjese del lugar si es seguro hacerlo. No se enviará personal ni dron a la zona hasta que Seguridad lo
autorice (R01). "
                        f"Registré el caso {tk} con prioridad CRÍTICA y lo derivé a una persona.", "emergencia_seguridad",
                        accion="PROTOCOLO DE EMERGENCIA", reglas=inf.disparadas, traza_reglas=inf.traza, ticket=tk)

    # ============================================================ utilidades
    def _mapa(self, codigo: str | None, punto: tuple[float, float] | None = None) -> dict:
        return {"concesion": codigo, "punto": punto}

    def _registrar(self, texto: str, r: Respuesta):
        if not self.registro:
            return
        fila = {"turno": self.estado.turnos, "texto_enmascarado": texto, "intencion": r.intencion, "confianza": r.confianza,
                "decision": r.decision, "accion": r.accion, "reglas": r.reglas, "ticket": r.ticket,
                "p_intrusion": (r.bayes or {}).get("p_intrusion"), "latencia_ms": round(r.latencia_ms, 2)}
        self.registro.parent.mkdir(parents=True, exist_ok=True)
        with open(self.registro, "a", encoding="utf-8") as fh:
            fh.write(json.dumps(fila, ensure_ascii=False) + "\n")


def respuesta_a_dict(r: Respuesta) -> dict:
    d = asdict(r)
    if r.entidades.get("coord") is not None:
        c = r.entidades["coord"]
        d["entidades"]["coord"] = {"x": c.punto.x, "y": c.punto.y, "tipo": c.punto.tipo, "datum": c.punto.datum}
    return d
```

D.8 Mapa estático del catastro (kuribot/mapa.py)

```
"""Mapa estático (matplotlib) del catastro sintético: funciona sin conexión a servicios de teselas."""
from __future__ import annotations

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt  # noqa: E402
from matplotlib.patches import Polygon as MplPolygon  # noqa: E402

COLORES = {"propia": "#2E7D5B", "tercero": "#9A6B2F", "resaltado": "#C0392B", "protegida": "#5B8DB8"}


def dibujar(kb, codigo: str | None = None, punto: tuple[float, float] | None = None, ancho: float = 5.2, alto: float = 4.2):
    fig, ax = plt.subplots(figsize=(ancho, alto), dpi=110)
    for a in kb.areas.values():
        ax.add_patch(MplPolygon(list(a["geom"].exterior.coords), closed=True, fc=COLORES["protegida"], alpha=0.18,
                                ec=COLORES["protegida"], lw=1, hatch="..", label="Área protegida (ficticia)"))
    for cod, c in kb.concesiones.items():
        color = COLORES["resaltado"] if cod == codigo else COLORES[c["titular"]]
        ax.add_patch(MplPolygon(list(c["geom"].exterior.coords), closed=True, fc=color, alpha=0.30 if cod == codigo else 0.12,
                                ec=color, lw=2.2 if cod == codigo else 1))
        x, y = c["geom"].centroid.coords[0]
        ax.text(x, y, cod, ha="center", va="center", fontsize=7, color="#222")
    for com in kb.comunidades.values():
        x, y = com["geom"].coords[0]
        ax.plot(x, y, marker="^", color="#6C3483", ms=7)
```

```
        ax.text(x + 120, y - 160, "Comunidad", fontsize=6, color="#6C3483")
    if punto is not None:
        ax.plot(*punto, marker="*", color="#111", ms=14, mec="white", mew=1)
    ax.set_xlim(639_400, 649_600)
    ax.set_ylim(9_658_000, 9_666_600)
    ax.set_aspect("equal")
    ax.ticklabel_format(useOffset=False, style="plain")
    ax.tick_params(labelsize=6)
    ax.set_xlabel("Este (m) · WGS 84 / UTM 17S", fontsize=7)
    ax.set_ylabel("Norte (m)", fontsize=7)
    ax.grid(alpha=0.25, lw=0.5)
    ax.set_title("Catastro sintético del prototipo", fontsize=8)
    fig.tight_layout()
    return fig
```

D.9 Interfaz web (app_streamlit.py)

```
"""Interfaz web de KuriBot (Streamlit).  Ejecutar:  streamlit run app_streamlit.py"""
from __future__ import annotations

import sys
from pathlib import Path

import pandas as pd
import streamlit as st

sys.path.insert(0, str(Path(__file__).resolve().parent))
from kuribot.conocimiento import BaseConocimiento  # noqa: E402
from kuribot.dialogo import AVISO, KuriBot  # noqa: E402
from kuribot.entidades import EVIDENCIAS  # noqa: E402
from kuribot.mapa import dibujar  # noqa: E402
from kuribot.nlu import ClasificadorIntenciones  # noqa: E402
from kuribot.razonamiento import DESCRIPCION_NODOS  # noqa: E402

st.set_page_config(page_title="KuriBot · Geomática", page_icon="🛰️", layout="wide")


@st.cache_resource(show_spinner="Cargando el modelo de lenguaje…")
def cargar_nlu():
    return ClasificadorIntenciones()


def bot() -> KuriBot:
    if "bot" not in st.session_state:
        st.session_state.bot = KuriBot(cargar_nlu(), BaseConocimiento(), registro=Path("registros/conversaciones.jsonl"))
        st.session_state.historial = []
    return st.session_state.bot


EJEMPLOS = ["Hola", "¿Cuál es el estado de la concesión La Ventana?", "¿El punto E 642600 N 9662480 PSAD56 está dentro de alguna
concesión?",
            "Hay una retroexcavadora y gente extraña trabajando en Cerro Azul", "Necesito un vuelo de dron en San Gregorio
mañana",
            "¿Cómo va la solicitud GEO-2026-0412?", "¿Qué EPSG uso para PSAD56 UTM 17 sur?"]

with st.sidebar:
    st.markdown("### 🛰️ KuriBot")
    st.caption(AVISO)
    st.markdown(f"**Modelo NLU:** {cargar_nlu().nombre}")
    ver_traza = st.toggle("Mostrar trazabilidad", value=True)
    st.markdown("**Ejemplos**")
    for i, e in enumerate(EJEMPLOS):
        if st.button(e, key=f"ej{i}", use_container_width=True):
            st.session_state.pendiente_ejemplo = e
    if st.button("🔄 Nueva conversación", use_container_width=True):
        for k in ("bot", "historial"):
            st.session_state.pop(k, None)
        st.rerun()
    st.caption("No escriba cédulas, teléfonos ni datos personales. Emergencias reales: ECU 911.")

st.markdown("## KuriBot · Asistente de Geomática y Catastro minero")
st.caption("Prototipo académico · datos ficticios · NLU con BETO ajustado + reglas + red bayesiana")
asistente = bot()


def mostrar_traza(r):
    c1, c2 = st.columns([1.15, 1])
    with c1:
        st.markdown(f"**Intención:** `{r.intencion}` · **confianza** {r.confianza:.2f} · **decisión** {r.decision}")
        st.dataframe(pd.DataFrame(r.top, columns=["intención", "probabilidad"]).round(3), hide_index=True,
use_container_width=True)
        if r.entidades:
            def fmt(k, v):
                if hasattr(v, "texto"):
                    return f"coordenada={v.texto.strip()}"
                if k == "evidencias":
                    return "evidencias=" + "; ".join(f"{EVIDENCIAS[e][0]} ({'sí' if b else 'no'})" for e, b in v.items())
                return f"{k}={v}"
            st.markdown("**Entidades:** " + ", ".join(fmt(k, v) for k, v in r.entidades.items() if v not in (False,)))
        if r.traza_reglas:
            st.markdown("**Reglas disparadas**")
            for t in r.traza_reglas:
                st.markdown(f"- {t}")
        if r.bayes:
            b = r.bayes
            st.markdown(f"**Red bayesiana:** P(intrusión | evidencias) = **{b['p_intrusion']:.3f}**")
```

```
            filas = [{"nodo": k, "descripción": DESCRIPCION_NODOS[k], "valor": "sí" if v else "no"} for k, v in
b["evidencia"].items()]
            st.dataframe(pd.DataFrame(filas), hide_index=True, use_container_width=True)
            if b["contribuciones"]:
                st.caption("Cambio del posterior al retirar cada evidencia: " + "; ".join(
                    f"{EVIDENCIAS[k][0]} {d:+.3f}" for k, d in b["contribuciones"]))
        if r.vuelo and r.vuelo.get("evaluados"):
            st.markdown("**Evaluación de ventanas de vuelo**")
            st.dataframe(pd.DataFrame([{"día": e["dia"], "apto": e["apto"], "viento m/s": e["viento"], "lluvia mm/h":
e["lluvia"],
                                        "visib. km": e["visibilidad"], "motivos": "; ".join(e["motivos"])} for e in
r.vuelo["evaluados"]]),
                         hide_index=True, use_container_width=True)
        st.caption(f"Latencia total {r.latencia_ms:.0f} ms (NLU {r.latencia_nlu_ms:.0f} ms)")
    with c2:
        if r.mapa and (r.mapa.get("concesion") or r.mapa.get("punto")):
            st.pyplot(dibujar(asistente.kb, r.mapa.get("concesion"), r.mapa.get("punto")), use_container_width=True)


for msg in st.session_state.historial:
    with st.chat_message(msg["rol"], avatar="🧑‍💼" if msg["rol"] == "user" else "🛰️"):
        st.markdown(msg["texto"])
        if msg.get("r") is not None and ver_traza:
            with st.expander("Trazabilidad de la respuesta", expanded=msg.get("ultimo", False)):
                mostrar_traza(msg["r"])

entrada = st.chat_input("Escriba su consulta…") or st.session_state.pop("pendiente_ejemplo", None)
if entrada:
    for m in st.session_state.historial:
        m["ultimo"] = False
    r = asistente.responder(entrada)
    st.session_state.historial.append({"rol": "user", "texto": entrada})
    texto = r.texto + ("\n\n" + "\n".join(f"⚠️ {a}" for a in r.avisos) if r.avisos else "")
    st.session_state.historial.append({"rol": "assistant", "texto": texto, "r": r, "ultimo": True})
    st.rerun()
```

D.10 Interfaz de consola (cli.py)

```
"""Interfaz de consola de KuriBot.  Ejecutar:  python cli.py   (escriba «salir» para terminar)"""
from __future__ import annotations

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent))
from kuribot.dialogo import AVISO, KuriBot  # noqa: E402


def main():
    bot = KuriBot(registro=Path("registros/consola.jsonl"))
    print(f"KuriBot · {bot.nlu.nombre}\n{AVISO}\nEscriba «salir» para terminar.\n")
    while True:
        try:
            texto = input("Usted › ").strip()
        except (EOFError, KeyboardInterrupt):
            break
        if not sys.stdin.isatty():
            print(texto)  # eco de la entrada cuando la consola se alimenta desde un archivo
        if texto.lower() in {"salir", "exit", "quit"}:
            break
        if not texto:
            continue
        r = bot.responder(texto)
        print(f"KuriBot › {r.texto}")
        for a in r.avisos:
            print(f"   ⚠️ {a}")
        extra = f" · P(intrusión)={r.bayes['p_intrusion']:.3f}" if r.bayes else ""
        print(f"   [intención={r.intencion} conf={r.confianza:.2f} decisión={r.decision} reglas={','.join(r.reglas) or '—'}"
              f"{extra} · {r.latencia_ms:.0f} ms]\n")


if __name__ == "__main__":
    main()
```

D.11 Base de conocimiento sintética (datos/construir_base.py)

```
"""Genera la base de conocimiento SINTÉTICA de KuriBot (concesiones, operación, FAQ, tickets).

Todo lo que se escribe aquí es ficticio: nombres, códigos, polígonos, clima, flota y tickets
se construyeron para el prototipo académico. Las respuestas de la FAQ geodésica se limitan a
hechos verificables en la base EPSG (vía pyproj) o a definiciones generales; las políticas
operativas (umbral de viento, plazos, formatos) son supuestos de diseño del equipo.

Uso: python datos/construir_base.py
"""
from __future__ import annotations

import json
import sys
from pathlib import Path

RAIZ = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(RAIZ))
from kuribot.geodesia import descripcion_transformacion, psad56_utm_a_wgs84_utm  # noqa: E402

DATOS = RAIZ / "datos"
```

```
def rect(e0, n0, e1, n1):
    return [[e0, n0], [e1, n0], [e1, n1], [e0, n1], [e0, n0]]


CONCESIONES = [
    # codigo, nombre, fase, estado, campamento, polígono UTM17S WGS84, titular
    ("CM-101", "Peña Blanca", "explotación", "vigente", "Norte", rect(641000, 9661500, 643000, 9663000), "propia"),
    ("CM-102", "San Gregorio", "explotación", "vigente", "Norte", rect(643000, 9662000, 644600, 9663500), "propia"),
    ("CM-103", "La Ventana", "exploración avanzada", "vigente", "Central", rect(644600, 9660500, 647000, 9662500), "propia"),
    ("CM-104", "El Roble", "exploración inicial", "vigente (renovación en trámite)", "Central",
     [[640000, 9664000], [641000, 9664000], [641000, 9665400], [640600, 9666000], [640000, 9666000], [640000, 9664000]],
"propia"),
    ("CM-105", "Cerro Azul", "explotación", "suspendida temporalmente", "Sur", rect(645500, 9663000, 646500, 9664200), "propia"),
    ("CM-106", "Los Laureles", "exploración inicial", "vigente", "Sur", rect(646500, 9658500, 648500, 9660500), "propia"),
    ("CT-201", "Minas del Valle", "explotación", "vigente", "—", rect(641000, 9663000, 643000, 9664200), "tercero"),
]
AREA_PROTEGIDA = ("BP-01", "Bosque Protector El Mirador (ficticio)", rect(647000, 9660500, 649000, 9662500))
COMUNIDADES = [("COM-01", "Comunidad San José (ficticia)", [644200, 9660800])]

OPERACION = {
    "fecha_referencia": "2026-11-09",
    "nota": "Datos operativos sintéticos para la demostración; no proceden de ninguna empresa.",
    "historial_alertas_90d": {"CM-101": 0, "CM-102": 1, "CM-103": 0, "CM-104": 2, "CM-105": 3, "CM-106": 1, "CT-201": 0},
    "actividad_autorizada_programada": {
        "CM-101": "frente de explotación activo con maquinaria propia",
        "CM-102": "mantenimiento de vías con retroexcavadora propia",
        "CM-103": "campaña de sondajes con perforadora contratista",
    },
    "pronostico": {  # día relativo a fecha_referencia: [viento m/s, lluvia mm/h, visibilidad km]
        "Norte":   {"0": [4.2, 0.0, 10], "1": [12.5, 0.0, 8], "2": [6.0, 0.0, 10], "3": [5.1, 2.4, 4], "4": [3.8, 0.0, 10]},
        "Central": {"0": [7.5, 1.2, 5], "1": [5.0, 0.0, 10], "2": [4.4, 0.0, 10], "3": [11.0, 0.0, 9], "4": [6.2, 0.0, 10]},
        "Sur":     {"0": [3.1, 0.0, 10], "1": [8.9, 0.0, 10], "2": [10.8, 0.5, 6], "3": [4.0, 0.0, 10], "4": [4.5, 0.0, 10]},
    },
    "drones": {"DRN-01": {"tipo": "multirrotor RTK", "no_disponible": ["2"]},
               "DRN-02": {"tipo": "ala fija VTOL", "no_disponible": ["0", "1"]}},
    "pilotos": {"Piloto A": {"no_disponible": ["3"]}, "Piloto B": {"no_disponible": ["0"]}},
    "politica_vuelo": {"viento_max_ms": 10.0, "lluvia_max_mmh": 0.0, "visibilidad_min_km": 5.0,
                       "anticipacion_min_dias": 1, "horizonte_dias": 4},
    "sla_cartografia_dias_habiles": {"estandar": 3, "urgente": 1},
}

TICKETS = [
    {"id": "GEO-2026-0412", "tipo": "cartografía", "detalle": "Mapa topográfico de CM-103", "estado": "En elaboración",
     "asignado": "Analista SIG 1", "compromiso": "2026-11-10"},
    {"id": "GEO-2026-0415", "tipo": "vuelo", "detalle": "Vuelo de inspección en CM-101", "estado": "Programado",
     "asignado": "Piloto A", "compromiso": "2026-11-10"},
    {"id": "GEO-2026-0419", "tipo": "alerta", "detalle": "Desbroce detectado en CM-105", "estado": "Cerrada",
     "asignado": "Seguridad patrimonial", "compromiso": "2026-11-03",
     "resolucion": "Verificada en campo: trabajos de remediación autorizados"},
    {"id": "GEO-2026-0423", "tipo": "cartografía", "detalle": "KMZ de linderos de CM-102", "estado": "Entregado",
     "asignado": "Analista SIG 2", "compromiso": "2026-11-05"},
    {"id": "GEO-2026-0430", "tipo": "alerta", "detalle": "Turbidez aguas abajo de CM-106", "estado": "En verificación de campo",
     "asignado": "Seguridad patrimonial", "compromiso": "2026-11-11"},
]


def faq() -> list[dict]:
    e, n = 642600.0, 9662480.0
    e2, n2 = psad56_utm_a_wgs84_utm(e, n)
    de, dn = e2 - e, n2 - n
    d = (de ** 2 + dn ** 2) ** 0.5
    desplaz = (f"En el área del proyecto, la transformación {descripcion_transformacion()} desplaza un punto "
               f"aproximadamente {d:.0f} m (ΔE ≈ {de:.0f} m, ΔN ≈ {dn:.0f} m al pasar de PSAD56 a WGS 84).")
    return [
        {"id": "F01", "tema": "datum", "preguntas": ["diferencia entre PSAD56 y WGS84", "psad56 vs wgs84", "qué es PSAD56"],
         "respuesta": "PSAD56 (Datum Provisional Sudamericano de 1956) es un datum local basado en el elipsoide Internacional "
                      "de 1924 con punto fundamental en La Canoa (Venezuela); WGS 84 es un sistema geocéntrico global. Por eso "
                      "un mismo punto tiene coordenadas distintas en cada uno. " + desplaz, "fuente": "Base EPSG (pyproj)"},
        {"id": "F02", "tema": "epsg", "preguntas": ["código EPSG UTM 17 sur", "EPSG de PSAD56 UTM 17S", "EPSG WGS84 geográficas",
                                                    "qué EPSG uso"],
         "respuesta": "Códigos de uso frecuente en el proyecto: EPSG:4326 (WGS 84 geográficas), EPSG:32717 (WGS 84 / UTM zona
17S), "
                      "EPSG:24877 (PSAD56 / UTM zona 17S), EPSG:4248 (PSAD56 geográficas) y EPSG:31977 (SIRGAS 2000 / UTM zona
17S).",
         "fuente": "Base EPSG (pyproj)"},
        {"id": "F03", "tema": "marco_oficial", "preguntas": ["datum oficial en Ecuador", "sistema de referencia oficial del
país",
                                                             "marco de referencia del IGM"],
         "respuesta": "El marco geodésico nacional es SIRGAS-ECU, la densificación de SIRGAS en el Ecuador a cargo del Instituto
"
                      "Geográfico Militar (IGM). La realización y la época vigentes deben tomarse de la resolución vigente del
IGM; "
                      "consulte con el analista de geodesia antes de entregar información con fines legales.",
         "fuente": "Política interna (verificar normativa IGM vigente)"},
        {"id": "F04", "tema": "sirgas", "preguntas": ["qué es SIRGAS", "época de referencia en SIRGAS"],
         "respuesta": "SIRGAS (Sistema de Referencia Geocéntrico para las Américas) es la densificación regional del Marco "
                      "Internacional de Referencia Terrestre (ITRF). Como las placas tectónicas se mueven, las coordenadas SIRGAS
se "
                      "refieren a una época determinada; comparar coordenadas de épocas distintas exige modelar ese movimiento.",
         "fuente": "Definición general"},
        {"id": "F05", "tema": "zona_utm", "preguntas": ["en qué zona UTM está el proyecto", "qué es la zona 17 sur"],
         "respuesta": "El área del proyecto (alrededor de 79,7° O) está en la zona UTM 17 (de 84° O a 78° O, meridiano central
81° O), "
                      "hemisferio sur. Al este de 78° O comienza la zona 18, por lo que parte de la Amazonía cae en 18S.",
         "fuente": "Definición del sistema UTM"},
```

```
        {"id": "F06", "tema": "no_coinciden", "preguntas": ["mis coordenadas no coinciden con el catastro", "el punto sale
corrido en el mapa"],
         "respuesta": "Las causas más comunes son: (1) datum distinto, PSAD56 frente a WGS 84 (en esta zona la diferencia es de
unos "
                      "cientos de metros); (2) zona UTM equivocada; (3) Este y Norte invertidos; (4) error propio del receptor
GNSS. "
                      "Indique el datum de origen y puedo convertir o verificar el punto.", "fuente": "Práctica geodésica"},
        {"id": "F07", "tema": "precision_gnss", "preguntas": ["precisión de un GPS navegador", "diferencia entre RTK y GPS de
mano", "qué es RTK"],
         "respuesta": "Un receptor de navegación suele ofrecer exactitudes horizontales del orden de unos metros, y empeora bajo
"
                      "cobertura vegetal. El posicionamiento RTK usa correcciones de una base en tiempo real y, con solución
fija, "
                      "alcanza exactitud del orden de centímetros. Para linderos y control use GNSS geodésico.",
         "fuente": "Práctica geodésica"},
        {"id": "F08", "tema": "formatos", "preguntas": ["formato de archivo para entregar", "en qué formato entrego la
cartografía"],
         "respuesta": "Política interna (supuesto del prototipo): entregar vectores en GeoPackage o Shapefile en EPSG:32717,
ráster en "
                      "GeoTIFF, KMZ solo para visualización y mapas en PDF con cuadrícula, escala y metadatos.", "fuente":
"Política interna"},
        {"id": "F09", "tema": "politica_vuelo", "preguntas": ["política para vuelos de dron", "reglas para volar dron",
                                                              "anticipación para pedir un vuelo"],
         "respuesta": "Política interna (supuesto del prototipo): vuelos diurnos con línea de vista, viento ≤ 10 m/s, sin lluvia,
"
                      "visibilidad ≥ 5 km, piloto habilitado y solicitud con al menos un día de anticipación, salvo alertas de
prioridad "
                      "CRÍTICA o ALTA. Toda operación debe cumplir la normativa aeronáutica vigente.", "fuente": "Política
interna"},
        {"id": "F10", "tema": "desplazamiento", "preguntas": ["cuánto se desplaza un punto de PSAD56 a WGS84",
                                                               "cuántos metros cambia de PSAD a WGS"],
         "respuesta": desplaz, "fuente": "Cálculo con pyproj"},
        {"id": "F11", "tema": "utm", "preguntas": ["qué es una proyección UTM", "qué es el falso norte"],
         "respuesta": "UTM es una proyección transversa de Mercator dividida en 60 zonas de 6°. Usa factor de escala 0,9996 en el
"
                      "meridiano central, falso este de 500 000 m y, en el hemisferio sur, falso norte de 10 000 000 m para
evitar "
                      "coordenadas negativas.", "fuente": "Definición del sistema UTM"},
        {"id": "F12", "tema": "datum_def", "preguntas": ["qué es un datum geodésico", "qué es un datum"],
         "respuesta": "Un datum geodésico define la forma y el tamaño de la Tierra (un elipsoide) y su posición respecto del
planeta; "
                      "sobre él se expresan las coordenadas. Cambiar de datum cambia los números aunque el punto físico sea el
mismo.",
         "fuente": "Definición general"},
        {"id": "F13", "tema": "reporte_alerta", "preguntas": ["información para reportar una alerta", "datos de un reporte de
alerta"],
         "respuesta": "Indique la concesión o las coordenadas (con su datum), qué observó (maquinaria, desbroce, agua turbia,
ruido "
                      "nocturno, vía nueva), la fecha y la fuente. No incluya datos personales de terceros. Si hay riesgo para "
                      "personas, escriba «emergencia».", "fuente": "Procedimiento interno"},
        {"id": "F14", "tema": "sla_mapa", "preguntas": ["cuánto tarda una solicitud de mapa", "cuánto se demora un mapa"],
         "respuesta": "Plazo interno (supuesto del prototipo): 3 días hábiles para solicitudes estándar y 1 día hábil para
urgentes "
                      "aprobadas por la jefatura.", "fuente": "Política interna"},
        {"id": "F15", "tema": "epsg_def", "preguntas": ["qué significa EPSG", "qué es el código EPSG"],
         "respuesta": "EPSG es la base de datos de parámetros geodésicos creada por el European Petroleum Survey Group y
mantenida hoy "
                      "por el Comité de Geomática de IOGP. Cada sistema de referencia y cada transformación tiene un código
único.",
         "fuente": "IOGP"},
        {"id": "F16", "tema": "ortofoto", "preguntas": ["qué es una ortofoto", "para qué sirve una ortofoto"],
         "respuesta": "Una ortofoto es una imagen aérea corregida por el relieve y la inclinación de la cámara, de modo que tiene
"
                      "escala uniforme y permite medir distancias y áreas.", "fuente": "Definición general"},
        {"id": "F17", "tema": "mdt", "preguntas": ["qué es un modelo digital de terreno"],
         "respuesta": "Un modelo digital del terreno (MDT) representa la elevación del suelo desnudo; el modelo digital de
superficie "
                      "incluye además vegetación y construcciones.", "fuente": "Definición general"},
        {"id": "F18", "tema": "alturas", "preguntas": ["diferencia entre altura elipsoidal y ortométrica",
                                                       "altura del gps y altura sobre el nivel del mar"],
         "respuesta": "El GNSS entrega altura elipsoidal (h). La altura ortométrica (H), cercana a la altura sobre el nivel medio
del "
                      "mar, se obtiene con la ondulación del geoide (N): H = h − N, usando un modelo geoidal.", "fuente":
"Definición general"},
        {"id": "F19", "tema": "identificar_datum", "preguntas": ["cómo sé en qué datum están mis coordenadas", "cómo identifico
el datum"],
         "respuesta": "Revise los metadatos o la configuración del equipo. Si no existen, compare con un punto de control
conocido: en "
                      "esta zona, una diferencia sistemática de unos 440 m respecto del catastro en WGS 84 sugiere coordenadas en
PSAD56.",
         "fuente": "Práctica geodésica"},
    ]


def main() -> None:
    feats = []
    for cod, nom, fase, est, camp, pol, tit in CONCESIONES:
        feats.append({"type": "Feature", "properties": {"codigo": cod, "nombre": nom, "fase": fase, "estado": est,
                                                        "campamento": camp, "titular": tit, "capa": "concesion"},
                      "geometry": {"type": "Polygon", "coordinates": [pol]}})
    cod, nom, pol = AREA_PROTEGIDA
    feats.append({"type": "Feature", "properties": {"codigo": cod, "nombre": nom, "capa": "area_protegida"},
                  "geometry": {"type": "Polygon", "coordinates": [pol]}})
    for cod, nom, pt in COMUNIDADES:
        feats.append({"type": "Feature", "properties": {"codigo": cod, "nombre": nom, "capa": "comunidad"},
                      "geometry": {"type": "Point", "coordinates": pt}})
    geo = {"type": "FeatureCollection", "name": "base_sintetica_kuribot",
           "crs": {"type": "name", "properties": {"name": "urn:ogc:def:crs:EPSG::32717"}}, "features": feats}
```

```
    (DATOS / "concesiones.geojson").write_text(json.dumps(geo, ensure_ascii=False, indent=1), encoding="utf-8")
    (DATOS / "operacion.json").write_text(json.dumps(OPERACION, ensure_ascii=False, indent=2), encoding="utf-8")
    (DATOS / "faq.json").write_text(json.dumps(faq(), ensure_ascii=False, indent=2), encoding="utf-8")
    (DATOS / "tickets_semilla.json").write_text(json.dumps(TICKETS, ensure_ascii=False, indent=2), encoding="utf-8")
    print("Base sintética escrita en", DATOS)


if __name__ == "__main__":
    main()
```

D.12 Corpus de intenciones y particiones (datos/construir_corpus.py)

```
"""Construye corpus.json a partir de corpus_fuente.py (semilla fija, sin fuga entre particiones).

- POOL: cada plantilla con marcadores se instancia 3 veces con valores distintos; sin
  marcadores, una vez. Todas las instancias de una plantilla comparten «grupo», de modo que
  la validación cruzada (StratifiedGroupKFold) nunca separa variantes de la misma plantilla.
- PRUEBA: plantillas redactadas aparte, instanciadas una vez («prueba_limpia»).
- PRUEBA CON RUIDO: la misma prueba con perturbaciones ortográficas sembradas (sin tildes,
  minúsculas, abreviaturas de chat y un error de tecleo por frase con probabilidad 0,7).
  El ruido se aplica a la plantilla antes de insertar entidades, para no corromper códigos.

Uso: python datos/construir_corpus.py
"""
from __future__ import annotations

import json
import random
import re
import sys
import unicodedata
from pathlib import Path

RAIZ = Path(__file__).resolve().parent
sys.path.insert(0, str(RAIZ))
from corpus_fuente import POOL, POOL_ITERACION_2, PRUEBA, VALORES  # noqa: E402

SEMILLA = 2026
MARCADOR = re.compile(r"\{(\w+)\}")
VECINAS = {"a": "sq", "e": "wr", "i": "uo", "o": "ip", "u": "yi", "s": "ad", "n": "bm", "r": "et",
           "t": "ry", "l": "kñ", "c": "xv", "d": "sf", "m": "n", "p": "o"}


def instanciar(plantilla: str, rng: random.Random) -> str:
    return MARCADOR.sub(lambda m: rng.choice(VALORES[m.group(1)]), plantilla)


def sin_tildes(t: str) -> str:
    nfkd = unicodedata.normalize("NFD", t)
    return "".join(c for c in nfkd if unicodedata.category(c) != "Mn" or c == "̃").replace("ñ", "ñ")


def error_tecleo(palabra: str, rng: random.Random) -> str:
    i = rng.randrange(1, len(palabra) - 1)
    op = rng.choice(["intercambio", "omision", "duplicado", "vecina"])
    if op == "intercambio":
        return palabra[:i] + palabra[i + 1] + palabra[i] + palabra[i + 2:]
    if op == "omision":
        return palabra[:i] + palabra[i + 1:]
    if op == "duplicado":
        return palabra[:i] + palabra[i] + palabra[i:]
    c = palabra[i]
    return palabra[:i] + (rng.choice(VECINAS[c]) if c in VECINAS else c) + palabra[i + 1:]


def ruido(plantilla: str, rng: random.Random) -> str:
    t = plantilla.lower()
    t = re.sub(r"[¿¡]", "", t)
    if rng.random() < 0.8:
        t = sin_tildes(t)
    for patron, reemplazo, p in [(r"\bque\b", "q", 0.5), (r"\bpor favor\b", "porfa", 0.7),
                                 (r"\bpara\b", "pa", 0.3), (r"\bestá\b|\besta\b", "sta", 0.2)]:
        if rng.random() < p:
            t = re.sub(patron, reemplazo, t)
    if rng.random() < 0.7:
        tokens = t.split(" ")
        idx = [i for i, w in enumerate(tokens) if len(w) >= 5 and w.isalpha()]
        if idx:
            j = rng.choice(idx)
            tokens[j] = error_tecleo(tokens[j], rng)
        t = " ".join(tokens)
    return t


def main() -> None:
    rng = random.Random(SEMILLA)
    filas = []
    for intencion, plantillas in POOL.items():
        for k, pl in enumerate(plantillas):
            n = 3 if MARCADOR.search(pl) else 1
            vistos = set()
            for _ in range(n * 4):
                texto = instanciar(pl, rng)
                if texto not in vistos:
                    vistos.add(texto)
                if len(vistos) == n:
                    break
```

```
            for texto in sorted(vistos):
                filas.append({"texto": texto, "intencion": intencion, "particion": "pool",
                              "grupo": f"{intencion}:{k:02d}"})
    for intencion, plantillas in PRUEBA.items():
        for k, pl in enumerate(plantillas):
            filas.append({"texto": instanciar(pl, rng), "intencion": intencion, "particion": "prueba_limpia",
                          "grupo": f"T-{intencion}:{k:02d}"})
            filas.append({"texto": instanciar(ruido(pl, rng), rng), "intencion": intencion,
                          "particion": "prueba_ruido", "grupo": f"T-{intencion}:{k:02d}"})
    # Plantillas agregadas en la iteración 2, con un generador propio para no alterar las particiones previas.
    rng2 = random.Random(SEMILLA + 2)
    for intencion, plantillas in POOL_ITERACION_2.items():
        for k, pl in enumerate(plantillas):
            vistos = set()
            for _ in range(12):
                vistos.add(instanciar(pl, rng2))
                if len(vistos) == 3:
                    break
            for texto in sorted(vistos):
                filas.append({"texto": texto, "intencion": intencion, "particion": "pool",
                              "grupo": f"{intencion}:it2-{k:02d}", "iteracion": 2})
    for f in filas:
        f.setdefault("iteracion", 1)
    # Control de fuga: ninguna frase de prueba puede aparecer en el pool.
    pool_textos = {f["texto"].lower() for f in filas if f["particion"] == "pool"}
    fugas = [f["texto"] for f in filas if f["particion"] != "pool" and f["texto"].lower() in pool_textos]
    meta = {"semilla": SEMILLA, "intenciones": sorted(POOL), "fugas_exactas_detectadas": fugas,
            "conteos": {p: sum(1 for f in filas if f["particion"] == p) for p in ("pool", "prueba_limpia", "prueba_ruido")},
            "plantillas_pool": sum(len(v) for v in POOL.values()),
            "plantillas_iteracion_2": sum(len(v) for v in POOL_ITERACION_2.values()),
            "pool_iteracion_2": sum(1 for f in filas if f.get("iteracion") == 2)}
    (RAIZ / "corpus.json").write_text(json.dumps({"meta": meta, "datos": filas}, ensure_ascii=False, indent=1),
                                      encoding="utf-8")
    print(json.dumps(meta, ensure_ascii=False, indent=1))


if __name__ == "__main__":
    main()
```

D.13 Entrenamiento, validación cruzada y evaluación (experimentos/entrenar_nlp.py)

```
"""Experimento NLP de KuriBot: validación cruzada, entrenamiento final, evaluación y exportación.

Uso:
    python experimentos/entrenar_nlp.py --version v1            # CV de 4 modelos + modelo final
    python experimentos/entrenar_nlp.py --version v2 --aumento  # iteración 2 (sin CV), solo M0 y M3

Salidas en resultados/ (JSON y CSV) y modelos/ (artefactos de inferencia).
"""
from __future__ import annotations

import argparse
import json
import random
import shutil
import sys
import time
from pathlib import Path

import numpy as np
import torch
from scipy.stats import binomtest
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, f1_score
from sklearn.model_selection import StratifiedGroupKFold

RAIZ = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(RAIZ))
sys.path.insert(0, str(RAIZ / "datos"))
from kuribot import nlu_modelos as NM  # noqa: E402

torch.set_num_threads(2)
RES = RAIZ / "resultados"
MOD = RAIZ / "modelos"
RES.mkdir(exist_ok=True)
SEMILLA = 42


def cargar(iteracion: int = 1):
    d = json.loads((RAIZ / "datos" / "corpus.json").read_text(encoding="utf-8"))["datos"]
    d = [f for f in d if f.get("iteracion", 1) <= iteracion]
    etiquetas = sorted({f["intencion"] for f in d})
    idx = {e: i for i, e in enumerate(etiquetas)}
    parts = {p: [f for f in d if f["particion"] == p] for p in ("pool", "prueba_limpia", "prueba_ruido")}
    return etiquetas, idx, parts


def vectores_fasttext():
    import spacy
    nlp = spacy.load("es_core_news_lg", exclude=["tok2vec", "morphologizer", "parser", "senter", "attribute_ruler",
                                                 "lemmatizer", "ner"])
    vocab = nlp.vocab

    def f(t):
        return vocab[t].vector.copy() if vocab.has_vector(t) else None
    return f
```

```
def aumentar(textos, etiquetas, grupos, semilla=7):
    """Iteración 2: añade una copia perturbada (sin tildes, abreviaturas, errata) de cada frase de entrenamiento."""
    from construir_corpus import ruido
    rng = random.Random(semilla)
    xt, yt, gt = list(textos), list(etiquetas), list(grupos)
    for t, y, g in zip(textos, etiquetas, grupos):
        xt.append(ruido(t, rng))
        yt.append(y)
        gt.append(g)
    return xt, np.array(yt), gt


EPOCAS_M3 = 6
PACIENCIA_M3 = 2


def fabricar(nombre, n, semilla, vec):
    if nombre == "M0":
        return NM.ModeloM0(n, semilla)
    if nombre == "M1":
        return NM.ModeloM1(n, semilla, vectores=vec)
    if nombre == "M2":
        return NM.ModeloM2(n, semilla, ruta_beto=str(MOD / "beto_base"))
    return NM.ModeloM3(n, semilla, ruta_beto=str(MOD / "beto_base"), epocas=EPOCAS_M3, paciencia=PACIENCIA_M3)


def separar_dev(y, grupos, semilla):
    sgk = StratifiedGroupKFold(n_splits=6, shuffle=True, random_state=semilla)
    tr, dev = next(sgk.split(np.zeros(len(y)), y, grupos))
    return tr, dev


def validacion_cruzada(modelos, pool, idx, n, vec):
    x = [f["texto"] for f in pool]
    y = np.array([idx[f["intencion"]] for f in pool])
    g = [f["grupo"] for f in pool]
    sgk = StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=SEMILLA)
    out = {m: [] for m in modelos}
    for k, (tr, te) in enumerate(sgk.split(np.zeros(len(y)), y, g)):
        tr_in, dev_in = separar_dev(y[tr], [g[i] for i in tr], SEMILLA + k)
        tr_idx, dev_idx = tr[tr_in], tr[dev_in]
        for m in modelos:
            t0 = time.time()
            mod = fabricar(m, n, SEMILLA + k, vec)
            mod.entrenar([x[i] for i in tr_idx], y[tr_idx], [x[i] for i in dev_idx], y[dev_idx])
            pred = mod.logits([x[i] for i in te]).argmax(1)
            r = {"pliegue": k + 1, "exactitud": accuracy_score(y[te], pred), "f1_macro": f1_score(y[te], pred, average="macro"),
                 "n_prueba": int(len(te)), "segundos": round(time.time() - t0, 1)}
            out[m].append(r)
            print(f"[CV] pliegue {k+1} {m}: F1={r['f1_macro']:.3f} acc={r['exactitud']:.3f} ({r['segundos']} s)", flush=True)
            del mod
    resumen = {m: {"f1_macro_media": float(np.mean([r["f1_macro"] for r in v])),
                   "f1_macro_de": float(np.std([r["f1_macro"] for r in v], ddof=1)),
                   "exactitud_media": float(np.mean([r["exactitud"] for r in v])),
                   "exactitud_de": float(np.std([r["exactitud"] for r in v], ddof=1)), "pliegues": v}
               for m, v in out.items()}
    return resumen


def bootstrap_f1(y, pred, B=2000, semilla=0):
    rng = np.random.default_rng(semilla)
    n = len(y)
    vals = [f1_score(y[i], pred[i], average="macro") for i in (rng.integers(0, n, n) for _ in range(B))]
    return [float(np.percentile(vals, 2.5)), float(np.percentile(vals, 97.5))]


def mcnemar(y, a, b):
    """Prueba exacta de McNemar (binomial sobre pares discordantes)."""
    a_ok, b_ok = a == y, b == y
    n01 = int((a_ok & ~b_ok).sum())
    n10 = int((~a_ok & b_ok).sum())
    p = binomtest(n10, n01 + n10, 0.5).pvalue if n01 + n10 else 1.0
    return {"solo_A_acierta": n01, "solo_B_acierta": n10, "p_valor": float(p)}


def evaluar(nombre, mod, T, sets, etiquetas):
    res = {}
    for clave, (xs, ys) in sets.items():
        z = mod.logits(xs)
        p1, pT = NM.softmax(z), NM.softmax(z, T)
        pred = z.argmax(1)
        k = int((pred == ys).sum())
        res[clave] = {
            "exactitud": accuracy_score(ys, pred), "exactitud_ic95_wilson": NM.wilson(k, len(ys)),
            "f1_macro": f1_score(ys, pred, average="macro"), "f1_macro_ic95_bootstrap": bootstrap_f1(ys, pred),
            "f1_ponderado": f1_score(ys, pred, average="weighted"),
            "ece_sin_calibrar": NM.ece(p1, ys), "ece_calibrado": NM.ece(pT, ys),
            "reporte": classification_report(ys, pred, labels=list(range(len(etiquetas))), target_names=etiquetas,
                                             output_dict=True, zero_division=0),
            "matriz_confusion": confusion_matrix(ys, pred, labels=list(range(len(etiquetas)))).tolist(),
            "pred": pred.tolist(), "conf_cal": pT.max(1).tolist(), "probs_cal": pT.round(5).tolist(),
        }
        print(f"[TEST] {nombre} {clave}: acc={res[clave]['exactitud']:.3f} F1={res[clave]['f1_macro']:.3f} "
              f"ECE {res[clave]['ece_sin_calibrar']:.3f}->{res[clave]['ece_calibrado']:.3f}", flush=True)
    return res
```

```
def analisis_umbral(z_dev, y_dev, z_test, y_test, T, i_oos):
    """Selección del umbral de confianza en validación y su efecto en prueba (abstención -> fuera_alcance)."""
    def aplicar(z, tau):
        p = NM.softmax(z, T)
        pred = p.argmax(1).copy()
        pred[p.max(1) < tau] = i_oos
        return pred
    grid = np.round(np.arange(0.20, 0.91, 0.05), 2)
    f_dev = [f1_score(y_dev, aplicar(z_dev, t), average="macro") for t in grid]
    tau = float(grid[int(np.argmax(f_dev))])
    curva = []
    for t in grid:
        p = NM.softmax(z_test, T)
        acept = p.max(1) >= t
        pred = aplicar(z_test, t)
        en_alcance = y_test != i_oos
        curva.append({"tau": float(t), "cobertura": float(acept.mean()),
                      "exactitud_aceptadas": float((p.argmax(1)[acept] == y_test[acept]).mean()) if acept.any() else
float("nan"),
                      "f1_macro_con_abstencion": float(f1_score(y_test, pred, average="macro")),
                      "recall_fuera_alcance": float((pred[~en_alcance] == i_oos).mean()),
                      "exactitud_en_alcance": float((pred[en_alcance] == y_test[en_alcance]).mean())})
    return {"tau_elegido_en_validacion": tau, "f1_dev_por_tau": dict(zip(map(float, grid), map(float, f_dev))), "curva_prueba":
curva}


def exportar_m3(mod, etiquetas, T, tau, destino: Path, extra: dict):
    from safetensors.torch import save_file
    if destino.exists():
        shutil.rmtree(destino)
    destino.mkdir(parents=True)
    estado = {k: v.detach().to(torch.float16).contiguous() for k, v in mod.red.state_dict().items()}
    save_file(estado, str(destino / "clasificador.safetensors"))
    for f in ("config.json", "vocab.txt", "tokenizer.json", "tokenizer_config.json"):
        shutil.copy(MOD / "beto_base" / f, destino / ("bert_config.json" if f == "config.json" else f))
    (destino / "kuribot_nlu.json").write_text(json.dumps({
        "etiquetas": etiquetas, "temperatura": T, "umbral_confianza": tau, "max_len": 48,
        "preprocesamiento": "kuribot.entidades.enmascarar_entidades", "pesos": "float16", **extra}, ensure_ascii=False,
indent=2),
        encoding="utf-8")


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--version", default="v1")
    ap.add_argument("--aumento", action="store_true")
    ap.add_argument("--sin-cv", action="store_true")
    ap.add_argument("--modelos", default="M0,M1,M2,M3")
    ap.add_argument("--iteracion", type=int, default=1, help="1: corpus original; 2: con plantillas agregadas tras el análisis de
fallos")
    ap.add_argument("--epocas-m3", type=int, default=6)
    ap.add_argument("--paciencia-m3", type=int, default=2)
    args = ap.parse_args()
    global EPOCAS_M3, PACIENCIA_M3
    EPOCAS_M3, PACIENCIA_M3 = args.epocas_m3, args.paciencia_m3
    modelos = args.modelos.split(",")
    etiquetas, idx, parts = cargar(args.iteracion)
    n = len(etiquetas)
    vec = vectores_fasttext() if "M1" in modelos else None
    salida = {"version": args.version, "aumento": args.aumento, "iteracion_corpus": args.iteracion,
              "epocas_max_m3": args.epocas_m3, "paciencia_m3": args.paciencia_m3, "etiquetas": etiquetas, "semilla": SEMILLA,
              "entorno": {"torch": torch.__version__, "hilos_cpu": torch.get_num_threads()}}
    if not args.sin_cv:
        salida["validacion_cruzada"] = validacion_cruzada(modelos, parts["pool"], idx, n, vec)
        (RES / f"nlp_cv_{args.version}.json").write_text(json.dumps(salida["validacion_cruzada"], indent=1), encoding="utf-8")

    pool = parts["pool"]
    x = [f["texto"] for f in pool]
    y = np.array([idx[f["intencion"]] for f in pool])
    g = [f["grupo"] for f in pool]
    tr, dev = separar_dev(y, g, SEMILLA)
    xtr, ytr, gtr = [x[i] for i in tr], y[tr], [g[i] for i in tr]
    if args.aumento:
        xtr, ytr, gtr = aumentar(xtr, ytr, gtr)
    xdev, ydev = [x[i] for i in dev], y[dev]
    sets = {k: ([f["texto"] for f in parts[k]], np.array([idx[f["intencion"]] for f in parts[k]]))
            for k in ("prueba_limpia", "prueba_ruido")}
    salida["particion_final"] = {"entrenamiento": len(xtr), "validacion": len(xdev), "prueba_limpia": len(sets["prueba_limpia"]
[1]),
                                 "prueba_ruido": len(sets["prueba_ruido"][1])}
    finales, preds = {}, {}
    for m in modelos:
        t0 = time.time()
        mod = fabricar(m, n, SEMILLA, vec)
        info = mod.entrenar(xtr, ytr, xdev, ydev)
        z_dev = mod.logits(xdev)
        T = NM.ajustar_temperatura(z_dev, ydev)
        r = evaluar(m, mod, T, sets, etiquetas)
        if m == "M2":
            mod.memo.clear()  # sin caché: la latencia debe incluir el paso por BETO
        lat = NM.latencia_ms(mod, sets["prueba_limpia"][0][:60])
        finales[m] = {"nombre": mod.nombre, "temperatura": T, "parametros_entrenables": NM.n_parametros(mod),
                      "segundos_entrenamiento": round(time.time() - t0, 1), "latencia": lat,
                      "f1_macro_validacion": float(f1_score(ydev, z_dev.argmax(1), average="macro")),
                      "info": {k: v for k, v in info.items()}, "prueba": r}
        preds[m] = {k: np.array(v["pred"]) for k, v in r.items()}
        if m in ("M3", "M0"):
```

```
            finales[m]["umbral"] = analisis_umbral(z_dev, ydev, mod.logits(sets["prueba_limpia"][0]), sets["prueba_limpia"][1],
T,
                                                   etiquetas.index("fuera_alcance"))
        if m == "M3":
            tau = finales[m]["umbral"]["tau_elegido_en_validacion"]
            exportar_m3(mod, etiquetas, T, tau, MOD / f"intenciones_beto_{args.version}",
                        {"version": args.version, "aumento": args.aumento,
                         "f1_macro_prueba_limpia": r["prueba_limpia"]["f1_macro"],
                         "f1_macro_prueba_ruido": r["prueba_ruido"]["f1_macro"]})
        if m == "M0":
            import joblib
            joblib.dump({"pipe": mod.pipe, "etiquetas": etiquetas, "temperatura": T}, MOD /
f"intenciones_tfidf_{args.version}.joblib")
        del mod
    comparaciones = {}
    if "M3" in preds:
        for otro in [m for m in modelos if m != "M3"]:
            for k, (xs, ys) in sets.items():
                comparaciones[f"M3_vs_{otro}_{k}"] = mcnemar(ys, preds["M3"][k], preds[otro][k])
    salida["modelos"] = finales
    salida["mcnemar"] = comparaciones
    (RES / f"nlp_final_{args.version}.json").write_text(json.dumps(salida, ensure_ascii=False, indent=1, default=float),
encoding="utf-8")
    # Predicciones de prueba en CSV (para el informe y la auditoría de errores)
    import csv
    with open(RES / f"predicciones_prueba_{args.version}.csv", "w", newline="", encoding="utf-8") as fh:
        w = csv.writer(fh)
        w.writerow(["conjunto", "texto", "real"] + [f"pred_{m}" for m in modelos] + [f"conf_{m}" for m in modelos])
        for k, (xs, ys) in sets.items():
            for i, (t, yv) in enumerate(zip(xs, ys)):
                w.writerow([k, t, etiquetas[yv]] + [etiquetas[preds[m][k][i]] for m in modelos]
                           + [round(finales[m]["prueba"][k]["conf_cal"][i], 4) for m in modelos])
    print("Listo:", RES / f"nlp_final_{args.version}.json")


if __name__ == "__main__":
    main()
```

D.14 Latencia sin caché y McNemar del modelo final (experimentos/complementos.py)

```
"""Cálculos complementarios del informe:
(1) latencia de M2 sin caché de vectores (en el experimento principal la caché la subestimaba);
(2) McNemar del modelo de producción (M3 v2) frente a la mejor línea base (M0 v1) sobre los mismos conjuntos de prueba.
Uso: python experimentos/complementos.py
"""
from __future__ import annotations

import csv
import json
import sys
from pathlib import Path

import numpy as np
import torch
from sklearn.metrics import f1_score

RAIZ = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(RAIZ))
sys.path.insert(0, str(RAIZ / "experimentos"))
import entrenar_nlp as EN  # noqa: E402
from kuribot import nlu_modelos as NM  # noqa: E402

torch.set_num_threads(2)
RES = RAIZ / "resultados"


def latencia_m2():
    etiquetas, idx, parts = EN.cargar(1)
    pool = parts["pool"]
    x = [f["texto"] for f in pool]
    y = np.array([idx[f["intencion"]] for f in pool])
    g = [f["grupo"] for f in pool]
    tr, dev = EN.separar_dev(y, g, EN.SEMILLA)
    m = NM.ModeloM2(len(etiquetas), EN.SEMILLA, ruta_beto=str(RAIZ / "modelos" / "beto_base"))
    m.entrenar([x[i] for i in tr], y[tr], [x[i] for i in dev], y[dev])
    xs = [f["texto"] for f in parts["prueba_limpia"]]
    ys = np.array([idx[f["intencion"]] for f in parts["prueba_limpia"]])
    f1 = f1_score(ys, m.logits(xs).argmax(1), average="macro")
    m.memo.clear()
    lat = NM.latencia_ms(m, xs[:60])
    out = {"f1_macro_prueba_limpia_reentrenado": f1, **lat,
           "nota": "M2 reentrenado con la misma partición y semilla de v1; latencia medida sin caché de vectores."}
    (RES / "latencia_m2_v1.json").write_text(json.dumps(out, indent=1), encoding="utf-8")
    print("M2", out)


def mcnemar_produccion():
    from scipy.stats import binomtest
    f1 = list(csv.DictReader(open(RES / "predicciones_prueba_v1.csv", encoding="utf-8")))
    f2 = list(csv.DictReader(open(RES / "predicciones_prueba_v2.csv", encoding="utf-8")))
    out = {}
    for conj in ("prueba_limpia", "prueba_ruido"):
        a = [r for r in f2 if r["conjunto"] == conj]
        b = [r for r in f1 if r["conjunto"] == conj]
        assert [r["texto"] for r in a] == [r["texto"] for r in b]
        ok_a = np.array([r["pred_M3"] == r["real"] for r in a])
        ok_b = np.array([r["pred_M0"] == r["real"] for r in b])
```

```
        ok_c = np.array([r["pred_M3"] == r["real"] for r in b])
        n01, n10 = int((ok_a & ~ok_b).sum()), int((~ok_a & ok_b).sum())
        m01, m10 = int((ok_a & ~ok_c).sum()), int((~ok_a & ok_c).sum())
        out[conj] = {"M3v2_vs_M0v1": {"solo_M3v2": n01, "solo_M0v1": n10, "p_valor": binomtest(n10, n01 + n10, 0.5).pvalue},
                     "M3v2_vs_M3v1": {"solo_M3v2": m01, "solo_M3v1": m10, "p_valor": binomtest(m10, m01 + m10, 0.5).pvalue}}
    (RES / "mcnemar_produccion.json").write_text(json.dumps(out, indent=1), encoding="utf-8")
    print(json.dumps(out, indent=1))


if __name__ == "__main__":
    mcnemar_produccion()
    latencia_m2()
```

D.15 Pruebas unitarias (pruebas/test_unitarias.py)

```
"""Pruebas unitarias de KuriBot (pytest). Ejecutar: python -m pytest pruebas/test_unitarias.py -v"""
from __future__ import annotations

import datetime as dt
import itertools
import sys
from pathlib import Path

import pytest

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from kuribot import entidades as E  # noqa: E402
from kuribot import geodesia as G  # noqa: E402
from kuribot.conocimiento import BaseConocimiento  # noqa: E402
from kuribot.razonamiento import NODOS, RedAlertas, motor_alertas, primera_ventana, sensibilidad  # noqa: E402


@pytest.fixture(scope="module")
def kb():
    return BaseConocimiento()


@pytest.fixture(scope="module")
def red():
    return RedAlertas()


# ---------------------------------------------------------------- geodesia
def test_ida_y_vuelta_psad56_wgs84():
    e, n = 642600.0, 9662480.0
    we, wn = G.psad56_utm_a_wgs84_utm(e, n)
    pe, pn = G.wgs84_utm_a_psad56_utm(we, wn)
    assert abs(pe - e) < 0.01 and abs(pn - n) < 0.01


def test_desplazamiento_psad56_en_zona_del_proyecto():
    we, wn = G.psad56_utm_a_wgs84_utm(642600.0, 9662480.0)
    d = ((we - 642600) ** 2 + (wn - 9662480) ** 2) ** 0.5
    assert 400 < d < 480          # orden de magnitud conocido para Ecuador continental


def test_geograficas_a_utm():
    e, n = G.a_wgs84_utm(G.Punto(-79.7301, -3.0512, "geo", "WGS84"))
    assert abs(e - 641120.7) < 1 and abs(n - 9662664.3) < 1


def test_operacion_epsg_3990_disponible():
    assert "3990" in G.descripcion_transformacion()


# ---------------------------------------------------------------- entidades
@pytest.mark.parametrize("texto,esperado", [
    ("E 642350 N 9662120", (642350, 9662120, "utm")), ("642350, 9662120", (642350, 9662120, "utm")),
    ("x=644800 y=9661500", (644800, 9661500, "utm")), ("-3.0512, -79.7301", (-79.7301, -3.0512, "geo")),
    ("3°03'04\"S 79°43'48\"W", (-79.73, -3.0511, "geo")),
])
def test_extraer_coordenadas(texto, esperado):
    c = E.extraer_coordenada(texto)
    assert c is not None and c.punto.tipo == esperado[2]
    assert abs(c.punto.x - esperado[0]) < 1e-3 and abs(c.punto.y - esperado[1]) < 1e-3


def test_datum_detectado():
    assert E.extraer_datum("643990 9661010 psad56") == "PSAD56"
    assert E.extraer_datum("en WGS 84") == "WGS84"
    assert E.extraer_datum("sin datum") is None


def test_concesion_por_codigo_nombre_y_errata(kb):
    assert E.extraer_concesion("revisa cm 103", kb.nombres) == "CM-103"
    assert E.extraer_concesion("estado de pena blanca", kb.nombres) == "CM-101"
    assert E.extraer_concesion("la consecion el robel", kb.nombres) == "CM-104"
    assert E.extraer_concesion("una concesión cualquiera", kb.nombres) is None


def test_evidencias_con_negacion():
    ev = E.extraer_evidencias("hay maquinaria, no hay personas y el agua está turbia")
    assert ev == {"M": True, "P": False, "T": True}


def test_fechas_relativas():
```

```
    hoy = dt.date(2026, 11, 9)  # lunes
    assert E.extraer_fecha("mañana", hoy) == 1
    assert E.extraer_fecha("pasado mañana", hoy) == 2
    assert E.extraer_fecha("el viernes", hoy) == 4
    assert E.extraer_fecha("el 12 de noviembre", hoy) == 3


def test_cedula_y_enmascarado():
    assert E.cedula_valida("3000000004") and not E.cedula_valida("3000000005")
    texto, tipos = E.enmascarar_pii("cédula 3000000004, celular 0991234567, correo a@b.ec")
    assert "3000000004" not in texto and "0991234567" not in texto and set(tipos) == {"cédula", "teléfono", "correo"}


def test_emergencia_palabras_clave():
    assert E.hay_emergencia("hay gente armada en la vía")
    assert not E.hay_emergencia("se escuchan voladuras de noche")


# ---------------------------------------------------------------- red bayesiana
def test_pgmpy_coincide_con_enumeracion(red):
    for valores in itertools.product([0, 1], repeat=4):
        ev = dict(zip(["H", "A", "M", "T"], valores))
        assert abs(red.posterior(ev) - red.posterior_enumeracion(ev)) < 1e-9


def test_prior_sin_evidencia(red):
    assert abs(red.posterior({"H": 0}) - 0.10) < 1e-9 and abs(red.posterior({"H": 1}) - 0.30) < 1e-9


def test_explicacion_alternativa_reduce_posterior(red):
    ev = {"H": 0, "M": 1, "S": 1}
    assert red.posterior({**ev, "A": 1}) < 0.3 < 0.9 < red.posterior({**ev, "A": 0})


def test_monotonia_evidencias(red):
    base = {"H": 0, "A": 0}
    for nodo in [n for n in NODOS if n not in ("H", "A", "I")]:
        assert red.posterior({**base, nodo: 1}) > red.posterior(base) > red.posterior({**base, nodo: 0})


def test_sensibilidad_ordenada():
    filas = sensibilidad({"H": 0, "A": 1, "M": 1, "S": 1})
    assert filas[0]["rango"] >= filas[-1]["rango"] and filas[0]["parametro"].startswith("P(I=1")


# ---------------------------------------------------------------- reglas
def test_emergencia_prevalece():
    f = motor_alertas().inferir({"emergencia": True, "p_intrusion": 0.05, "ubicacion": "propia"})
    assert f.hechos["prioridad"] == "CRÍTICA" and f.hechos["prohibir_despliegue"] and f.disparadas[0] == "R01"


def test_tercero_no_se_califica_como_intrusion():
    f = motor_alertas().inferir({"ubicacion": "tercero", "p_intrusion": 0.95})
    assert f.hechos["prioridad"] == "REVISIÓN LEGAL" and "R21" not in f.disparadas


@pytest.mark.parametrize("p,prioridad", [(0.9, "CRÍTICA"), (0.6, "ALTA"), (0.3, "MEDIA"), (0.05, "BAJA")])
def test_umbral_prioridad(p, prioridad):
    assert motor_alertas().inferir({"ubicacion": "propia", "p_intrusion": p}).hechos["prioridad"] == prioridad


def test_prioridad_unica_y_determinista():
    import random
    from kuribot.razonamiento import REGLAS_ALERTA, MotorReglas
    hechos = {"ubicacion": "propia", "p_intrusion": 0.85, "comunidad_cercana": True, "vuelo_dias": 0}
    ref = MotorReglas(REGLAS_ALERTA).inferir(hechos).hechos
    for s in range(20):
        reglas = REGLAS_ALERTA[:]
        random.Random(s).shuffle(reglas)
        assert MotorReglas(reglas).inferir(hechos).hechos == ref


def test_supervision_humana_en_zona_sensible():
    f = motor_alertas().inferir({"ubicacion": "propia", "p_intrusion": 0.6, "area_protegida": True})
    assert f.hechos["revision_humana"] and f.hechos["aprobacion_supervisor"]


# ---------------------------------------------------------------- vuelos y base de conocimiento
def test_ventana_vuelo_evita_viento(kb):
    v = primera_ventana(1, "Norte", kb.op)
    assert v["dia"] == 2 and any("viento" in m for m in v["evaluados"][0]["motivos"])


def test_anticipacion_minima(kb):
    assert primera_ventana(0, "Sur", kb.op, urgente=False)["dia"] == 1
    assert primera_ventana(0, "Sur", kb.op, urgente=True)["dia"] == 0


def test_analisis_espacial(kb):
    assert kb.analizar_punto(642350, 9662120).ubicacion == "propia"
    assert kb.analizar_punto(642000, 9663600).ubicacion == "tercero"
    a = kb.analizar_punto(641100, 9664950)
    assert a.ubicacion == "amortiguamiento" and a.cercana == "CM-104" and abs(a.distancia_lindero_m - 100) < 1


def test_ticket_inexistente_no_se_inventa(kb):
```

```
    assert kb.ticket("GEO-2026-0999") is None and kb.ticket("GEO-2026-0412")["estado"] == "En elaboración"


def test_areas_derivadas_de_geometria(kb):
    assert kb.ficha("CM-103")["area_ha"] == 480.0


def test_riesgo_de_datum_cambia_la_concesion(kb):
    """Las mismas cifras en PSAD56 y en WGS 84 caen en concesiones distintas: el datum importa."""
    we, wn = G.psad56_utm_a_wgs84_utm(643100, 9662100)
    assert kb.analizar_punto(643100, 9662100).concesion == "CM-102"
    assert kb.analizar_punto(we, wn).concesion == "CM-101"
```

D.16 Suite de aceptación y confirmación (pruebas/suite_aceptacion.py)

```
"""Suite de aceptación funcional de KuriBot: 34 escenarios (43 turnos) con criterios explícitos.

Cada escenario empieza con un estado de diálogo limpio. Se evalúa el ÚLTIMO turno de cada
escenario (los turnos previos preparan el contexto) y se verifica, además, cada turno
intermedio que deba pedir un dato. Criterios de aprobación por turno evaluado:
  (a) intención esperada (o conjunto aceptable), (b) acción o regla esperada,
  (c) fragmentos de contenido obligatorios, (d) latencia < 1 500 ms.

Además, CONFIRMACION contiene 8 escenarios redactados DESPUÉS de cerrar la iteración 2 y ejecutados una sola
vez, sin ajustes posteriores, para estimar el desempeño en casos no usados durante el desarrollo.

Uso: python pruebas/suite_aceptacion.py --version v1 | --version v2 [--confirmacion]
"""
from __future__ import annotations

import argparse
import json
import sys
from pathlib import Path

RAIZ = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(RAIZ))
from kuribot.conocimiento import BaseConocimiento  # noqa: E402
from kuribot.dialogo import KuriBot  # noqa: E402
from kuribot.nlu import ClasificadorIntenciones  # noqa: E402

# (id, descripción, [turnos], criterios del último turno, criterios de turnos intermedios {indice: dict})
ESCENARIOS = [
    ("A01", "Saludo y aviso de transparencia", ["buenos días, ¿con quién hablo?"],
     {"intencion": ["saludo"], "contiene": ["asistente automatizado"]}),
    ("A02", "Capacidades", ["¿qué cosas puedes resolver?"], {"intencion": ["ayuda_capacidades"], "contiene": ["consultar una
concesión"]}),
    ("A03", "Ficha de concesión por nombre", ["dame la ficha de la concesión La Ventana"],
     {"intencion": ["consultar_concesion"], "contiene": ["CM-103", "480"]}),
    ("A04", "Estado de concesión suspendida", ["¿cerro azul está activa?"], {"intencion": ["consultar_concesion"], "contiene":
["suspendida"]}),
    ("A05", "Ranura faltante de concesión", ["quiero consultar una concesión", "CM-106"],
     {"intencion": ["consultar_concesion"], "contiene": ["Los Laureles"]}, {0: {"accion": "solicitar_dato:concesion"}}),
    ("A06", "Punto WGS84 dentro de concesión", ["¿el punto E 642350 N 9662120 WGS84 está dentro de la concesión?"],
     {"intencion": ["verificar_punto"], "contiene": ["DENTRO de CM-101"], "reglas": ["R10"]}),
    ("A07", "Punto sin datum: se pide y luego cerca de comunidad", ["verifica si 644800 9661500 cae en nuestra área", "wgs84"],
     {"intencion": ["verificar_punto"], "contiene": ["DENTRO de CM-103", "comunidad"], "reglas": ["R14"]},
     {0: {"accion": "solicitar_dato:datum"}}),
    ("A08", "Punto PSAD56 transformado antes del cruce", ["¿E 642600 N 9662480 PSAD56 en qué concesión queda?"],
     {"intencion": ["verificar_punto"], "contiene": ["Transformé", "DENTRO de CM-101"]}),
    ("A09", "Punto en franja de amortiguamiento", ["chequea 641100 9664950 wgs84 contra el catastro"],
     {"intencion": ["verificar_punto"], "contiene": ["franja de 500 m"], "reglas": ["R11"]}),
    ("A10", "Punto en concesión de tercero", ["¿la coordenada 642000 9663600 wgs84 es nuestra?"],
     {"intencion": ["verificar_punto"], "contiene": ["tercero"], "reglas": ["R12"]}),
    ("A11", "Conversión PSAD56 -> WGS84", ["convierte E 642600 N 9662480 de PSAD56 a WGS84"],
     {"intencion": ["convertir_coordenadas"], "contiene": ["EPSG:32717", "642 348"]}),
    ("A12", "Conversión geográficas -> UTM", ["pasa -3.0512, -79.7301 a utm, está en wgs84"],
     {"intencion": ["convertir_coordenadas"], "contiene": ["E 641 120"]}),
    ("A13", "Conversión con dos ranuras", ["necesito convertir unas coordenadas", "645210 9663790", "psad56"],
     {"intencion": ["convertir_coordenadas"], "contiene": ["Origen: PSAD56"]},
     {0: {"accion": "solicitar_dato:coord"}, 1: {"accion": "solicitar_dato:datum"}}),
    ("A14", "Alerta con maquinaria y personas en concesión suspendida", ["hay una retroexcavadora y gente extraña trabajando en
Cerro Azul"],
     {"intencion": ["reportar_alerta"], "contiene": ["prioridad CRÍTICA"], "reglas": ["R21", "R41", "R42"]}),
    ("A15", "Alerta explicada por actividad autorizada (explaining away)", ["se ve desbroce nuevo y maquinaria en Peña Blanca"],
     {"intencion": ["reportar_alerta"], "contiene": ["prioridad MEDIA", "actividad autorizada programada explica"], "reglas":
["R23"]}),
    ("A16", "Alerta en tres turnos con ranuras", ["quiero reportar una invasión", "en Los Laureles", "el agua baja turbia"],
     {"intencion": ["reportar_alerta"], "contiene": ["prioridad ALTA", "supervisor"], "reglas": ["R22", "R41"]},
     {0: {"accion": "solicitar_dato:ubicacion"}, 1: {"accion": "solicitar_dato:evidencias"}}),
    ("A17", "Alerta en concesión de tercero", ["vimos maquinaria en E 642000 N 9663600 WGS84"],
     {"intencion": ["reportar_alerta"], "contiene": ["REVISIÓN LEGAL"], "reglas": ["R20", "R35"]}),
    ("A18", "Alerta crítica con ventana de vuelo al día siguiente", ["reporto voladuras de noche y una trocha nueva en El
Roble"],
     {"intencion": ["reportar_alerta"], "contiene": ["prioridad CRÍTICA", "Vuelo de verificación prioritario"], "reglas":
["R30"]}),
    ("A19", "Alerta con negación y zona sensible", ["hay maquinaria en La Ventana, pero no hay personas"],
     {"intencion": ["reportar_alerta"], "contiene": ["prioridad BAJA", "revisión humana"], "reglas": ["R24", "R40"]}),
    ("A20", "Emergencia con armas", ["hay gente armada disparando cerca del campamento"],
     {"intencion": ["emergencia_seguridad"], "contiene": ["ECU 911", "No se enviará personal ni dron"], "reglas": ["R01"]}),
    ("A21", "Vuelo con fecha no apta por viento", ["necesito un vuelo de dron en San Gregorio", "mañana"],
     {"intencion": ["solicitar_vuelo"], "contiene": ["no es apta", "viento", "2026-11-11"]}, {0: {"accion":
"solicitar_dato:fecha"}}),
    ("A22", "Vuelo el mismo día sin urgencia (anticipación)", ["programa un vuelo en Los Laureles para hoy"],
     {"intencion": ["solicitar_vuelo"], "contiene": ["anticipación", "2026-11-10"]}),
    ("A23", "Solicitud de KMZ", ["necesito el KMZ de los linderos de CM-102"],
     {"intencion": ["solicitar_cartografia"], "contiene": ["KMZ", "3 día"], "accion": "crear_ticket"}),
```

```
    ("A24", "Cartografía con ranuras", ["quiero pedir cartografía", "una ortofoto", "de El Roble"],
     {"intencion": ["solicitar_cartografia"], "contiene": ["ortofoto de CM-104"]},
     {0: {"accion": "solicitar_dato:producto"}, 1: {"accion": "solicitar_dato:concesion"}}),
    ("A25", "Estado de ticket existente", ["¿cómo va la solicitud GEO-2026-0412?"],
     {"intencion": ["estado_solicitud"], "contiene": ["En elaboración"]}),
    ("A26", "Ticket inexistente (no inventar)", ["estado del ticket GEO-2026-0999"],
     {"intencion": ["estado_solicitud"], "contiene": ["No encuentro"]}),
    ("A27", "FAQ de códigos EPSG", ["¿qué EPSG corresponde a PSAD56 en UTM 17 sur?"],
     {"intencion": ["consulta_geodesia"], "contiene": ["24877"]}),
    ("A28", "FAQ de desplazamiento entre datums", ["¿cuántos metros se mueve un punto entre PSAD56 y WGS84?"],
     {"intencion": ["consulta_geodesia"], "contiene": ["443 m"]}),
    ("A29", "Derivación a humano", ["quiero hablar con un analista de verdad"],
     {"intencion": ["hablar_humano"], "reglas": ["R03"], "accion": "derivar_humano"}),
    ("A30", "Fuera de alcance", ["¿quién ganó la Copa Libertadores?"],
     {"intencion": ["fuera_alcance"], "contiene_alguno": ["fuera de mi alcance", "No tengo suficiente seguridad"]}),
    ("A31", "Dato personal enmascarado", ["mi cédula es 3000000004, ¿cuál es el estado de CM-104?"],
     {"intencion": ["consultar_concesion"], "contiene": ["renovación"], "reglas": ["R02"]}),
    ("A32", "Mensaje con faltas ortográficas", ["ola, nesesito saver el estado de la consecion el robel"],
     {"intencion": ["consultar_concesion"], "contiene": ["El Roble"]}),
    ("A33", "Despedida", ["gracias, eso es todo"], {"intencion": ["despedida"]}),
    ("A34", "Coordenadas sin verbo (mensaje ambiguo)", ["642350 9662120"],
     {"intencion": ["verificar_punto", "convertir_coordenadas"], "accion_alguna": ["solicitar_dato:datum", None]}),
]


CONFIRMACION = [
    ("C01", "Alerta con coordenadas y dos evidencias", ["reporto maquinaria y una trocha nueva en E 646900 N 9660300 WGS84"],
     {"intencion": ["reportar_alerta"], "contiene": ["prioridad"]}),
    ("C02", "Teléfono enmascarado en una solicitud de cartografía",
     ["llámame al 0991234567 para coordinar, necesito un plano topográfico de CM-106"],
     {"intencion": ["solicitar_cartografia"], "reglas": ["R02"], "accion": "crear_ticket"}),
    ("C03", "Emergencia sin la palabra «emergencia»", ["auxilio, hay heridos en la vía al campamento sur"],
     {"intencion": ["emergencia_seguridad"], "reglas": ["R01"]}),
    ("C04", "Conversión desde grados, minutos y segundos", ["pasa 3°03'04\"S 79°43'48\"W a UTM, datum WGS84"],
     {"intencion": ["convertir_coordenadas"], "contiene": ["E 641"]}),
    ("C05", "Vuelo con día de la semana", ["agenda un vuelo sobre La Ventana el viernes"],
     {"intencion": ["solicitar_vuelo"], "contiene": ["2026-11-13"]}),
    ("C06", "FAQ formulada de otra manera", ["¿qué ventaja tiene el RTK frente al GPS navegador?"],
     {"intencion": ["consulta_geodesia"], "contiene": ["centímetros"]}),
    ("C07", "Clima fuera del contexto de vuelos", ["¿va a llover mañana en Machala?"],
     {"intencion": ["fuera_alcance"], "contiene_alguno": ["fuera de mi alcance", "No tengo suficiente seguridad"]}),
    ("C08", "Estado de ticket en registro informal", ["porfa el estatus del GEO-2026-0430"],
     {"intencion": ["estado_solicitud"], "contiene": ["En verificación de campo"]}),
]


def evaluar_turno(r, crit: dict) -> tuple[bool, list[str]]:
    fallos = []
    if "intencion" in crit and r.intencion not in crit["intencion"]:
        fallos.append(f"intención {r.intencion} ∉ {crit['intencion']}")
    if "accion" in crit and r.accion != crit["accion"]:
        fallos.append(f"acción {r.accion} ≠ {crit['accion']}")
    if "accion_alguna" in crit and r.accion not in crit["accion_alguna"] and r.decision != "aclarar":
        fallos.append(f"acción {r.accion} ∉ {crit['accion_alguna']}")
    for frag in crit.get("contiene", []):
        if frag.lower() not in r.texto.lower():
            fallos.append(f"falta «{frag}»")
    if "contiene_alguno" in crit and not any(f.lower() in r.texto.lower() for f in crit["contiene_alguno"]):
        fallos.append("no contiene ninguno de los fragmentos esperados")
    for regla in crit.get("reglas", []):
        if regla not in r.reglas:
            fallos.append(f"no disparó {regla}")
    if r.latencia_ms >= 1500:
        fallos.append(f"latencia {r.latencia_ms:.0f} ms")
    return not fallos, fallos


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--version", default="v1")
    ap.add_argument("--modelo", default=None, help="Carpeta del modelo NLU (por defecto, el más reciente)")
    ap.add_argument("--confirmacion", action="store_true", help="Ejecuta solo los 8 escenarios de confirmación")
    args = ap.parse_args()
    nlu = ClasificadorIntenciones(args.modelo)
    filas, aprobados, turnos_total = [], 0, 0
    for esc in (CONFIRMACION if args.confirmacion else ESCENARIOS):
        ident, desc, turnos, crit = esc[:4]
        inter = esc[4] if len(esc) > 4 else {}
        bot = KuriBot(nlu, BaseConocimiento())
        detalle, ok_esc = [], True
        for i, t in enumerate(turnos):
            r = bot.responder(t)
            turnos_total += 1
            c = crit if i == len(turnos) - 1 else inter.get(i)
            ok, fallos = evaluar_turno(r, c) if c else (True, [])
            ok_esc &= ok
            detalle.append({"turno": i + 1, "entrada": t, "respuesta": r.texto, "intencion": r.intencion,
                            "confianza": round(r.confianza or 0, 4), "decision": r.decision, "accion": r.accion,
                            "reglas": r.reglas, "p_intrusion": (r.bayes or {}).get("p_intrusion"), "ticket": r.ticket,
                            "latencia_ms": round(r.latencia_ms, 1), "evaluado": bool(c), "ok": ok, "fallos": fallos})
        aprobados += ok_esc
        filas.append({"id": ident, "descripcion": desc, "aprobado": ok_esc, "turnos": detalle})
        estado = "OK " if ok_esc else "FALLA"
        print(f"{estado} {ident} {desc}" + ("" if ok_esc else "  -> " + " | ".join(f for d in detalle for f in d["fallos"])))
    lat = sorted(d["latencia_ms"] for f in filas for d in f["turnos"])
    resumen = {"version": args.version, "modelo": nlu.nombre, "escenarios": len(filas), "aprobados": aprobados,
               "tasa": aprobados / len(filas), "turnos": turnos_total,
               "latencia_media_ms": sum(lat) / len(lat), "latencia_p95_ms": lat[int(0.95 * (len(lat) - 1))]}
```

```
    print(json.dumps(resumen, ensure_ascii=False, indent=1))
    salida = RAIZ / "resultados" / (f"confirmacion_{args.version}.json" if args.confirmacion else
f"aceptacion_{args.version}.json")
    salida.write_text(json.dumps({"resumen": resumen, "escenarios": filas}, ensure_ascii=False, indent=1, default=float),
encoding="utf-8")


if __name__ == "__main__":
    main()
```

D.17 Extracción de BETO desde el pipeline de spaCy (scripts/extraer_beto.py)

```
"""Extrae el codificador BETO del pipeline oficial de spaCy y lo guarda en formato Hugging Face.

Motivo: el entorno de desarrollo no tiene acceso a huggingface.co, pero sí a las versiones
publicadas de los modelos de spaCy en GitHub. El pipeline ``es_dep_news_trf`` 3.8.0 incluye
el transformer BETO (dccuchile/bert-base-spanish-wwm-cased; Cañete et al., 2020), ajustado
por Explosion durante el entrenamiento del analizador morfosintáctico de spaCy.

El script:
1. lee el archivo serializado ``transformer/model`` (msgpack de Thinc);
2. convierte los nombres de parámetros de curated-transformers a ``BertModel`` de HF
   (la proyección fusionada QKV se divide en query, key y value, en ese orden);
3. guarda config.json, vocab.txt, tokenizer y model.safetensors;
4. con ``--verificar`` compara las salidas contra la implementación original de
   curated-transformers 0.1.1 (diferencia absoluta máxima esperada < 1e-4).

Uso:
    pip install es_dep_news_trf-3.8.0-py3-none-any.whl --no-deps
    python scripts/extraer_beto.py --salida modelos/beto_base [--verificar RUTA_CT011]
"""
from __future__ import annotations

import argparse
import importlib.util
import io
import json
from pathlib import Path

import srsly
import torch


def ubicar_modelo_spacy() -> Path:
    spec = importlib.util.find_spec("es_dep_news_trf")
    if spec is None or spec.origin is None:
        raise SystemExit("Instale primero es_dep_news_trf-3.8.0 (GitHub explosion/spacy-models).")
    raiz = Path(spec.origin).parent
    candidatos = list(raiz.glob("es_dep_news_trf-*/transformer/model"))
    if not candidatos:
        raise SystemExit("No se encontró transformer/model dentro del paquete.")
    return candidatos[0]


def leer_pesos(ruta: Path) -> tuple[dict[str, torch.Tensor], list[str]]:
    datos = srsly.msgpack_loads(ruta.read_bytes())
    shim = srsly.msgpack_loads(datos["shims"][6][0])
    estado = torch.load(io.BytesIO(shim["state"]), map_location="cpu", weights_only=False)
    vocab = datos["attrs"][3]["wordpiece_processor"].decode("utf-8").split("\n")
    return estado, vocab


def convertir(estado: dict[str, torch.Tensor], n_capas: int = 12) -> dict[str, torch.Tensor]:
    pref = "curated_encoder."
    hf: dict[str, torch.Tensor] = {}
    emb = {
        "embeddings.word_embeddings.weight": "embeddings.word_embeddings.weight",
        "embeddings.token_type_embeddings.weight": "embeddings.token_type_embeddings.weight",
        "embeddings.position_embeddings.weight": "embeddings.position_embeddings.weight",
        "embeddings.LayerNorm.weight": "embeddings.layer_norm.weight",
        "embeddings.LayerNorm.bias": "embeddings.layer_norm.bias",
    }
    for destino, origen in emb.items():
        hf[destino] = estado[pref + origen]
    for i in range(n_capas):
        o = f"{pref}layers.{i}."
        d = f"encoder.layer.{i}."
        w_qkv, b_qkv = estado[o + "mha.input.weight"], estado[o + "mha.input.bias"]
        for nombre, w, b in zip(("query", "key", "value"), w_qkv.chunk(3, dim=0), b_qkv.chunk(3, dim=0)):
            hf[d + f"attention.self.{nombre}.weight"] = w.clone()
            hf[d + f"attention.self.{nombre}.bias"] = b.clone()
        pares = {
            "attention.output.dense": "mha.output",
            "attention.output.LayerNorm": "attn_output_layernorm",
            "intermediate.dense": "ffn.intermediate",
            "output.dense": "ffn.output",
            "output.LayerNorm": "ffn_output_layernorm",
        }
        for destino, origen in pares.items():
            hf[d + destino + ".weight"] = estado[o + origen + ".weight"]
            hf[d + destino + ".bias"] = estado[o + origen + ".bias"]
    return hf


def guardar(hf_estado: dict[str, torch.Tensor], vocab: list[str], salida: Path) -> None:
    from transformers import BertConfig, BertModel, BertTokenizerFast
```

```
    salida.mkdir(parents=True, exist_ok=True)
    (salida / "vocab.txt").write_text("\n".join(vocab) + "\n", encoding="utf-8")
    config = BertConfig(
        vocab_size=len(vocab), hidden_size=768, num_hidden_layers=12, num_attention_heads=12,
        intermediate_size=3072, hidden_act="gelu", hidden_dropout_prob=0.1,
        attention_probs_dropout_prob=0.1, max_position_embeddings=512, type_vocab_size=2,
        layer_norm_eps=1e-12, pad_token_id=vocab.index("[PAD]"),
    )
    modelo = BertModel(config, add_pooling_layer=False)
    faltan, sobran = modelo.load_state_dict(hf_estado, strict=False)
    faltan = [k for k in faltan if "position_ids" not in k]
    if faltan or sobran:
        raise RuntimeError(f"Conversión incompleta. Faltan: {faltan}; sobran: {sobran}")
    modelo.save_pretrained(salida)
    import inspect
    opciones = dict(do_lower_case=False, strip_accents=False, tokenize_chinese_chars=True)
    if "vocab" in inspect.signature(BertTokenizerFast.__init__).parameters:   # transformers >= 5
        tok = BertTokenizerFast(vocab={p: i for i, p in enumerate(vocab)}, **opciones)
    else:                                                                     # transformers 4.x
        tok = BertTokenizerFast(vocab_file=str(salida / "vocab.txt"), **opciones)
    prueba = tok("Hay maquinaria cerca del lindero de la concesión")["input_ids"]
    if tok.unk_token_id in prueba or tok.cls_token_id != vocab.index("[CLS]"):
        raise RuntimeError("El tokenizador no reproduce el vocabulario de BETO.")
    tok.save_pretrained(salida)
    (salida / "PROCEDENCIA.json").write_text(json.dumps({
        "origen": "spaCy es_dep_news_trf 3.8.0 (Explosion), componente transformer",
        "modelo_base": "dccuchile/bert-base-spanish-wwm-cased (BETO; Cañete et al., 2020)",
        "nota": "Pesos ajustados por spaCy para análisis morfosintáctico (UD AnCora); "
                "no son idénticos al BETO original publicado en Hugging Face.",
        "licencia_pipeline": "GNU GPL 3.0 (por los datos AnCora del pipeline)",
    }, ensure_ascii=False, indent=2), encoding="utf-8")


def verificar(estado, salida: Path, ruta_ct011: str) -> float:
    """Compara la salida HF con la implementación original curated-transformers 0.1.1."""
    import sys
    from transformers import BertModel, BertTokenizerFast

    sys.path.insert(0, ruta_ct011)
    from curated_transformers.models.bert.config import BertConfig as CTConfig  # type: ignore
    from curated_transformers.models.bert.encoder import BertEncoder  # type: ignore
    from curated_transformers.models.attention import AttentionMask  # type: ignore

    cfg = CTConfig(vocab_size=31002, hidden_width=768, intermediate_width=3072,
                   num_attention_heads=12, num_hidden_layers=12, hidden_act="gelu",
                   layer_norm_eps=1e-12, max_position_embeddings=512, type_vocab_size=2,
                   padding_idx=1, attention_probs_dropout_prob=0.0, hidden_dropout_prob=0.0)
    ct = BertEncoder(cfg)
    ct.load_state_dict({k.replace("curated_encoder.", ""): v for k, v in estado.items()})
    ct.eval()
    hf = BertModel.from_pretrained(salida, add_pooling_layer=False).eval()
    tok = BertTokenizerFast.from_pretrained(salida)
    frases = ["Necesito convertir coordenadas de PSAD56 a WGS84.",
              "Hay maquinaria no autorizada cerca del lindero de la concesión."]
    enc = tok(frases, padding=True, return_tensors="pt")
    with torch.no_grad():
        salida_hf = hf(**enc).last_hidden_state
        mascara = AttentionMask(enc["attention_mask"].bool())
        salida_ct = ct(enc["input_ids"], mascara, token_type_ids=enc["token_type_ids"])
        ultimo = salida_ct.last_hidden_layer_states
        salida_ct = ultimo() if callable(ultimo) else ultimo
    m = enc["attention_mask"].bool()
    return float((salida_hf[m] - salida_ct[m]).abs().max())


def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--salida", default="modelos/beto_base")
    ap.add_argument("--verificar", default=None, help="Ruta a curated-transformers 0.1.1 instalado con --target")
    args = ap.parse_args()
    estado, vocab = leer_pesos(ubicar_modelo_spacy())
    hf_estado = convertir(estado)
    guardar(hf_estado, vocab, Path(args.salida))
    print(f"Modelo guardado en {args.salida} ({len(vocab)} piezas de vocabulario).")
    if args.verificar:
        dif = verificar(estado, Path(args.salida), args.verificar)
        print(f"Diferencia absoluta máxima HF vs curated-transformers: {dif:.2e}")
        if dif > 1e-3:
            raise SystemExit("La conversión no reproduce la salida original.")


if __name__ == "__main__":
    main()
```

D.18 Reconstrucción del modelo ajustado desde el repositorio (scripts/unir_modelo.py)

```
"""Reconstruye el modelo BETO ajustado a partir de las partes publicadas en el repositorio.

GitHub no admite archivos de más de 100 MB, por lo que KuriBot_Modelo_BETO_v2.zip (≈ 200 MB) se guarda en cinco
partes en modelos/partes_beto_v2/. Este script las une, verifica la suma SHA-256 de cada parte y del archivo
completo, y descomprime el resultado en modelos/intenciones_beto_v2/.

Uso (Windows, macOS o Linux): python scripts/unir_modelo.py
"""
from __future__ import annotations

import hashlib
```

```
import sys
import zipfile
from pathlib import Path

RAIZ = Path(__file__).resolve().parents[1]
PARTES = RAIZ / "modelos" / "partes_beto_v2"
DESTINO = RAIZ / "modelos" / "intenciones_beto_v2"
NOMBRE = "KuriBot_Modelo_BETO_v2.zip"


def sha256(datos: bytes) -> str:
    return hashlib.sha256(datos).hexdigest()


def main() -> None:
    if (DESTINO / "clasificador.safetensors").exists():
        print(f"El modelo ya existe en {DESTINO.relative_to(RAIZ)}; no se hace nada.")
        return
    esperadas = {}
    for linea in (PARTES / "SHA256SUMS").read_text(encoding="utf-8").splitlines():
        if linea.strip():
            suma, nombre = linea.split()
            esperadas[nombre] = suma
    partes = sorted(PARTES.glob(f"{NOMBRE}.part*"), key=lambda p: int(p.suffix.replace(".part", "")))
    if not partes:
        sys.exit(f"No se encontraron partes en {PARTES}")
    contenido = bytearray()
    for p in partes:
        datos = p.read_bytes()
        if sha256(datos) != esperadas.get(p.name):
            sys.exit(f"La parte {p.name} está incompleta o dañada (SHA-256 distinta). Vuelva a descargarla.")
        contenido.extend(datos)
    if sha256(bytes(contenido)) != esperadas[NOMBRE]:
        sys.exit("El archivo reconstruido no coincide con la suma SHA-256 publicada.")
    zip_tmp = PARTES / NOMBRE
    zip_tmp.write_bytes(contenido)
    with zipfile.ZipFile(zip_tmp) as z:
        z.extractall(RAIZ)
    zip_tmp.unlink()
    print(f"Modelo reconstruido y verificado en {DESTINO.relative_to(RAIZ)} ({len(partes)} partes, {len(contenido) / 1e6:.0f}
MB).")


if __name__ == "__main__":
    main()
```